



Turnkey Key Management System

Security Assessment and Fix Review

June 16, 2026

Prepared for:

Jack Kearney

Turnkey

Prepared by: **Artur Cygan, Paweł Płatek, and Thomas Chauchefoin**

Table of Contents

Table of Contents	1
Project Summary	4
Executive Summary	5
Project Goals	11
Project Targets	13
Project Coverage	14
Summary of Findings	15
Detailed Findings	20
1. Nested policy expressions can overflow available stack space	20
2. Parser enclave uses local filesystem	23
3. Incorrect Organization encoding for signature verification	25
4. Notarizer does not validate smart_contract_address from user intent versus smart_contract_address from the context	26
5. Notarizer signs smart contract interfaces from Parser context instead of user intent	27
6. Weak validation of parsed Solana IDLs against notarized organization data	30
7. Turnkey accepts ABIs/IDLs with colliding functions	34
8. ConditionValidator membership operator has exponential time complexity	36
9. Integer overflow in constrained vector allocation method may lead to panics in the Parser enclave	39
10. Solana transactions with repeated accounts are accepted	42
11. Organization policies may misinterpret WSOL transfers	44
12. Weak validation of SmartContractInterfaceType in enclaves	46
13. Solana dynamic address lookups may hinder policies	49
14. Solana programs cannot be resolved via dynamic address lookup tables	51
15. Uploading Solana IDLs with colliding addresses is allowed	52
16. Solana transaction metadata for policies does not include transfers made by calls to programs	54
17. Solana transaction metadata for policies does not include transfers made by some methods of supported system programs	55
18. Incomplete mitigation of user existence oracles in coordinator	57
19. Discrepancy between Parser and Signer transaction parsing	59
20. Inconsistent enforcement of AllowedAuthMethods	61
21. Desynchronization between Ump intents allowlist and Auth Proxy configuration	

63	
22. Incomplete mitigation of user existence oracles via Auth Proxy signup	65
23. Defined types in IDLs can overflow available stack space	68
24. Parser enclave bug allows anything to be signed as a Solana IDL	71
25. ValidateSmartContractInterface does not validate addresses	73
26. Ethereum int256 type is not supported	74
27. System account Auth Proxy user can be made root user	76
28. Potential race condition while disabling the Auth Proxy feature	79
29. Use of unverified signed tokens	81
30. Verbose technical error messages	84
31. ACTIVITY_TYPE_UPDATE_AUTH_PROXY_CONFIG does not restrict Enterprise-only features	86
32. Unreachable SQL injection in GetKeyAndWalletCountInRangeWithInterval	87
33. Lack of network isolation for Auth Proxy pods	89
34. OIDC ID tokens can be replayed on other organizations because audience claims are not validated	91
35. Journalized OTP verification tokens can be front run	93
36. Verification tokens can be replayed outside of signup and login endpoints	95
37. Segwit bitcoin transactions vulnerable to fee overpayment attack	97
38. Missing validations defined in BIP-0174	99
39. Supported versions of PSBTs are not validated explicitly and are not documented	101
40. Lack of test vectors for BIP-0174 in the code	102
41. Users can add invalid public keys	103
42. Valid Solana IDLs are not accepted due to "types" key requirement	104
43. Weak validation of Tron transactions	105
44. Bitcoin address type is not case insensitive in Policy Engine	106
45. Risk of mix-up attacks on ciphertext in Auth Proxy	107
46. Auth Proxy does not validate enclave signatures	109
47. Pre-production environment is incorrectly set up in the export-and-sign iframe	110
48. The decryptedKey is not cleared from memory after expiration	111
49. The decryptedKey is not removed from DOM after expiration and clear events	112
50. The decryptedKey is always displayed and is not masked on the UI level	113
51. Expiration time of decryptedKey is incorrectly calculated	114

52. Redundant event listeners in export-and-sign	115
54. Single client website opened in multiple tabs cannot be used with the export-and-sign iframe	116
55. The fromDerSignature function in the export-and-sign iframe is missing validations	117
56. Possible mitigations for supply chain attacks are not implemented	118
58. Dependencies of the export-and-sign iframe are not minimal	119
59. Signer does not validate the number of private keys to be created	120
60. Signer crashes on empty transaction unsigned_payload	121
61. Signer leaks malformed decrypted mnemonics	123
A. Vulnerability Categories	125
B. Code Quality Findings	127
C. Fuzzing Recommendations	130
Rust Code	130
Go Code	135
D. Open-Sourcing of the Signer Enclave	136
E. Fix Review Results	140
Detailed Fix Review Results	145
F. Fix Review Status Categories	153
About Trail of Bits	154
Notices and Remarks	155

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Keith Hoodlet, Engineering Director, AI/ML and Application Security
keith.hoodlet@trailofbits.com

The following consultants were associated with this project:

Paweł Płatek, Consultant **Artur Cygan**, Consultant
pawel.platek@trailofbits.com artur.cygan@trailofbits.com

Thomas Chauchefoin, Consultant
thomas.chauchefoin@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
October 15, 2025	Pre-project kickoff call
October 27, 2025	Status update meeting #1
October 30, 2025	Technical discussion
November 5, 2025	Status update meeting #2
November 10, 2025	Status update meeting #3
November 17, 2025	Delivery of report draft
November 17, 2025	Report readout meeting
February 18, 2026	Delivery of report draft with fix review appendix
March 24, 2026	Delivery of final comprehensive report
June 16, 2026	Delivery of final comprehensive report with updated fix review

Executive Summary

Engagement Overview

Turnkey engaged Trail of Bits to review the security of the Turnkey key management system.

The system consists of a few trust zones: AWS Nitro enclaves, the backend infrastructure (mostly the Coordinator), the Auth Proxy application, and client-side components (Turnkey iframes, the SDK, the website). The end goal of the system is to provide end-to-end authentication via means of remote attestation of a trusted execution environment, and cryptographic authentication and encryption of communication and data at rest.

A team of three consultants conducted the review from October 20 to November 14, 2025, for a total of eight engineer-weeks of effort. Our testing efforts focused on seven distinct areas:

- Review and fuzzing of the Parser enclave code for the Solana, Bitcoin, and Tron chains, focusing on identifying parsing logic bugs and enclave panics
- Review of the Ethereum smart contract interfaces and Solana IDL uploading code
- Review of the cryptographic properties of newly added features
- Review and fuzzing of the Policy Engine, focusing on identifying logic bugs and enclave panics
- High-level review of the new Auth Proxy component
- Study of the risks related to open-sourcing the Signer enclave code
- Review of browser-based signing code (the new export-and-sign iframe)

With full access to source code and documentation, we performed static and dynamic testing of the components in scope, using automated and manual processes. For existing automated testing processes (e.g., unit tests, fuzzing), we reviewed their implementation and extended them for our own testing.

Observations and Impact

The reviewed components from the Turnkey codebase are robust and thoroughly tested through various dynamic methods (end-to-end tests, unit tests, fuzzing), requiring a manual review to uncover most of this report's findings.

We identified a large number of findings during this audit, most of which are rated as informational severity; see [appendix A](#) for details on the criteria used to assess the severity and difficulty of findings.

The findings related to the transaction parsing functionalities are not high in severity, but they confirm the need for further work in this area to achieve parity with current standards and third-party parsers. Moreover, we identified edge cases that cannot be addressed by code fixes, indicating the need to improve the documentation available for Turnkey users to enable them to operate the system securely.

In addition, we identified several easily addressable issues related to the architecture and implementation of the new Auth Proxy feature, which may indicate that the feature has not yet reached the same level of maturity as other components.

The export-and-sign iframe inherits the same security properties and risks as other iframes (which were previously audited by Trail of Bits). This specific instance of an iframe was found to be vulnerable to a few, mostly low-severity, issues. Strategic recommendations that may further improve the iframe's security are presented in the Recommendations section below.

We uncovered three findings in the Signer enclave, one of which concerns sensitive information leakage, and another a denial-of-service opportunity. While both of these issues are of high difficulty and are unlikely to be exploitable at the moment, they indicate room for improvement in the enclave's security. In addition to these findings, we found that while the code is in a good condition, there are many minor details that can be improved; these are documented in [appendix D](#). Finally, the in-repository documentation and build transparency needs to be polished before the open-source release.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Turnkey take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.

Transaction Parsing Recommendations

- **Add sandboxes to the Parser and Policy enclaves.** Reduce the impact of pathological policies by validating and evaluating them in separate processes for better isolation and enforcement of resource quotas ([TOB-TK3-1](#), [TOB-TK3-8](#), [TOB-TK3-9](#), [TOB-TK3-23](#)). Such a change would cause enclave panics to simply kill Unix processes instead of pausing the whole enclave, greatly limiting denial-of-service opportunities for external attackers. .
- **Review functionalities related to Ethereum transaction parsing, smart contract uploading, and Ethereum policies against findings from this report.** Specifically, the following findings related to Solana should be used for cross-validation against similar Ethereum features: [TOB-TK3-4](#), [TOB-TK3-5](#), [TOB-TK3-12](#), [TOB-TK3-14](#), [TOB-TK3-15](#).

- **Continue reviewing user' use cases of the Turnkey system and improving the public documentation.** Some edge cases, especially during writing of organization policies, cannot be fully mitigated by changes to the enclaves. Users should be made aware of the possible footguns and guided by Turnkey to write secure policies (TOB-TK3-15, TOB-TK3-16, TOB-TK3-17, TOB-TK3-26, TOB-TK3-44).
- **Review and resolve recommendations from the previous audit's Summary of Recommendations section.** In particular, the "formalize cryptographic protocols created for the Turnkey system" item should be reconsidered, as the new functionalities keep increasing the complexity of the system from a cryptographic perspective (TOB-TK3-5, TOB-TK3-6, TOB-TK3-12, TOB-TK3-46).

Auth Proxy Recommendations

- **Continue to decouple the Auth Proxy feature from the rest of the platform.** From a threat modeling perspective, end users and Auth Proxy instances should be placed within the same trust boundary, like how backends operated by users already are. While a perfect decoupling is not possible, because the Auth Proxy is and must be operated by Turnkey, isolating the Auth Proxy from the rest of the infrastructure is advisable. This would limit the impact both of Auth Proxy compromise and of malicious insiders trying to attack users via the Auth Proxy. The proposed isolation could be implemented on the network level, the authentication and access-control level, and the operational level (for example, by granting different sets of Turnkey employees code access or other privileges for external services to build and deploy the components). Reduce custom code in enclave components that was added only to support this feature. See TOB-TK3-28 and TOB-TK3-33.
- **Reinforce the isolation of organizations and users on the Auth Proxy, in particular against malicious organization administrators.** The Auth Proxy is a multitenant service on which organization administrators (and sometimes members) have full visibility over the authentication process of users. Mitigating the risks of this model requires breaking changes on the OTP authentication flow and safer handling of signed tokens (TOB-TK3-29, TOB-TK3-34, TOB-TK3-35, TOB-TK3-36, TOB-TK3-45).
- **Formalize and publicly document anti-abuse mechanisms enforced by the Auth Proxy.** With the Auth Proxy being maintained and operated by Turnkey, most anti-abuse mechanisms that had to be implemented by users' backends are now Turnkey's responsibility. While some of them are implemented downstream in enclave code (i.e., OTP rate limiting), the public documentation should cover the ones implemented by the Auth Proxy so they can be evaluated in future audits.

Export-and-Sign IFrame Recommendations

- **Remove fallback (old) communication methods between the iframe and its parent website.** The new communication via ports is more secure, and supporting old methods makes downgrade attacks possible. It is likely that the fallbacks exist to support users that have not yet updated their code. Deprecation and a plan for eventual removal of the fallbacks is recommended.
- **Remove the stand-alone version of the iframe.** Since the export-and-sign iframe is supposed to be loaded as an iframe, and is not supposed to be accessed directly at <https://export-and-sign.turnkey.com>, removing the stand-alone code paths would reduce the attack surface. If the stand-alone version is important for debugging purposes, then it should be available only in debug builds, not in production builds.
- **Instruct users to write their websites to open export-and-sign iframes on-demand.** Instead of the parent (user's) website keeping the iframe open for the lifetime of the website, it is more secure to make the parent website open a new iframe instance only when it is needed and close it when no longer needed. This secure behavior is currently implemented in the [SDK examples](#).
- **Stay updated on the development of the Web Application Integrity, Consistency, and Transparency and Web-Based Code Assurance and Transparency standards.** The standards are works in progress at the time this report was written. Once they are available, they may greatly improve security of the Turnkey iframes on the threat-model level. These would be the main benefits:
 - Integrity and consistency: Compromised Turnkey servers would be able to serve malicious iframes only to all users or to none of them; they would not be able to serve iframes to only a few selected victims. Along with providing transparency, this mechanism would minimize the chance of successful attacks against users.
 - Authenticity and provenance: Users would be able to cryptographically verify the iframe's code.
- **Discuss possible ways to protect private keys in the iframe from a parent website compromised via attacks like cross-site scripting.** The main idea is to make the export-and-sign iframe require user interaction before it produces a signature. This mitigation would not prevent the parent website from requesting signatures, but could effectively rate-limit the attacker.
- **Consider allowing users to host their own instances of the export-and-sign iframes.** This would allow them to customize their iframes to accept communication only from specific origins. Moreover, this would minimize the attack surface of the iframe: using the global iframe at export-and-sign.turnkey.com adds an attack

vector—the export-and-sign.turnkey.com server. Compromise of the single server hosting the iframe may lead to compromise of all private keys imported by all users using the iframe. We do not believe this attack vector can be mitigated. Allowing users to host their own instances of the iframe would minimize the impact of a successful server compromise.

- **If the specific users that will use the export-and-sign frame are already known to Turnkey, then consider implementing an allowlist of origins in the iframe.** To implement this feature, add a database with known, allowed user origins to the server hosting the export-and-sign.turnkey.com domain. The server could then render the iframe with the allowlist of origins, and the iframe could validate the sources of messages against that list. Moreover, the iframe could use the allowlist in the `frame-ancestors` CSP directive, and the iframe could be hosted on unique subdomains for every user. These new protections would not prevent specific known attacks but would provide defense in depth.
- **Unify the code between all iframes by extracting shared code into reusable libraries.** There is a lot of code repetition between export-and-sign and other iframes, like the import and export iframes. The code should be consolidated to improve maintenance, make security audits more efficient, and ensure that security patches and improvements implemented in one iframe are propagated to other iframes.
- **Add subresource integrity hashes to all loaded resources, even if the resources are first party.** The generated HTML page of the export-and-sign iframe references a few resources (JavaScript files, a CSS file). Subresource integrity can be added to the resources to prevent them from being tampered with. The resources are in the same trust zone as the main HTML page. However, if validation of the iframe integrity becomes possible, subresource integrity will be critical for the solution to be sound.

Signer Enclave Open-Sourcing Recommendations

- **Revamp the in-repository documentation with a focus on build transparency.** Review and document any nonobvious design decisions that can be useful while reviewing the code.
- **Review the code for issues similar to findings [TOB-TK3-59](#), [TOB-TK3-60](#), and [TOB-TK3-61](#).**
- **Review [appendix D](#) for operational and quality recommendations before open-sourcing the Signer enclave.**

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	1
Medium	20
Low	9
Informational	27
Undetermined	2

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Access Controls	9
Authentication	2
Configuration	1
Cryptography	5
Data Exposure	4
Data Validation	21
Denial of Service	7
Error Reporting	3
Patching	2
Testing	1
Timing	1
Undefined Behavior	3

Project Goals

The engagement was scoped to provide a security assessment of the Turnkey key management system. Specifically, we sought to answer the following non-exhaustive list of questions:

- Transaction parsing
 - Can a malicious input to the Parser or Ump enclave make the code panic?
 - Are Solana, Bitcoin, and Tron transactions parsed correctly?
 - Are there discrepancies between the transaction metadata as parsed by enclaves and interpreted on-chain?
 - Do policies behave as users would expect them to, with regard to their general logic and expressiveness?
- Smart contract interfaces
 - Are transactions parsed correctly given user-provided interfaces?
 - Can a malicious smart contract interface cause the Ump or Parser enclave to panic?
- Cryptography
 - Is communication between enclaves and between users and enclaves correctly authenticated via cryptographic means?
 - Are mix-up attacks possible against newly added, signed data?
- Auth Proxy
 - Are there dangerous configuration settings that could hinder the security posture of users using the Auth Proxy?
 - Are the various authentication methods implemented safely to prevent impersonation of suborganizations?
 - What could attackers achieve with a compromised Auth Proxy instance?
- Browser-based signing
 - Does the implementation of the export-and-sign iframe contain technical vulnerabilities?
 - Can the new iframe be improved with new security mitigations?
- Open-sourcing of the Signer enclave code

- What are the risks associated with open-sourcing the Signer code?
- Are there outstanding, low-hanging bugs in the Signer code?

Project Targets

The engagement involved reviewing and testing the following target.

mono

Repository	https://github.com/tkhq/mono
Version	7a2d26db7fb791552cb3a5b26eff82783b19d6e5
Type	Rust
Platform	Unix, AWS Nitro Enclaves

frames

Repository	https://github.com/tkhq/frames
Version	588e0619030764b2cd56703aae16468aefc98261
Type	JavaScript
Platform	HTML

core-signer-enclave

Repository	https://github.com/tkhq/core-signer-enclave
Version	318656421d2aeea4eff5bcd753b1cfb7eaa928d2 (v2025.10.6)
Type	Rust
Platform	Unix, AWS Nitro Enclaves

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Manual review and dynamic testing of the Policy Engine and Parser crates. The functionalities related to Solana, Tron, and Bitcoin transaction parsing were reviewed against logic issues and possible panics. The new smart contract notarized data (ABI/IDL uploading) code was reviewed for similar issues and for vulnerabilities on the cryptographic protocol level. The EIP-7702 and EIP-712 crates were reviewed as well.
- Review of current fuzzing practices. We provide recommendations for improving the existing harnesses and developing new ones in [appendix C](#).
- Manual review and dynamic testing of the Auth Proxy Go components, as well as minimal fuzzing of low-risk functions to verify their correctness. The audit focused on identifying high-level design issues and risks for the component. Specific bugs in the code were identified only as a byproduct of the review.
- Manual review of the browser-based signing feature (the new export-and-sign iframe).
- A review of the Signer enclave for any remaining issues before it is open-sourced.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- Per Turnkey's request, review of dependencies (to detect unmaintained and outdated dependencies, for example) was excluded from the scope of this review.
- Our review of the EIP-7702 and EIP-712 crates did not include consideration of hyperliquid-specific risks.
- A deep, technical code review of the auth-proxy code was not performed.
- The export-and-sign iframe was not audited against issues that were reported in previous reports. Notably, parent-iframe communication channels and event listener configurations were not audited. Moreover, the high-level threat model of the new iframe was only discussed offline, and this report contains only technical bugs and design recommendations, no broader discussion on the threat model.
- The wallet crate was not sufficiently reviewed before open-sourcing.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Nested policy expressions can overflow available stack space	Denial of Service	Medium
2	Parser enclave uses local filesystem	Undefined Behavior	Informational
3	Incorrect Organization encoding for signature verification	Cryptography	Medium
4	Notarizer does not validate smart_contract_address from user intent versus smart_contract_address from the context	Data Validation	Medium
5	Notarizer signs smart contract interfaces from Parser context instead of user intent	Data Validation	Medium
6	Weak validation of parsed Solana IDLs against notarized organization data	Data Validation	Medium
7	Turnkey accepts ABIs/IDLs with colliding functions	Data Validation	Medium
8	ConditionValidator membership operator has exponential time complexity	Denial of Service	Medium
9	Integer overflow in constrained vector allocation method may lead to panics in the Parser enclave	Denial of Service	Medium
10	Solana transactions with repeated accounts are accepted	Data Validation	Informational
11	Organization policies may misinterpret WSOL transfers	Access Controls	Medium

12	Weak validation of SmartContractInterfaceType in enclaves	Data Validation	Medium
13	Solana dynamic address lookups may hinder policies	Access Controls	Medium
14	Solana programs cannot be resolved via dynamic address lookup tables	Denial of Service	Informational
15	Uploading Solana IDLs with colliding addresses is allowed	Data Validation	Medium
16	Solana transaction metadata for policies does not include transfers made by calls to programs	Access Controls	Low
17	Solana transaction metadata for policies does not include transfers made by some methods of supported system programs	Access Controls	Medium
18	Incomplete mitigation of user existence oracles in coordinator	Error Reporting	Informational
19	Discrepancy between Parser and Signer transaction parsing	Denial of Service	Informational
20	Inconsistent enforcement of AllowedAuthMethods	Access Controls	Medium
21	Desynchronization between Ump intents allowlist and Auth Proxy configuration	Access Controls	Medium
22	Incomplete mitigation of user existence oracles via Auth Proxy signup	Error Reporting	Informational
23	Defined types in IDLs can overflow available stack space	Denial of Service	Medium
24	Parser enclave bug allows anything to be signed as a Solana IDL	Data Validation	Medium

25	ValidateSmartContractInterface does not validate addresses	Data Validation	Informational
26	Ethereum int256 type is not supported	Denial of Service	Low
27	System account Auth Proxy user can be made root user	Access Controls	Undetermined
28	Potential race condition while disabling the Auth Proxy feature	Timing	Low
29	Use of unverified signed tokens	Cryptography	Low
30	Verbose technical error messages	Error Reporting	Informational
31	ACTIVITY_TYPE_UPDATE_AUTH_PROXY_CONFIG does not restrict Enterprise-only features	Access Controls	Informational
32	Unreachable SQL injection in GetKeyAndWalletCountInRangeWithInterval	Data Validation	Informational
33	Lack of network isolation for Auth Proxy pods	Configuration	Informational
34	OIDC ID tokens can be replayed on other organizations because audience claims are not validated	Authentication	Undetermined
35	Journalized OTP verification tokens can be front run	Authentication	Medium
36	Verification tokens can be replayed outside of signup and login endpoints	Access Controls	Informational
37	Segwit bitcoin transactions vulnerable to fee overpayment attack	Data Validation	Medium
38	Missing validations defined in BIP-0174	Data Validation	Informational

39	Supported versions of PSBTs are not validated explicitly and are not documented	Undefined Behavior	Informational
40	Lack of test vectors for BIP-0174 in the code	Testing	Informational
41	Users can add invalid public keys	Cryptography	Informational
42	Valid Solana IDLs are not accepted due to "types" key requirement	Data Validation	Informational
43	Weak validation of Tron transactions	Data Validation	Informational
44	Bitcoin address type is not case insensitive in Policy Engine	Data Validation	Informational
45	Risk of mix-up attacks on ciphertext in Auth Proxy	Cryptography	Low
46	Auth Proxy does not validate enclave signatures	Cryptography	Low
47	Pre-production environment is incorrectly set up in the export-and-sign iframe	Data Validation	Informational
48	The decryptedKey is not cleared from memory after expiration	Data Exposure	Low
49	The decryptedKey is not removed from DOM after expiration and clear events	Data Exposure	Low
50	The decryptedKey is always displayed and is not masked on the UI level	Data Exposure	Low
51	Expiration time of decryptedKey is incorrectly calculated	Data Validation	Informational
52	Redundant event listeners in export-and-sign	Data Validation	Informational
54	Single client website opened in multiple tabs cannot be used with the export-and-sign iframe	Undefined Behavior	Informational

55	The fromDerSignature function in the export-and-sign iframe is missing validations	Data Validation	Informational
56	Possible mitigations for supply chain attacks are not implemented	Patching	Informational
58	Dependencies of the export-and-sign iframe are not minimal	Patching	Informational
59	Signer does not validate the number of private keys to be created	Data Validation	Informational
60	Signer crashes on empty transaction unsigned_payload	Data Validation	Medium
61	Signer leaks malformed decrypted mnemonics	Data Exposure	High

Detailed Findings

1. Nested policy expressions can overflow available stack space

Severity: Medium

Difficulty: Low

Type: Denial of Service

Finding ID: TOB-TK3-1

Target: `mono/src/rust/policy/src/evaluator/traits.rs`

Description

The Policy engine implements a recursive evaluation algorithm without setting strict depth limits. An attacker can craft deeply nested expressions to exhaust the available stack space, ultimately causing the engine to crash. For instance, 35 nested accesses were found to be enough to trigger a stack overflow in our testing environment (8 MiB stack size, Ubuntu 24.04 LTS on x86_64).

In the code snippet in figure 1.1, each expression type case calls into the specialized evaluation implementation—for instance, `self.evaluate_access()` for `Expr::Access` nodes.

```
fn evaluate(&mut self, expr: Box<Expr>) -> Result<Value, EvaluatorError> {
    let value = match *expr {
        Expr::Function(function) => self.evaluate_fn(function)?,
        Expr::List(list) => self.evaluate_list(list)?,
        Expr::Access(left, op) => self.evaluate_access(left, op)?,
        Expr::Logical(left, op, right) => self.evaluate_logical(left, op, right)?,
        Expr::Comparison(left, op, right) => self.evaluate_comparison(left, op,
right)?,
        Expr::Unary(op, right) => self.evaluate_unary(op, right)?,
        Expr::Literal(literal) => self.evaluate_literal(literal)?,
        Expr::Ident(ident) => self.evaluate_ident(ident)?,
    };
    // [...]
}
```

Figure 1.1: Main dispatcher of the evaluator
([mono/src/rust/policy/src/evaluator/traits.rs#L57-L75](#))

In turn, most of these handlers must also evaluate the target expression. Keeping with the example of `Expr::Access` nodes, it immediately calls `evaluate()` on its left member (figure 1.2).

```

/// evaluate an access operation
fn evaluate_access(&mut self, left: Box<Expr>, op: AccessOp) -> Result<Value,
EvaluatorError> {
    let left = self.evaluate(left)?;
    // [...]
    Ok(value)
}

```

Figure 1.2: Handler for Expr::Access nodes
([mono/src/rust/policy/src/evaluator/traits.rs#L271-L335](#))

The following handlers recursively call `evaluate()` and are affected by this issue:

- Expr::Function / `evaluate_fn()`, evaluating left and/or right expressions, depending on the operation
- Expr::List / `evaluate_list()`, evaluating each list item
- Expr::Access / `evaluate_access`, evaluating its left expression
- Expr::Logical / `evaluate_logical()`, evaluating both left and right expressions
- Expr::Comparison / `evaluate_comparison()`, evaluating both left and right expressions
- Expr::Unary / `evaluate_unary()`, evaluating its right expression

Exploit Scenario

An attacker crafts a malicious policy with deeply nested expressions, such as field accesses (e.g., `x.field.field.[...].field`) and submits it to the policy evaluation system for validation or evaluation. The recursive nature of the policy evaluator causes it to overflow its stack and the Rust runtime to abort the process with a runtime error.

An attacker can attempt to trigger the crash on many enclave instances in parallel. While mitigations like auto-scaling and restarts may limit the impact, a sustained attack could still cause service degradation for other Turnkey users.

Recommendations

Short term, implement a recursion depth counter in the Evaluator trait by adding a depth parameter that is incremented on entry, and impose a reasonable threshold that causes the evaluation to stop when exceeded. The threshold should be determined based on the stack limit configured in the enclave. Note that this may affect existing policies deployed by users.

This issue was found through fuzzing. See [appendix C](#) for more recommendations on improving the fuzzing setup to trigger this bug more easily.

Long term, refactor the evaluator to use an iterative algorithm, which would eliminate this risk entirely. Implement further validation passes to reject policies that exceed complexity thresholds during the parsing phase, thereby preventing malicious policies from reaching the evaluation stage. This would prevent similar recursion issues from being introduced in future code modifications.

Additionally, process policies in an isolated process with strong resource constraints (CPU, memory, wall time). This will help reduce the impact of such bugs in the policy engine (this recommendation has already been discussed with the Turnkey team; see PR [tkhq/mono#4881](#)).

2. Parser enclave uses local filesystem

Severity: **Informational**

Difficulty: **High**

Type: Undefined Behavior

Finding ID: TOB-TK3-2

Target: `mono/src/rust/evm_parser/app/src/routes/evm.rs`

Description

The Parser enclave conditionally uses a local filesystem to retrieve smart contract data (figure 2.1). This functionality is most likely not used in production and should be removed, as it creates an unnecessary attack surface.

```
fn contract(&self) -> Option<BaseContract> {  
    if self.abi_string == String::new() {  
        let path = format!("{}", self.abi_dir_path,  
self.to().to_lowercase());  
  
        fs::read_to_string(path)  
            .ok()  
            .and_then(|str_abi| serde_json::from_str::<Abi>(&str_abi).ok())  
            .map(BaseContract::from)  
    } else {  
        extract_contract(self.abi_string.as_str())  
    }  
}
```

*Figure 2.1: The redundant if path in the contract method
([mono/src/rust/evm_parser/app/src/routes/evm.rs#L233-L244](#))*

Exploit Scenario

An attacker sends specially crafted data to the Parser enclave that makes it access the local filesystem. The attacker finds a way to exploit the possible path traversal issue in the contract method to extract local JSON files from the enclave.

Recommendations

Short term, remove unused code in the Parser enclave that is related to the `abi_dir_path` configuration. Note that the enclave contains other instances of unused code beyond the contract method.

Long term, periodically review the code for unused features and remove them. For enclave codebases, as they are particularly important from a security perspective, consider keeping track of authors/owners of specific classes/files/methods (e.g., via Git), and whenever an owner leaves the company, perform a code review of the author's code and assign a new

owner. This should help to ensure that temporary functionalities are eventually removed from enclave code.

3. Incorrect Organization encoding for signature verification

Severity: **Medium**

Difficulty: **Medium**

Type: Cryptography

Finding ID: TOB-TK3-3

Target: `mono/src/rust/organization/src/organization.rs`

Description

The `Digest` trait for the `Organization` structure uses the incorrect schema to encode organization data. Version 15 is used instead of the version 16 (figure 3.1).

```
Version::V15 => {
    let schema: v15::OrganizationData = self.data.to_version_schema(&self.version);
    borsh::to_vec(&schema).map_err(|e|
VerifiableDataError::InvalidDigest(format!("{e:?}")))?
}
Version::V16 => {
    let schema: v15::OrganizationData = self.data.to_version_schema(&self.version);
    borsh::to_vec(&schema).map_err(|e|
VerifiableDataError::InvalidDigest(format!("{e:?}")))?
}
```

Figure 3.1: The invalid schema encoding

([mono/src/rust/organization/src/organization.rs#L565-L571](#))

Use of version 15 means that the new `smart_contract_interface_references` field introduced in version 16 is not included in the organization hash and, therefore, the organization is not protected by a Notarizer signature.

Exploit Scenario

An internal attacker modifies notarized organization data between requests to enclaves by setting arbitrary smart contract interfaces. This allows the attacker to bypass the organization's policy and sign transactions that otherwise would be denied.

Recommendations

Short term, replace `v15::OrganizationData` with `v16` in the relevant branch in the code shown in figure 3.1.

Long term, rewrite the `Organization::Digest` code with Rust macros to avoid manual repetition of similar code blocks. Refactor other methods that may suffer from similar problems. Add negative test cases for signed data that would bit-flip parts of data and check if signature verification fails as expected.

4. Notarizer does not validate smart_contract_address from user intent versus smart_contract_address from the context

Severity: **Medium**

Difficulty: **Medium**

Type: Data Validation

Finding ID: TOB-TK3-4

Target:
mono/src/rust/organization/src/mutations/smart_contract_interfaces.rs

Description

The `create` method of the Notarizer enclave takes as inputs user intent and SmartContractInterface context. Both of these structures include a `smart_contract_address` field. This field is not cross-validated between the intent and context.

```
data.smart_contract_interface_references
    .push(SmartContractInterfaceReference {
        smart_contract_interface_id: smart_contract_interface_id.clone(),
        smart_contract_address: intent.smart_contract_address.clone(),
        digest: signature.message,
    });
```

Figure 4.1: Part of the create method

(mono/src/rust/organization/src/mutations/smart_contract_interfaces.rs#L36-L41)

Exploit Scenario

An internal attacker captures a user intent to create a smart contract interface and replaces the `smart_contract_interface` context with a new one that has a different address and a different digest. The Notarizer adds the smart contract interface to the organization's data with the invalid, attacker-provided digest.

Recommendations

Short term, have the Notarizer validate that the `smart_contract_address` field from user intent matches the relevant field from the provided context.

5. Notarizer signs smart contract interfaces from Parser context instead of user intent

Severity: **Medium**

Difficulty: **Medium**

Type: Data Validation

Finding ID: TOB-TK3-5

Target:
mono/src/rust/organization/src/mutations/smart_contract_interfaces.rs

Description

There is a design issue in the flow for creating address–digest mappings in an organization’s notarized data.

The Notarizer enclave creates smart contract interfaces in an organization’s notarized data based on the following (figure 5.1):

- Address taken from user intent
- Digest taken from a context object signed by Parser enclave

Anyone can ask the Parser enclave to parse and sign arbitrary (syntactically valid) interfaces for arbitrary addresses, as the enclave does not deal with user intents. Therefore, it is possible to mix up contexts and the user’s intent.

In other words, the Notarizer cannot ensure that an IDL address–digest mapping is correct given only the address from the user.

```
/// create smart contract interface (ABI/IDL)
pub fn create(
    data: &mut OrganizationData,
    intent: &CreateSmartContractInterfaceIntent,
    _timestamp: Timestamp,
    context: Option<NotarizeContext>,
) -> Result<ResultEnum, MutationError> {
    let interface = context
        .ok_or_else(|| MutationError::MissingContext("context
missing".to_string()))?
        .smart_contract_interface
        .ok_or_else(|| MutationError::MissingContext("smart contract interface
missing".to_string()))?;

    let payload = interface
```

```

        .payload
        .ok_or_else(|| MutationError::MissingContext("smart contract interface
payload missing".to_string()))?;

    let smart_contract_interface_id = payload.smart_contract_interface_id;

    let signature = interface
        .signature
        .ok_or_else(|| MutationError::MissingContext("smart contract signature
missing".to_string()))?;

    data.smart_contract_interface_references
        .push(SmartContractInterfaceReference {
            smart_contract_interface_id: smart_contract_interface_id.clone(),
            smart_contract_address: intent.smart_contract_address.clone(),
            digest: signature.message,
        });

    Ok(ResultEnum::CreateSmartContractInterfaceResult(
        CreateSmartContractInterfaceResult {
            smart_contract_interface_id,
        },
    ))
}

```

Figure 5.1: The create method of the Notarizer enclave

([mono/src/rust/organization/src/mutations/smart_contract_interfaces.rs#L14-L48](#))

Exploit Scenario

An internal attacker captures a user intent to create a smart contract interface, and replaces the `smart_contract_interface` context with a new one that has a different digest. The Notarizer adds the smart contract interface to the organization's data with the invalid, attacker-provided digest.

Recommendations

Short term, have the Notarizer accept and use the smart contract interface digest from the user intent instead of the context. Then, have the enclave validate the digest and address from the user intent against relevant data from the context. Note that this solution requires users to know (compute) the correct digest value and include it in the intent.

Alternatively, have user clients include the whole IDL interface in the intent and the Notarizer enclave validate that data against data from the context. Note that relevant fields already exist in the `CreateSmartContractInterfaceIntent` structure but are not used by Notarizer.

The goal of this alternative recommendation is to ensure that the Notarizer only creates data that the user wants to create and uses the context from Parser enclave only to ensure

that user-provided information corresponds to a smart contract that had been successfully parsed by the Parser enclave.

Long term, create a formal model for the cryptographic protocol implemented by enclaves. It will be much easier to spot protocol-level issues like this one by using a formalized model than by reviewing code. Moreover, the model would provide more assurance to users who may be willing to review such simplified, yet formal and automatically verifiable, data rather than reconstructing the model from the source code. Of course, guaranteeing that the model matches the implementation would be a hard task, but it is still easier than reviewing the protocol without any specification. Ideally, the model should be encoded with a cryptographic tool like CryptoVerif or ProVerif, but even a set of diagrams would be a good starting point.

6. Weak validation of parsed Solana IDLs against notarized organization data

Severity: **Medium**

Difficulty: **Medium**

Type: Data Validation

Finding ID: TOB-TK3-6

Target: `mono/src/rust/ump/app/src/routes/decider.rs`

Description

There is a design issue in the flow for enforcing policies based on parser transaction data in the Ump enclave. The Ump enclave takes the following as input:

- User intent as `{unsigned_transaction}`
- `SolanaParsedTransaction` context structure (signed by the Parser enclave) as `{unsigned_payload, transaction_metadata, SolanaIdlDigests[(program_id, digest)]}`
- Notarized organization data as `SmartContractInterfaceReference: {smart_contract_interface_id, smart_contract_address, digest}`

The `SolanaParsedTransaction` context is generated by the Parser from the following inputs:

- Vector of signed `SmartContractInterfaces`:
 - Random `smart_contract_interface_id`
 - Arbitrary user-provided `organization_id`, `smart_contract_address`, `smart_contract_interface`, `label`, and `notes` values
- `unsigned_payload`

The Ump enclave performs these validations:

- `Intent.unsigned_transaction == Context.unsigned_payload`
- For all instructions from `transaction_metadata` that are also in the organization data (compared by address):
 - The instruction is in the `Context.SolanaIdlDigests` vector.
 - The `digest` from the context equals the `digest` from the notarized data.

These are the issues with the current protocol:

1. The `organization_id` field in the `SmartContractInterface` can be set to an arbitrary value. Moreover, the field is not used for any access control. This behavior makes the field misleading for code reviewers.
2. If the organization data does not contain an IDL with a given address (program key), then an arbitrary IDL definition can be provided for that address. This behavior makes it possible to bypass organization policies in some cases.
3. The `smart_contract_interface_id` fields in notarized organization data are not validated against the IDs generated by the Parser enclave. Since the IDs are included in the digest, the missing validation is not critical, but it unnecessarily gives attackers more degrees of freedom.

The following statement in the public documentation (figure 6.1) indicates that if an IDL is used that corresponds to a program that has not been uploaded to an organization's data, then the `parsed_instruction_data` should not be available to the Policy Engine. As described above, this assumption is false.

Policy Formats

On the Solana side, if an IDL corresponding to a program has been uploaded, then IDL related policies for instructions calling that program will be available in each instruction under the `parsed_instruction_data` namespace. The subfields will be as follows:

Figure 6.1: An excerpt from public documentation

(<https://docs.turnkey.com/concepts/policies/smart-contract-interfaces#policy-formats-2>)

To demonstrate problem 2, the following test case can be added to the `ump/app/src/routes/decider.rs` file (figure 6.2).

```
#[test]
fn solana_parsed_transaction_with_unknown_idl_fails() {
    use gen::private::data::v1::OrganizationData;
    use gen::private::data::v1::SmartContractInterfaceReference;
    use gen::private::parser::v1::SolanaIdlDigest;

    // org data contains an IDL for program "11111111111111111111111111111111"
    let org_data = OrganizationData {
        smart_contract_interface_references: vec![SmartContractInterfaceReference {
            smart_contract_address: "11111111111111111111111111111111".to_string(),
            digest: "expected_digest".to_string(),
            ..Default::default()
        }],
        ..Default::default()
    };
};
```

```

// Parser provided IDLs: one known and one unknown to the organization
let mut sol_tx_bad = get_sign_solana_context_with_parsed_accounts();
if let Some(ref mut payload) = sol_tx_bad.payload {
    payload.idl_interface_digests = vec![SolanaIdlDigest {
        program_id: "11111111111111111111111111111111".to_string(),
        digest: "expected_digest".to_string(),
    }, SolanaIdlDigest {
        program_id: "111111111111111111111111111111111337".to_string(),
        digest: "other_digest".to_string(),
    } ];
}
let context_bad = DecisionContext {
    parsed_transaction: None,
    account: None,
    oauth_issuers: vec![],
    solana_parsed_transaction: Some(sol_tx_bad),
    parsed_wallet_accounts: vec![],
    tron_parsed_transaction: None,
    bitcoin_parsed_transaction: None,
};

let res_bad = verify_sol_parsed_tx(&context_bad, "fake_tx", &org_data);
assert_eq!(
    res_bad,
    Err(TurnkeyError::internal(&format!(
        "Solana IDL with unknown program id {} was used for transaction
parsing",
        "111111111111111111111111111111111337"
    )))
);
}

```

Figure 6.2: A test case that demonstrates that unknown IDLs are accepted

Exploit Scenario

An organization has one IDL in its organization data: a program with address 1111 and an instruction with discriminator X. The organization has a policy that only allows transactions that call the specific instruction (`SolanaParsedInstructionData.discriminator == X`). Since only one contract is uploaded to the organization, the organization administrators assume that only calls to this contract are allowed.

An attacker constructs a transaction that calls the program instruction with discriminator Y in a program with address 1337. The program is allowed by the policy engine, since the address is not explicitly validated by the organization's policy. The attacker sends a fake IDL (`SolanaParsedTransaction` context structure) along with the transaction. This fake IDL makes the Ump enclave think that instruction X in program 1337 is called, while in reality instruction Y will be called on-chain.

The attacker bypassed the organization's policy.

Recommendations

Short term, implement the following changes:

1. Remove the `organization_id` field from the `SmartContractInterface` structure, as it is not used and is misleading.
2. Have the Ump enclave validate that all `SolanaParsedTransaction.SolanaIdlDigests` in the `SmartContractInterfaceReference` context are included in organization data. Both address/program_id and digests must be cross-validated. This check must ensure that if an IDL that was not previously uploaded to the organization is used to parse a transaction, then the transaction is rejected by the Ump enclave. If such behavior is too strict and does not meet Turnkey's business requirements, then make one of the following changes:
 - a. Have the Ump enclave mark metadata for program calls parsed from IDLs that are not part of the organization's data as "untrusted." With this change, users can specify in a policy whether such "untrusted" metadata is allowed.
 - b. Add a new global flag for organizations that would specify if transaction parsing with IDLs not included in organization's data is allowed.
3. Add a `smart_contract_interface_ids` field to the `SolanaParsedTransaction.SolanaIdlDigests` structure and have the Parser enclave populate that ID from the `SmartContractInterfaces` input. Have the Ump enclave then validate these IDs against IDs in the organization's notarized data. This new validation should not break business logic, as the IDs are already implicitly validated via digests. Making the ID validation explicit is only a defense-in-depth measure.

Long term, create a formal model for the cryptographic protocol implemented by enclaves. At the very least, create an informal document that describes the protocol at a high level and that can be easily mapped to the implementation.

7. Turnkey accepts ABIs/IDLs with colliding functions

Severity: **Medium**

Difficulty: **High**

Type: Data Validation

Finding ID: TOB-TK3-7

Target: `mono/src/rust/evm_parser/app/src/routes/{evm.rs,solana.rs}`

Description

The transaction parser (`evm_parser`) accepts ABIs with colliding function selectors. For instance, the program in figure 7.1 has two colliding functions. The Solidity compiler will refuse to compile such a program due to the collision, but Turnkey accepts its ABI, making function resolving ambiguous.

```
pragma solidity ^0.8.0;

contract Collision {
    function transfer(uint256 amount) external {} // 0x12514bba
    function transfer997327884(uint256 amount) external {} // 0x12514bba
}
```

Figure 7.1: An invalid Solidity contract with colliding function selectors

We verified this issue by signing a colliding ABI (figure 7.2) and parsing a transfer call that resolved to a `transfer997327884` call instead (figure 7.3).

```
grpcurl -plaintext -d '{"smart_contract_interface": "[{"type":
\"function\", \"name\": \"transfer\", \"inputs\": [{\"name\": \"amount\", \"type\":
\"uint256\", \"internalType\": \"uint256\"}], \"outputs\": [], \"stateMutability\":
\"nonpayable\" }, { \"type\": \"function\", \"name\": \"transfer997327884\",
\"inputs\": [ { \"name\": \"amount\", \"type\": \"uint256\", \"internalType\":
\"uint256\" } ], \"outputs\": [], \"stateMutability\": \"nonpayable\" } ]",
"smart_contract_address": "0x5FbDB2315678afecb367f032d93F642f64180aa3", "type":1}'
localhost:41178 services.parser.v1.ParserService/ValidateSmartContractInterface
```

Figure 7.2: A call to `ValidateSmartContractInterface` with a colliding ABI

```
$ grpcurl -plaintext -d '{"unsigned_payload":
"0x02f849827a69800184773594018203e8945fbd2315678afecb367f032d93f642f64180aa380a4125
14bba0000000000000000000000000000000000000000000000000000000000000000000000000000c0",
"smartContractInterface": { <response from figure 7.2> } } ' localhost:41178
services.parser.v1.ParserService/Parse | jq
.parsedTransaction.payload.transactionMetadata[13]
{
  "key": "function_name",
  "value": "transfer997327884"
```

```
}
```

Figure 7.3: Function name resolved to the colliding `transfer997327884` name

A similar issue exists in Solana IDLs where discriminators are 8 bytes long, which makes them much harder to collide than Solidity function selectors; however, a collision is nevertheless possible with sufficient computing power.

Exploit Scenario

A Turnkey user uses an ABI with colliding function selectors that produce an unexpected function name. The wrong function name is used in a policy, making it ineffective.

Recommendations

Short term, update the `validate_abi` and `validate_idl` functions to reject ABIs/IDLs with colliding function selectors/discriminators. This will remove the ambiguity when resolving function names while parsing transactions.

Long term, expand the test suite for transaction parsing.

8. ConditionValidator membership operator has exponential time complexity

Severity: **Medium**

Difficulty: **Low**

Type: Denial of Service

Finding ID: TOB-TK3-8

Target: mono/src/rust/policy/src/validators/condition.rs,
mono/src/rust/policy/src/evaluator/traits.rs

Description

The internal crate organization (used by ump, signer, etc.) invokes policy's ConditionValidator evaluator to ensure that a given condition expression looks valid. ConditionValidator implements different logic than other evaluators for Expr::Comparison nodes: it allows a comparison with Value::GenericArgument() nodes, whose type will only be known later, during the effective evaluation of the policy (figure 8.1).

```
#[allow(clippy::boxed_local)]
fn evaluate(&mut self, expr: Box<Expr>) -> Result<Value, EvaluatorError> {
    match *expr {
        Expr::Function(function) => self.evaluate_fn(function),
        Expr::List(list) => self.evaluate_list(list),
        Expr::Access(left, op) => self.evaluate_access(left, op),
        Expr::Logical(left, op, right) => self.evaluate_logical(left, op, right),
        Expr::Comparison(left, op, right) =>
self.condition_validation_evaluate_comparison(left, op, right),
        Expr::Unary(op, right) => self.evaluate_unary(op, right),
        Expr::Literal(literal) => self.evaluate_literal(literal),
        Expr::Ident(ident) => self.evaluate_ident(ident),
    }
}
```

Figure 8.1: Excerpt of condition.rs

(mono/src/rust/policy/src/validators/condition.rs#L56–L68)

The catch-all will fall back to calling the original implementation of Evaluator.evaluate_comparison(), after recursively evaluating both left and right expressions (figure 8.2).

```
fn condition_validation_evaluate_comparison(
    &mut self,
    left: Box<Expr>,
    op: ComparisonOp,
    right: Box<Expr>,
) -> Result<Value, EvaluatorError> {
```

```

let left_val = self.evaluate(left.clone())?;
let right_val = self.evaluate(right.clone())?;
match (&left_val, &right_val, &op) {
    // [...]
    (_, Value::GenericArgument(_), ComparisonOp::In) => Ok(Value::Bool(false)),
    _ => self.evaluate_comparison(left, op, right),
}
}

```

Figure 8.2: Excerpt of `condition.rs`

([mono/src/rust/policy/src/validators/condition.rs#L77-L139](#))

`Evaluator.evaluate_comparison()` recursively evaluates both left and right again, exhibiting an exponential time complexity (figure 8.3).

```

/// evaluate a comparison operation
fn evaluate_comparison(
    &mut self,
    left: Box<Expr>,
    op: ComparisonOp,
    right: Box<Expr>,
) -> Result<Value, EvaluatorError> {
    let left = self.evaluate(left)?;
    let right = self.evaluate(right)?;

    if let ComparisonOp::In = op {
        let values = match &right {
            Value::List(list) if !list.is_empty() => list,
            Value::List(_) => return Ok(Value::Bool(false)),
            _ => return Err(EvaluatorError::InvalidComparison(left, op, right)),
        };
    };
}

```

Figure 8.3: Excerpt of `traits.rs`

([mono/src/rust/policy/src/evaluator/traits.rs#L351-L366](#))

Such worst-case complexity can arise by chaining comparison expressions that are caught by the catch-all arm in

`ConditionEvaluator.condition_validation_evaluate_comparison()`.

Additionally, this function first clones both the left and right variables, which is a relatively expensive operation compared to the rest of the evaluation loop.

This can be demonstrated by evaluating nested comparisons between the membership operator (`ComparisonOp::In`) and a `Value::List` (figures 8.4, 8.5).

```

$ time -- cargo +nightly run --example tester "$(python3 -c "print('1' + ' in []' * 25)")"
[...]
real 29.69
user 29.62
sys 0.06

```

Figure 8.4: Test with 25 comparisons, leading to about 33 million evaluations

```
$ time -- cargo +nightly run --example tester "$(python3 -c "print('1' + ' in []' * 30)")"
[...]
real 916.64
user 916.42
sys 0.07
```

Figure 8.5: Test with 30 comparisons, leading to more than a billion evaluations

Exploit Scenario

An attacker crafts a malicious policy with nested comparison expressions that specifically target this edge case, such as membership comparisons (e.g., `1 in [] in [] in [] [...]` in `[]`), and submits it for validation. The exponential complexity of the validator evaluator causes it to consume the resources of a CPU core for an extended period.

The impact of this scenario is similar to the scenario provided for [TOB-TK3-1](#), except that the process will not consume excessive memory and will not crash. An attacker can still attempt to trigger it on many enclave instances in parallel, and a sustained attack could cause service degradation for other Turnkey users.

Recommendations

Short term, implement a recursion depth counter in the `ConditionValidator` trait by adding a depth parameter that is incremented on entry, and impose a reasonable threshold that causes the evaluation to stop when exceeded. The threshold should be determined based on the available compute resources of the enclave; for example, a policy evaluation should not take more than a few seconds to complete. A similar mitigation in the base implementation of the evaluator was suggested for [TOB-TK3-1](#).

Additionally, the `1_validate_evaluate` and `2_validate_evaluate_filter` fuzzing harnesses that cover `ConditionValidator` are too low; see [appendix C](#) for suggestions on how to improve their speed.

Long term, refactor the evaluator to use an iterative algorithm, which would eliminate this risk entirely. Additionally, implement further validation passes to reject policies that exceed complexity thresholds during the parsing phase, thereby preventing malicious policies from reaching the evaluation stage. This will prevent similar recursion issues from being introduced in future code modifications.

Additionally, process policies in an isolated process with strong resource constraints (CPU, memory, wall time). This will help reduce the impact of such bugs in the Policy Engine (this recommendation has already been discussed with the Turnkey team; see PR [tkhq/mono#4881](#)).

9. Integer overflow in constrained vector allocation method may lead to panics in the Parser enclave

Severity: **Medium**

Difficulty: **Low**

Type: Denial of Service

Finding ID: TOB-TK3-9

Target:

mono/src/rust/evm_parser/app/src/parsing_utilities/idl_parser.rs

Description

There is an integer overflow issue in the Parser enclave that allows the “budget” control for memory allocations to be bypassed.

The issue is that the user-provided `len` integer is multiplied by 32 (the size of the Value JSON object) before comparison with the `remaining_budget` variable, but only the `len` is used for memory allocation (figure 9.1).

Memory allocation for a vector with a huge capacity causes a panic.

```
// This method checks to see whether there is enough memory left in the budget to be
// allocated, and if so, creates a vector of arg type objects of the correct length
fn create_allocated_arg_vector(&mut self, len: usize) -> Result<Vec<ArgType>,
TurnkeyError> {
    let amount = len * size_of::<serde_json::Value>();

    if amount > self.remaining_budget {
        return Err(TurnkeyError::new(
            Code::InvalidArgument,
            "Memory allocation exceeded maximum allowed budget while parsing IDL
call data -- check your uploaded IDL or call data",
        ));
    }
    self.remaining_budget -= amount;

    Ok(Vec::with_capacity(len))
}
```

Figure 9.1: The vulnerable function

(mono/src/rust/evm_parser/app/src/parsing_utilities/idl_parser.rs#L281-L294)

The test in figure 9.2 can be used to confirm the issue.

```
#[test]
```

```

fn test_parse_type_array_u32() {
    let array_id1 = r#"
    {
        "types": [],
        "instructions": [
            {
                "name": "arrayTest",
                "accounts": [],
                "args": [
                    {
                        "name": "numbers",
                        "type": { "array": ["u32", 576460752303423489] }
                    }
                ]
            }
        ]
    }
    "#;

    let id1 = decode_id1_data(array_id1).unwrap();
    let discriminator = compute_default_anchor_discriminator("arrayTest").unwrap();

    let mut instruction_data = discriminator.clone();
    instruction_data.extend_from_slice(&[
        0x01, 0x00, 0x00, 0x00, // 1 in little-endian
        0x02, 0x00, 0x00, 0x00, // 2 in little-endian
        0x03, 0x00, 0x00, 0x00, // 3 in little-endian
    ]);
    let instruction =
        find_instruction_by_discriminator(&instruction_data,
        id1.instructions.clone()).unwrap();
    let result = parse_data_into_args(&instruction_data, &instruction,
    &id1).unwrap();

    assert_eq!(result.len(), 1);
    assert_eq!(result[0].arg_name, "numbers");
}

```

Figure 9.2: A new test for the `idl_parser.rs` file that causes a panic

Note that to observe the panic, the test must be run with integer overflow checks disabled (as in release mode), using `RUSTFLAGS="-C overflow-checks=off" cargo test parsing_utilities::idl_parser::tests::test_parse_type_array_u32`.

Exploit Scenario

An external attacker continuously sends malicious IDL definitions, crashing Parser enclaves and causing a global denial of service for the system.

Recommendations

Short term, have the Parser validate the user-provided length against overflows before using it for budget calculation.

Long term, fuzz the enclave codebases in debug mode to check that integer overflows are reported as panics.

10. Solana transactions with repeated accounts are accepted

Severity: Informational	Difficulty: High
Type: Data Validation	Finding ID: TOB-TK3-10
Target: mono/src/rust/evm_parser	

Description

The Solana transaction parser implemented in the Parser enclave accepts and parses transactions with repeated accounts (the `account_keys` field in the `SolanaMetadata` structure). Such transactions are rejected when sent to blockchain nodes via the RPC API.

The severity of this issue is informational because even if the issue allows a policy to be bypassed, the generated signature would be for an invalid transaction and should not enable any attacks.

The test case in figure 10.1 can be used to observe that transactions with repeated accounts are successfully parsed.

[illegible]

Figure 10.1: A new test case that should fail because of duplicated accounts

Sending the (Base58-encoded) transaction to the Solana RPC API returns an `AccountLoadedTwice` error (figure 10.2).

```
curl https://api.devnet.solana.com -s -X \
  POST -H "Content-Type: application/json" -d '
  {
    "jsonrpc": "2.0",
```

```

    "id": 1,
    "method": "simulateTransaction",
    "params": [
      "AQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
      AAAABAAIGZBCdiHdyLGp2mTaKS+mkZ+3SOSI+ARZczt40me6NgSxkEJ2Id3IsanaZNopL6aRn7dI5Ij4BF1z
      03jSZ7o2BLAabiFf+q4GE+2h/Y0YYwDXaxDncGus7VZig8AAAAABCCx4+caX3vccDS6EL0crfYM+yxvjcV4
      GUMYGTWkb5DAG3fbh12Whk9nL4Ub063msHLSF7V9bN5E6jPWFfv8AqQAAAAAAAAAAAAAAAAAAAAAAAAAAAA
      AAAAAAAAAAAAAQJCQkJCQkJCQkJCQkJCQkJCQkJCQkJCQkJCQkJCQkIBBAQBAGMACgwAXtCyAAAAAAk=",
      {
        "commitment": "confirmed",
        "encoding": "base64",
        "replaceRecentBlockhash": true
      }
    ]
  },
  .

```

Figure 10.2: RPC call that returns an error

Recommendations

Short term, decide if semantically invalid transactions should be accepted by the Parser. If so, document that fact publicly. Otherwise, add basic validations to the Parser enclave (like for duplicated accounts) and/or have the backend service (outside of enclaves) use the `simulateTransaction` RPC API for the validation.

Since there are many semantic errors possible, some of which likely require on-chain data to be detected, we suggest keeping the semantic validation simple and asking users to make sure that the transactions they want to sign are valid. Examples of such semantic validations include checking for closed accounts, account discriminators, **duplicated accounts loaded via address table lookups**, and incorrect indexes provided for account table lookups.

Consider creating an issue in the **solana-program crate** to add validation for repeated accounts in the **Message::sanitize** method used by Turnkey.

11. Organization policies may misinterpret WSOL transfers

Severity: **Medium**

Difficulty: **Medium**

Type: Access Controls

Finding ID: TOB-TK3-11

Target: `mono/src/rust/`

Description

Wrapped SOL (WSOL) tokens are special SPL tokens with unique semantics: transferring WSOL “**not only modif[ies] the token balance but also transfer[s] an equal amount of SOL.**” Turnkey transaction metadata includes two fields: `transfers` (for SOL native currency transfers) and `spl_transfers` (for token transfers). WSOL transfers are included only in the latter field, but on-chain, both WSOL tokens and SOL native currency are effectively transferred.

The discrepancy between on-chain behavior and Turnkey transaction metadata may cause organization policies to be ineffective.

Exploit Scenario

An organization has a policy that limits the allowed SOL transfer amount:

```
solana.tx.transfers.any(t, t.amount < "MAX_AMOUNT")
```

A malicious user bypasses this policy by sending an arbitrary amount of WSOL tokens.

A safe version of the policy would look like the following:

```
solana.tx.transfers.any(t, t.amount < "HALF_MAX_AMOUNT") &&  
solana.spl_transfers(t, t.token_mint == "WSOL_TOKEN_MINT" &&  
t.amount < "HALF_MAX_AMOUNT")
```

Note that the second policy does not perfectly resemble the user’s intention, as it must use half of the maximal token amount; otherwise, the WSOL transfers would have to be added (in the policy) to SOL transfers and then compared against the `MAX_AMOUNT`.

Recommendations

Short term, inform users about the special WSOL tokens and the fact that they should take their behavior into account when creating policies.

Additionally, to make it easier for users to write secure policies, consider making the Parser handle WSOL tokens specially by including the transfers in both the `transfers` and `sp1_transfers` metadata fields.

12. Weak validation of SmartContractInterfaceType in enclaves

Severity: **Medium**

Difficulty: **High**

Type: Data Validation

Finding ID: TOB-TK3-12

Target: `mono/src/rust/`

Description

The `SmartContractInterfacePayload` structure is used to store both Ethereum and Solana smart contract interfaces (IDL program definitions) in notarized organization data. The two use cases are differentiated by the `r#type` field in the structure. The type is not validated by the enclaves. This behavior may enable data swap attacks where an Ethereum smart contract is provided in place of a Solana program or vice versa.

```
#[allow(clippy::derive_partial_eq_without_eq)]
#[derive(Clone, PartialEq, ::prost::Message)]
pub struct ParseRequest {
    #[prost(string, tag = "1")]
    pub unsigned_payload: ::prost::alloc::string::String,
    #[prost(message, optional, tag = "2")]
    pub smart_contract_interface: ::core::option::Option<
        super::super::super::immutable::models::v1::SmartContractInterface,
    >,
}

#[allow(clippy::derive_partial_eq_without_eq)]
#[derive(Clone, PartialEq, ::prost::Message)]
pub struct SolanaParseRequest {
    #[prost(string, tag = "1")]
    pub unsigned_payload: ::prost::alloc::string::String,
    #[prost(message, repeated, tag = "2")]
    pub smart_contract_interfaces: ::prost::alloc::vec::Vec<
        super::super::super::immutable::models::v1::SmartContractInterface,
    >,
}
```

Figure 12.1: Parser enclave accepts the same structure for both Ethereum and Solana parsing requests.

(`mono/src/rust/gen/src/gen/services.parser.v1.rs#L57–L84`)

The severity of this finding is medium because the issue may allow some organization policies to be bypassed. The difficulty is high because the conditions to make exploitation possible are strict: either the organization must have Ethereum–Solana contracts/programs with colliding addresses in their notarized data, or an Ethereum transaction sent by a user must also be a valid Solana transaction (or vice versa).

Exploit Scenario

An organization has a few Ethereum and Solana contracts stored in its notarized data. One of the Ethereum smart contracts happens to have the same address as one of the Solana programs.

A user sends an Ethereum transaction to Turnkey for signing. The transaction references the Ethereum smart contract with the colliding address.

An internal attacker intercepts a user's request to sign the transaction. The attacker sends his own request to the Parser enclave (a `ParseRequest` request for Ethereum) with the intercepted raw transaction and `SmartContractInterface` referencing the Solana program (instead of the Ethereum contract). The Parser does not have validation to detect that the request is for an Ethereum transaction but is referencing a Solana program. It is confused (as it interprets the Solana program as an Ethereum smart contract) and produces incorrect metadata for the transaction.

The attacker forwards signed output from the Parser along with the user intent to the Ump enclave, which makes the decision based on incorrect transaction metadata. The policy is bypassed.

Exploit Scenario 2

An organization has a few Ethereum and Solana contracts stored in its notarized data. One of the Ethereum smart contracts happens to have the same address as one of the Solana programs.

A user sends an Ethereum transaction to Turnkey for signing. The transaction references the Ethereum smart contract with the colliding address.

An internal attacker intercepts a user's request to sign the transaction. The attacker sends his own request to the Parser enclave (a `SolanaParseRequest` request for Solana) with the intercepted raw transaction and `SmartContractInterface` referencing the Solana program (instead of the Ethereum contract). The Parser is confused (as it parses the Ethereum transaction as if it were a Solana transaction) and produces incorrect metadata for the transaction.

The attacker forwards signed output from the Parser along with the user intent to the Ump enclave, which does not have validation to detect that the Parser enclave parsed the transaction as if it were for Solana even though the user intent is to sign an Ethereum transaction. The Ump enclave makes the decision based on incorrect transaction metadata. The policy is bypassed.

Recommendations

Short term, add validation for smart contract and IDL types in the Parser, Notarizer, and Ump enclaves:

- Have the Notarizer validate the type in `CreateSmartContractInterfaceIntent` against the type in the `SmartContractInterface` context in the `create method`. This will prevent mix-up attacks when new contracts are added to notarized data.
- Have the Parser validate that all contracts provided as context inputs (`SmartContractInterface` structures) have the expected type (Solana for `SolanaParseRequest` and Ethereum for `ParseRequest` RPC calls).
- Add a new type field to the `SolanaIdlDigest` structure and have the Ump enclave validate the type in `SignTransactionIntent` against newly added types in `SolanaParsedTransactionPayload::idl_interface_digests` structures.

Note that all three fixes from the list above are required to completely prevent mix-up attacks between Ethereum and Solana contracts.

Review the Ethereum functionality in all three enclaves and add validations for smart contract types in relevant functions.

Long term, create a formal model for the cryptographic protocol implemented by enclaves. At the very least, create an informal document that describes the protocol at a high level and that can be easily mapped to the implementation.

13. Solana dynamic address lookups may hinder policies

Severity: **Medium**

Difficulty: **High**

Type: Access Controls

Finding ID: TOB-TK3-13

Target: `mono/src/rust/`

Description

Solana allows transactions to specify addresses (for accounts and programs) both statically (when a full address is provided in the transaction) and dynamically (when full addresses are stored on-chain and transactions only reference them by indexes).

The Parser enclave sets dynamic addresses to the `ADDRESS_TABLE_LOOKUP` constant. Because of that, the Ump enclave will execute organization policies against that constant instead of fully resolved addresses. This behavior may be surprising to users and may enable policies to be bypassed.

Exploit Scenario

An organization has a policy that prevents execution of a program at address X. An attacker sends a transaction that references the program at address X via dynamic address lookup table. The transaction is allowed as the policy compares X address against the `ADDRESS_TABLE_LOOKUP` constant. The policy is bypassed.

Exploit Scenario 2

An organization has a policy that only allows transfers to an account with address Y. A user sends a transaction that transfers tokens to address Y, but the transaction references the address via dynamic address lookup table. The transaction is incorrectly forbidden, causing confusion in the user's organization.

Recommendations

Short term, document the use of the `ADDRESS_TABLE_LOOKUP` constant in Turnkey policies. Instruct users on how to write policies that correctly handle the constant (to prevent policy bypass and denial-of-service issues).

Consider replacing the `to` and `from` fields in transactions with a union that holds either a string (full address explicitly) or a reference to an item in the `solana.tx.address_table_lookups` table's `writable_indexes` or `readonly_indexes` list. Next, change the Policy Engine to handle the union and the Ump enclave to populate the addresses as needed. With these changes, users can write policies that refer to on-chain address table lookups, as they can know about on-chain data and trust the relevant addresses.

Long term, add a feature that enables users to import address table lookups into their organization's notarized data. The feature should work similarly to how users can now add IDLs. With that addition, make the Parser and/or Ump enclave resolve dynamic address lookups based on the imported tables.

Consider adding an organization-wide setting that can disallow Solana transactions with address table lookups, so that only transactions with all addresses provided explicitly in the transaction can be accepted.

Alternatively, consider resolving dynamic Solana addresses with on-chain data. Such a feature would require Turnkey to run a Solana validator node and would require a careful design to establish trust between enclaves and the node, guarantee data freshness, and prevent denial-of-service issues in the system.

14. Solana programs cannot be resolved via dynamic address lookup tables

Severity: **Informational**

Difficulty: **Medium**

Type: Denial of Service

Finding ID: TOB-TK3-14

Target: `mono/src/rust/`

Description

Solana addresses of accounts and programs can be resolved statically or dynamically via address lookup tables (on-chain data). The Parser forbids transactions with dynamic program resolution by calling the `VersionedMessage::sanitize` method from the `solana-sdk` (figure 14.1). This fact is not documented by Turnkey.

```
// reject program ids loaded from lookup tables so that
// static analysis on program instructions can be performed
// without loading on-chain data from a bank
let max_program_id_ix =
    // `expect` is safe because of earlier check that
    // `num_static_account_keys` is non-zero
    num_static_account_keys
        .checked_sub(1)
        .expect("message doesn't contain any static account keys");
```

*Figure 14.1: Part of the sanitize method
([solana-sdk/message/src/versions/v0/mod.rs#L158-L166](#))*

Exploit Scenario

A user sends a valid transaction that calls a program referenced via address table lookup. The transaction is rejected by Turnkey. The user is confused.

Recommendations

Short term, document that Turnkey rejects transactions with Solana programs referenced via address table lookup.

Long term, implement the long-term recommendations provided for finding [TOB-TK3-13](#) to support dynamic resolution of program addresses in the Parser and Ump enclaves.

15. Uploading Solana IDLs with colliding addresses is allowed	
Severity: Medium	Difficulty: Medium
Type: Data Validation	Finding ID: TOB-TK3-15
Target: <code>mono/src/rust/</code>	

Severity: **Medium**

Difficulty: **Medium**

Type: Data Validation

Finding ID: TOB-TK3-15

Target: mono/src/rust/

Description

The Notarizer enclave accepts Solana IDLs with colliding addresses:

- The Notarizer does not check if a new IDL to be added into organization data does not have an address that is already in the smart_contract_interface_references list.
- The Notarizer does not check if a new IDL to be added into organization data does not have an address colliding with constant program addresses like 11111111111111111111111111111111 (SOL_SYSTEM_PROGRAM_KEY) and TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA (TOKEN_PROGRAM_KEY).

Exploit Scenario

An attacker uploads an IDL with the address 11 into an organization's notarized data. Then, the attacker sends a transaction that makes a SOL transfer by calling the system program. The Parser enclave interprets the single program call as two calls: one to a well-known program at the SOL_SYSTEM_PROGRAM_KEY address and one to the program with a previously uploaded IDL.

The Ump enclave uses incorrect transaction metadata to execute the policy.

Recommendations

Short term, have the Notarizer validate addresses of programs to be imported into notarized data:

- The address must not already be in the `smart_contract_interface_references` list.
- The address must not collide with well-known program addresses.

Document the requirements above in the public documentation.

Consider adding similar validations in the Parser enclave as a defense-in-depth measure:

- The `SolanaParseRequest::smart_contract_interfaces` list must not contain duplicated addresses.
- The `SolanaParseRequest::smart_contract_interfaces` list must not contain addresses colliding with well-known program addresses.

Long term, add a custom attribute to Rust structures that must be unique within an organization. Have enclaves automatically validate fields with this new attribute against organization data when deserializing structures (if the organization data is also available). Having such a uniqueness validation implemented on the structure level will make the code more readable and should help prevent vulnerabilities resulting from broken uniqueness assumptions.

16. Solana transaction metadata for policies does not include transfers made by calls to programs

Severity: **Low**

Difficulty: **High**

Type: Access Controls

Finding ID: TOB-TK3-16

Target: `mono/src/rust/`

Description

The Solana transaction metadata contains lists of SOL transfers and SPL token transfers. Only top-level transfers executed by direct calls to system programs are recognized and included in these lists. Transfers made indirectly by programs calling system programs are not recognized. This behavior may be misleading for users writing policies and may enable policy bypass.

Exploit Scenario

An organization has a policy that limits the maximal transfer amount from a given address. An attacker creates a transaction that calls a custom program that makes transfers with arbitrary amounts from the restricted address. The policy is bypassed.

Recommendations

Short term, add documentation about this behavior. Guide users on how to write secure policies that cannot be bypassed with calls to unknown programs. For example, inform users that when a policy limits SOL or SPL token transfers, then it must also block calls to all programs except the relevant system programs and possibly some allowlisted programs.

17. Solana transaction metadata for policies does not include transfers made by some methods of supported system programs

Severity: **Medium**

Difficulty: **High**

Type: Access Controls

Finding ID: TOB-TK3-17

Target: `mono/src/rust/`

Description

Solana programs supported by Turnkey (System, Token, Token-2022) have many methods (instructions) to move funds between accounts. Only a few of these methods are currently supported by the Parser (Transfer, TransferChecked, TransferCheckedWithFee). This fact is not documented in the public documentation and may enable policy bypass.

These are examples of System program instructions that may transfer funds:

- **TransferWithSeed**
- **WithdrawNonceAccount**

These are examples of Token program instructions that may transfer funds:

- **MintTo**
- **Burn**

These are examples of Token-2022 program instructions that may transfer funds:

- **WithdrawExcessLamports**
- **Confidential transfers**: This is an upcoming feature that enables confidential transfers, where the amounts being transferred are hidden. This feature is currently not available on the mainnet but may be enabled in the near future.
- **Transfer hooks**

Exploit Scenario

An organization has a policy that limits the maximal transfer amount from a given address. An attacker creates a transaction that calls the `WithdrawExcessLamports` instruction from the Token-2022 program that makes transfers with arbitrary amounts from the restricted address. The policy is bypassed.

Recommendations

Short term, add documentation about this behavior. Guide users on how to write secure policies that cannot be bypassed with calls to unsupported instructions from supported programs. For example, inform users that when a policy limits SOL or SPL token transfers, then it must also block calls to all unsupported instructions of the System, Token, and Token-2022 programs.

Long term, consider implementing parsing of transfers made by all methods of the supported system programs.

18. Incomplete mitigation of user existence oracles in coordinator

Severity: Informational

Difficulty: Low

Type: Error Reporting

Finding ID: TOB-TK3-18

Target: `src/go/coordinator/internal/dashboard/dashboard_api_server.go`

Description

The Turnkey coordinator attempts to limit user existence oracles by ensuring that users always receive a verification email upon login and sign-up, regardless of whether an account with the same email address already exists on the platform (figure 18.1).

```
func (s *DashboardApiServer) SendLoginMagicLink(ctx context.Context, request
*dashboard_api.SendLoginMagicLinkRequest)
(*dashboard_api.SendLoginMagicLinkResponse, error) {
    // [...]
    user, err := s.db.User().GetByEmail(ctx, request.Email)
    if err != nil {
        if err == db.ErrNoRows {
            slog.Warn("user attempting to get login magic link with
unregistered account")

            return &dashboard_api.SendLoginMagicLinkResponse{}, nil
        }

        return nil, err
    }
    // [...]
    return &dashboard_api.SendLoginMagicLinkResponse{}, nil
}
```

Figure 18.1: Excerpt of `dashboard_api_server.go`

(`mono/src/go/coordinator/internal/dashboard/dashboard_api_server.go#L500–L550`)

However, in the case where a user has just created their account and has not yet completed their onboarding, the user entry is not associated with an organization, and an `InvalidArg` error is returned as part of the API response (figure 18.2).

```
func (s *DashboardApiServer) SendLoginMagicLink(ctx context.Context, request
*dashboard_api.SendLoginMagicLinkRequest)
(*dashboard_api.SendLoginMagicLinkResponse, error) {
    // check if a valid email has been passed in
    if !ValidateEmail(request.Email) {
        return nil, tk_errors.InvalidArg("invalid email")
    }
}
```

```

    }

    user, err := s.db.User().GetByEmail(ctx, request.Email)
    // [...]
    // user in onboarding state will get an error; this situation opens up a
    small account enumeration
    if user.OrganizationID.IsNil() {
        return nil, tk_errors.InvalidArg("user has not finished onboarding;
        check for signup link")
    }
    // [...]
    return &dashboard_api.SendLoginMagicLinkResponse{}, nil
}

```

Figure 18.2: Excerpt of dashboard_api_server.go

(mono/src/go/coordinator/internal/dashboard/dashboard_api_server.go#L500–L550)

The risk associated with such oracles is deemed low and is only documented because this behavior circumvents the existing mitigation.

Exploit Scenario

An attacker wants to know if a given email address is associated with an account on the Turnkey platform. With this email address, they attempt a login, and then they use the sign-up form to force the creation of an account if it does not already exist. They then attempt to log in again on Turnkey:

- If both login attempts return “Verify your email to sign in,” this means the user already exists on Turnkey.
- If the first login attempt returns “Verify your email to sign in” and the second one returns “user has not finished onboarding; check for sign-up link,” the user does not exist on Turnkey.
- If both attempts return “user has not finished onboarding; check for sign-up link,” the user already has an account (which could have been created by a previous oracle attempt) and never completed the onboarding process.

Recommendations

Short term, refactor SendLoginMagicLink so that it never returns tk_errors.InvalidArg() errors. Instead, it should log the error with slog and respond with an empty SendLoginMagicLinkResponse to prevent the error from being displayed to the user, as is done for other error cases on this endpoint.

19. Discrepancy between Parser and Signer transaction parsing

Severity: **Informational**

Difficulty: **Medium**

Type: Denial of Service

Finding ID: TOB-TK3-19

Target: `mono/src/rust/`

Description

The Signer enclave assumes that the number of signatures (the first byte of a Solana transaction) is less than 20 (figure 19.1). The Parser enclave does not validate this property (figure 19.2).

```
// Ignore the first byte (indicator for num signatures), and all subsequent
signatures
let num_signatures = decoded_transaction[0] as usize;

// Max number of signatures is 19. An unsigned transaction has 0 signatures.
// Thus the range is [0, 19] (inclusive notation)
if num_signatures > 19 {
    return Err(TurnkeyError::new(
        Code::InvalidArgument,
        &format!("transaction has unexpected number of signatures:
{num_signatures:?}"),
    ));
}
```

Figure 19.1: Part of the Signer enclave
(`mono/src/rust/signer/app/src/routes/signing.rs#L126-L136`)

```
let num_signatures = unsigned_tx_bytes[0] as usize;
let signatures: Vec<Signature> = unsigned_tx_bytes[1..]
    .chunks_exact(LEN_SOL_SIGNATURE_BYTES)
    .take(num_signatures)
    .map(<[u8]>::to_vec)
    .collect();
```

Figure 19.2: Part of the Parser enclave
(`mono/src/rust/evm_parser/app/src/routes/solana.rs#L236-L241`)

Recommendations

Short term, check why the limitation was added in the Signer enclave and either remove the validation from the Signer, add the same validation in the Parser, or document the discrepancy.

Long term, add tests (possibly fuzzing harnesses) that check if a transaction parsed by the Parser is also accepted by the Signer. Add similar tests checking for other discrepancies, like those between the Parser and the Notarizer and between the Parser and the Ump.

20. Inconsistent enforcement of AllowedAuthMethods

Severity: **Medium**

Difficulty: **Low**

Type: Access Controls

Finding ID: TOB-TK3-20

Target: `mono/src/go/auth-proxy/internal/server/public_api_server.go`

Description

The coordinator dashboard provides functionality to disable alternative login methods, such as OTP. Disabling a login method in the dashboard exposes this setting through `cfg.AllowedAuthMethods` in the Auth Proxy, but it does not enforce the restriction uniformly across all authentication endpoints.

The `OAuthLogin` and `OAuth2Authenticate` methods correctly validate whether the OAuth authentication method is enabled in the proxy configuration before proceeding with authentication (figures 20.1, 20.2):

```
func (p *ProxyServer) OAuthLogin(ctx context.Context, req
*auth_proxy.OAuthLoginRequest) (*auth_proxy.OAuthLoginResponse, error) {
    proxyID := ctx.Value(middleware.ProxyIDKey).(string)
    cfg := ctx.Value(middleware.ProxyConfigKey).(*models.AuthProxyConfig)

    if !slices.Contains(cfg.AllowedAuthMethods, constants.AuthMethodOAUTH) {
        return nil, status.Error(codes.PermissionDenied, "OAUTH disallowed")
    }
}
```

Figure 20.1: Excerpt of `public_api_server.go`

(`mono/src/go/auth-proxy/internal/server/public_api_server.go#L436-L442`)

```
func (p *ProxyServer) OAuth2Authenticate(ctx context.Context, req
*auth_proxy.OAuth2AuthenticateRequest) (*auth_proxy.OAuth2AuthenticateResponse,
error) {
    proxyID := ctx.Value(middleware.ProxyIDKey).(string)
    cfg := ctx.Value(middleware.ProxyConfigKey).(*models.AuthProxyConfig)

    if !slices.Contains(cfg.AllowedAuthMethods, constants.AuthMethodOAUTH) {
        return nil, status.Error(codes.PermissionDenied, "OAUTH disallowed")
    }
}
```

Figure 20.2: Excerpt of `public_api_server.go`

(`mono/src/go/auth-proxy/internal/server/public_api_server.go#L661-L667`)

However, this verification is missing from the `OtpLogin`, `InitOtp`, `VerifyOtp`, and `Signup` methods. This allows users to authenticate using OTP-based methods even when administrators have disabled them through the coordinator dashboard.

This observation was confirmed on the staging instance by completing a full suborganization sign-up flow with `OTP_TYPE_EMAIL` on an Auth Proxy configured with OTP disabled. Note that we could not test whether disabling `OTP_TYPE_EMAIL` and/or `OTP_TYPE_SMS` is correctly enforced, as the latter requires an Enterprise plan, but this is likely to also arise as we have not identified such code in the Auth Proxy.

Exploit Scenario

An organization administrator configures the Auth Proxy through the coordinator dashboard to allow only OAuth authentication. The administrator intends to disable OTP-based authentication methods to enforce stronger authentication mechanisms and mitigate classes of social engineering attacks against their users, but this has no effect on the Auth Proxy. An attacker that is able to obtain a victim's OTP code—for instance, through phishing—will be able to obtain a verification token and sign up or log in on the victim's behalf.

Recommendations

Short term, add `cfg.AllowedAuthMethods` validation to all authentication-related endpoints. The `OtpLogin`, `InitOtp`, `VerifyOtp`, and `Signup` handlers should check whether the corresponding authentication method is enabled.

On OTP-based endpoints, add a similar validation of the OTP method (`OTP_TYPE_EMAIL` or `OTP_TYPE_SMS`) to enforce the administrator's authentication policy consistently across all endpoints.

Long term, implement a centralized authentication method validation middleware that executes before all handlers. Based on `cfg.AllowedAuthMethods`, the middleware should reject requests to endpoints associated with disabled authentication methods. Additionally, automate end-to-end testing to verify the enforcement of this setting on all authentication endpoints; this will prevent future regressions when new methods are added.

21. Desynchronization between Ump intents allowlist and Auth Proxy configuration

Severity: Medium

Difficulty: High

Type: Access Controls

Finding ID: TOB-TK3-21

Target: `mono/src/rust/ump/app/src/routes/decider.rs`

Description

The Ump service enforces authorization for Auth Proxy requests using a set allowlist of allowed activity types (figure 21.1), while the coordinator dashboard provides administrators with granular control over authentication methods through the AllowedAuthMethods configuration field (figure 21.2).

```
/// Returns true if the intent is one of the "auth proxy" intents, update this when
new proxy necessary activities are added.
fn auth_proxy_allowed_intent(intent: &IntentEnum) -> bool {
    use IntentEnum::{
        CreateOauthProvidersIntent, CreateSubOrganizationIntentV7, InitOtpIntent,
        InitUserEmailRecoveryIntent,
        Oauth2AuthenticateIntent, OauthLoginIntent, OtpLoginIntent, VerifyOtpIntent,
    };
    matches!(
        intent,
        InitUserEmailRecoveryIntent(_)
        | InitOtpIntent(_)
        | VerifyOtpIntent(_)
        | OtpLoginIntent(_)
        | CreateSubOrganizationIntentV7(_)
        | CreateOauthProvidersIntent(_)
        | OauthLoginIntent(_)
        | Oauth2AuthenticateIntent(_)
    )
}
```

Figure 21.1: Excerpt of `decider.rs`

(`mono/src/rust/ump/app/src/routes/decider.rs#L1059-L1076`)

```
// AuthProxyConfig maps to the auth_proxy_config table.
type AuthProxyConfig struct {
    ID                                uuid.UUID          `db:"id"`
    OrganizationID                    uuid.UUID
    `db:"organization_id"`
    AllowedOrigins                    []string
    `db:"allowed_origins"`
}
```

```

    AllowedAuthMethods []string
    `db:"allowed_auth_methods"`
    EncryptedApiKey     string
    `db:"encrypted_api_key"`
    // [...]
    OtpAlphanumeric     *bool
    `db:"otp_alphanumeric"`
    OtpLength            *int           `db:"otp_length"`
    UpdatedAt            time.Time      `db:"updated_at"`
    CreatedAt            time.Time      `db:"created_at"`
}

```

*Figure 21.2: Excerpt of auth_proxy_config.go
(mono/src/go/pkg/models/auth_proxy_config.go#L10-L33)*

These two authorization mechanisms operate independently and can become desynchronized, creating a privilege escalation risk where compromised Auth Proxy API keys retain broader permissions than administrators intend.

Exploit Scenario

An organization administrator configures the Auth Proxy to permit only OAuth authentication, explicitly removing OTP_TYPE_EMAIL and OTP_TYPE_SMS from the configuration through the coordinator dashboard to mitigate classes of social engineering attacks against its users. An attacker compromises the Auth Proxy API keys through a separate vulnerability. The attacker uses the stolen API keys to directly submit InitOtpIntent, VerifyOtpIntent, and OtpLoginIntent activities to the Ump service. The Ump service authorizes these intents, bypassing the administrator's configuration. The attacker successfully authenticates users via OTP despite the administrator's explicit configuration to disable this authentication method.

Recommendations

Short term, synchronize both access control logics by making the Ump service pull Auth Proxy settings in auth_proxy_allowed_intent() and compare the current intent with the allowed authentication methods.

Long term, introduce a way to scope API keys based on the associated public key to make it more generic and to reduce coupling with the Auth Proxy feature in particular.

22. Incomplete mitigation of user existence oracles via Auth Proxy signup

Severity: Informational

Difficulty: Low

Type: Error Reporting

Finding ID: TOB-TK3-22

Target: `src/go/auth-proxy/internal/server/public_api_server.go`

Description

The Auth Proxy allows unauthenticated attackers to determine whether specific email addresses and phone numbers belong to users within a given organization.

The `Signup()` endpoint extracts these attributes from unverified OIDC tokens (TOB-TK3-29) if they appear to come from Google. The `checkDup()` function then ensures that no suborganization already exists with the same attributes and returns an error otherwise (figure 22.1).

```
func (p *ProxyServer) Signup(ctx context.Context, req *auth_proxy.SignupRequest)
(*auth_proxy.SignupResponse, error) {
    proxyID := ctx.Value(middleware.ProxyIDKey).(string)
    cfg := ctx.Value(middleware.ProxyConfigKey).(*models.AuthProxyConfig)
    // [...]
    orgID := cfg.OrganizationID.String()

    // Infer email ONLY from Google OIDC providers; keep other providers
    untouched.
    if req.UserEmail == nil && req.OauthProviders != nil {
        for _, prov := range req.OauthProviders {
            if prov == nil || prov.OidcToken == "" {
                continue
            }

            _, _, iss, email, verified, err :=
oidc.ParseOidcToken(prov.OidcToken)
            if err != nil {
                return nil, status.Errorf(codes.InvalidArgument, "invalid
OIDC token: %v", err)
            }

            if verified && email != "" && helpers.IsGoogleIssuer(iss) {
                req.UserEmail = &email
                break
            }
        }
    }
    // [...]
```

```

if req.UserEmail != nil {
    if err := checkDup("EMAIL", *req.UserEmail); err != nil {
        return nil, err
    }
}

if req.UserPhoneNumber != nil {
    if err := checkDup("PHONE_NUMBER", *req.UserPhoneNumber); err != nil {
        return nil, err
    }
}
// [...]

```

Figure 22.1: Excerpt of public_api_server.go

(mono/src/go/auth-proxy/internal/server/public_api_server.go#L127-L275)

The Auth Proxy also does not validate OTP verification tokens before testing for the presence of the target user under the current organization (figure 22.2).

```

func (p *ProxyServer) OtpLogin(ctx context.Context, req *auth_proxy.OtpLoginRequest)
(*auth_proxy.OtpLoginResponse, error) {
    // [...]
    filterType, contact, err :=
helpers.ExtractContactFromJWT(req.VerificationToken)
    if err != nil {
        return nil, status.Errorf(codes.InvalidArgument, "invalid token: %v",
err)
    }

    var subOrgID string
    if req.OrganizationId != nil && *req.OrganizationId != "" {
        // Use the org ID provided in the request
        subOrgID = *req.OrganizationId
    } else {
        // Fall back to lookup flow
        subOrgID, err = helpers.FetchVerifiedSubOrgID(ctx, p.dbh, proxyID,
p.target, cfg.OrganizationID.String(), filterType, contact, p.signer)
        if err != nil {
            if errors.Is(err, helpers.ErrNoVerifiedSubOrg) {
                return nil, status.Error(codes.NotFound, "account not
found")
            }

            return nil, status.Errorf(codes.Internal, "lookup failed: %v",
err)
        }
    }
}

```

Figure 22.2: Excerpt of public_api_server.go

(mono/src/go/auth-proxy/internal/server/public_api_server.go#L380-L403)

Finally, the Auth Proxy handler `GetAccount()` can act as an existence oracle based on most of the user's details (EMAIL, PHONE_NUMBER, IDC_TOKEN, CREDENTIAL_ID, PUBLIC_KEY, NAME, USERNAME). The Auth Proxy configuration directive "Verification Token Required for Account Lookups" (`VerificationTokenRequiredForGetAccountPii`) attempts to mitigate this type of oracle by requiring a signed token; however, the setting is disabled by default.

The risk associated with such oracles is deemed low and is documented only because this behavior circumvents existing mitigations.

Exploit Scenario

An attacker wants to know if a given email address is associated with an account on the Turnkey platform under a specific organization. With this email address, they attempt to sign up through the Auth Proxy without having to prove ownership of this email address:

- If the attempt returns "account with this EMAIL exists," this means the user already exists under this organization.
- Otherwise, the error "invalid verification_token: failed to verify token signature: token signature is invalid: crypto/ecdsa: verification error" is shown; this means the token cannot be verified later during user creation.

Recommendations

Short term, address **TOB-TK3-29** at the Auth Proxy level to limit attackers' ability to tamper with these tokens and to assert the current user's identity before any operation. Enable the "Verification Token Required for Account Lookups" feature by default for all organizations.

Long term, update the "Verification Token Required for Account Lookups" feature to restrict all other lookup means (IDC_TOKEN, CREDENTIAL_ID, PUBLIC_KEY, NAME, USERNAME) to prevent the associated existence oracles.

23. Defined types in IDLs can overflow available stack space

Severity: **Medium**

Difficulty: **Low**

Type: Denial of Service

Finding ID: TOB-TK3-23

Target: `src/rust/evm_parser/app/src/parsing_utilities/idl_parser.rs`

Description

The Solana IDL parser has a check against recursive defined types. The check is weak, as it does not follow defined types wrapped around standard types like `Array` or `Optional`. As a result, an IDL with a defined type that references itself via a standard type is not forbidden. Transaction parsing that tries to resolve instruction arguments with such a malicious IDL goes into an infinite recursive call loop and could fail with a stack overflow and panic.

Specifically, a recursive defined type wrapped in an `Array` of a small size causes a huge amount of recursive function calls, as the allocation budget is very slowly consumed (figure 23.1).

```
fn cycle_recursive_check(
    type_cache: HashMap<String, &IdlTypeDefinition>,
    type_name: &str,
    mut path: HashSet<String>,
) -> Result<(), TurnkeyError> {
    // Check to see whether max depth has been exceeded
    if path.len() > MAX_DEFINED_TYPE_DEPTH {
        return Err(TurnkeyError::new(
            [skipped]
        ));
    }

    // Check to see whether a cycle has been detected
    if path.contains(type_name) {
        return Err(TurnkeyError::new(
            [skipped]
        ));
    }
    path.insert(type_name.to_string());

    let ty_def = type_cache
        .get(type_name)
        .copied()
        .ok_or_else(|| TurnkeyError::new(Code::InvalidArgument, &format!("type {type_name} not found in IDL")))?;
```

```

match &ty_def.r#type {
  IdlTypeDefinitionType::Struct { fields } => {
    [skipped]
  }
  IdlTypeDefinitionType::Enum { variants } => {
    [skipped]
  }
  IdlTypeDefinitionType::Alias { value } => {
    if let IdlType::Defined(ddefined) = value {
      let type_name = ddefined.to_string();
      cycle_recursive_check(type_cache, &type_name, path.clone())?;
    }
    Ok(())
  }
}
}

```

Figure 23.1: Excerpt of `idl_parser.rs`

([mono/src/rust/evm_parser/app/src/parsing_utilities/idl_parser.rs#L922-L980](#))

The test in figure 23.2 can be used to confirm this issue.

```

#[test]
fn test_invalid_cyclic_check_for_deftypes() {
  let idl_malicious = r#"
  {
    "types": [
      {
        "name": "A",
        "type": {
          "kind": "alias",
          "value": {
            "array": [
              {"defined": "A"},
              1
            ]
          }
        }
      }
    ],
    "instructions": [
      {
        "name": "dangerousString",
        "accounts": [],
        "args": [
          {
            "name": "a",
            "type": {"defined": "A"}
          }
        ]
      }
    ]
  }
  "#
}

```

```

    ]
}";

// Instruction data: [discriminator] + nested lengths
let string_data = vec![
    77, 67, 244, 160, 33, 24, 166, 14, // Discriminator for 'dangerous_string'
    0, 0, 0, 1,
    65,
];

let idl = decode_idl_data(idl_malicious).unwrap();

// will crash with stack overflow
let instruction = find_instruction_by_discriminator(&string_data,
idl.instructions.clone()).unwrap();
let composite_err = parse_data_into_args(&string_data, &instruction, &idl)
    .unwrap_err()
    .message;
assert_eq!(composite_err, "failed to parse IDL argument with error: Memory
allocation exceeded maximum allowed budget while parsing IDL call data -- check your
uploaded IDL or call data");
}

```

Figure 23.2: Test confirming the risk of stack overflow in Solana instruction parsing with a malicious IDL

Exploit Scenario

Eve adds a malicious IDL into her organization. The IDL contains a recursive defined type. Eve then repeatedly sends transaction parsing requests to the Parser enclave, causing it to crash. Turnkey users cannot use the system for the duration of the attack.

Recommendations

Short term, improve the `cycle_recursive_check` method so that it validates defined types wrapped into standard types.

Also consider improving the budget calculation to decrease the available budget by some predefined minimal amount for small allocations, so that an allocation of less bytes than the threshold results in a budget decrease equal to the threshold.

24. Parser enclave bug allows anything to be signed as a Solana IDL

Severity: **Medium**

Difficulty: **High**

Type: Data Validation

Finding ID: TOB-TK3-24

Target: `mono/src/rust/evm_parser/app/src/routes/solana.rs`

Description

The `evm_parser` does not validate Solana IDL parsing requests because of a typo in the code that causes validation errors to be discarded (figure 24.1). The enclave will sign empty requests and requests containing an arbitrary IDL (figures 24.2, 24.3).

```
// Decode the IDL to see if it works correctly
let _ =
idl_parser::decode_idl_data(&parse_request.smart_contract_interface).map_err(|e| {
    TurnkeyError::new(
        Code::InvalidArgument,
        &format!("Failed to validate Solana IDL with error: {}", e.message),
    )
});
```

Figure 24.1: Validation errors are discarded due to a missing `?` operator.
([mono/src/rust/evm_parser/app/src/routes/solana.rs#L85](#)).

```
grpcurl -plaintext -d '{"type":2}' localhost:41178
services.parser.v1.ParserService/ValidateSmartContractInterface
{
  "smartContractInterface": {
    "payload": {
      "smartContractInterfaceId": "50ba2351-1b19-410c-a91d-9a25fac7c359",
      "type": "SMART_CONTRACT_INTERFACE_TYPE_SOLANA"
    },
    "signature": { ... }
  }
}
```

Figure 24.2: An empty request to `evm_parser` that will be signed

```
grpcurl -plaintext -d '{"type":2, "smart_contract_interface": "anything here"}'
localhost:41178 services.parser.v1.ParserService/ValidateSmartContractInterface
{
  "smartContractInterface": {
    "payload": {
      "smartContractInterfaceId": "e0a57411-78a3-49bd-ad62-08c7e01a2ebe",
      "smartContractInterface": "anything here",
      "type": "SMART_CONTRACT_INTERFACE_TYPE_SOLANA"
    }
  }
}
```

```

    },
    "signature": { ... }
}

```

Figure 24.3: A request to `evm_parser` with an invalid IDL that will be signed

Exploit Scenario

An attacker uses the weak validation to sign an obviously invalid Solana IDL. Other parts of the system assume that the Parser already validated the IDL structure and use crashing functions to assert that. The attacker is able to trigger a crash with the invalid signed payload.

Recommendations

Short term, add the missing `?` operator to the code shown in figure 24.1 to ensure the validation errors are not discarded.

Long term, add more end-to-end test coverage. Also, consider using a type system as a defense-in-depth measure to catch similar issues. If the variable were explicitly typed as `_ : idl_parser::Idl`, the compiler would reject this program and catch the missing unwrap error (figure 24.4).

```

error[E0308]: mismatched types
--> evm_parser/app/src/routes/solana.rs:80:30
   |
80 |         let _ : idl_parser::Idl = idl_parser::decode_idl_data(&parse_request....
   |         -----^
   |         |
   |         expected due to this
81 |         TurnkeyError::new(
82 |             Code::InvalidArgument,
83 |             &format!("Failed to validate Solana IDL with error: {}", e.m...
84 |         )
85 |     });
   |     ^-----^ expected `Idl`, found `Result<Idl, TurnkeyError>`
   |
   = note: expected struct `Idl`
             found enum `std::result::Result<Idl, errors::TurnkeyError>`
help: use the `?` operator to extract the `std::result::Result<Idl,
errors::TurnkeyError>` value, propagating a `Result::Err` value to the caller
85 |     }?);

```

Figure 24.4: Rust's type system catching the missing error propagation bug

25. ValidateSmartContractInterface does not validate addresses

Severity: **Informational**

Difficulty: **High**

Type: Data Validation

Finding ID: TOB-TK3-25

Target: `mono/src/rust/evm_parser/app/src/service.rs`

Description

The `services.parser.v1.ParserService/ValidateSmartContractInterface` endpoint does not validate Ethereum or Solana addresses, and it accepts any string (figure 25.1).

```
grpcurl -plaintext -d '{"smart_contract_interface": "...", "smart_contract_address":  
" ", "type":1}' localhost:41178  
services.parser.v1.ParserService/ValidateSmartContractInterface  
{  
  "smartContractInterface": {  
    "payload": {  
      "smartContractInterfaceId": "5dd5a927-980a-408f-aa30-a7ad8027be01",  
      "smartContractAddress": " ",  
      "smartContractInterface": "...",  
      "type": "SMART_CONTRACT_INTERFACE_TYPE_ETHEREUM"  
    },  
    "signature": { ... }  
  }  
}
```

Figure 25.1: A request to evm_parser with an invalid address that will be signed

Recommendations

Short term, add validation of the Ethereum and Solana addresses.

Long term, expand the test suite to include cases in which invalid addresses are passed.

26. Ethereum int256 type is not supported

Severity: Low

Difficulty: Medium

Type: Denial of Service

Finding ID: TOB-TK3-26

Target: src/rust/

Description

Ethereum smart contracts support signed and unsigned integers of 256 bits, with `int` and `uint` types defaulting to this bit-size. The Parser enclave stores all integers as strings. The Ump enclave parses numeric strings as either signed 128-bit or unsigned 256-bit integers. The Ump enclave does not support signed 256-bit integers. This limitation results in some valid transactions being denied by the Ump due to a parsing error.

The limitation is likely known, as Turnkey documents the types supported in the policy engine and the documentation does not mention signed 256-bit types. However, lack of explicit documentation about this Turnkey/Ethereum discrepancy may cause confusion for users.

```
pub enum Kind {
    #[prost(enumeration = "NullValue", tag = "1")]
    NullValue(i32),
    #[prost(double, tag = "2")]
    FloatValue(f64),
    /// uses string protos to support i128 and u256
    #[prost(string, tag = "3")]
    IntValue(::prost::alloc::string::String),
    #[prost(string, tag = "4")]
    StringValue(::prost::alloc::string::String),
    #[prost(bool, tag = "5")]
    BoolValue(bool),
    #[prost(message, tag = "6")]
    StructValue(StructValue),
    #[prost(message, tag = "7")]
    ListValue(ListValue),
}
```

Figure 26.1: Parser's struct holds integers as strings.

([mono/src/rust/gen/src/gen/private/parser.v1.rs#L330-L346](#))

```
Token::Int(num) | Token::Uint(num) => ArgType {
    kind: Some(Kind::IntValue(num.to_string())),
},
```

Figure 26.2: The `evaluate_contract_arg` method of the Parser enclave
([mono/src/rust/evm_parser/app/src/routes/evm.rs#L477-L479](#))

```
Some(Kind::IntValue(i)) => {
    // Try to parse as i128 first
    if let Ok(val) = i.parse::<i128>() {
        Ok(Value::Int(val))
    } else {
        // Try to parse as U256
        match U256::from_dec_str(i) {
            Ok(val) => Ok(Value::UnsignedInt(val)),
            Err(_) => Err(EvaluatorError::MissingKeyword(
                "cannot parse smart contract argument as i128 or U256".to_string(),
            )),
        }
    }
}
```

Figure 26.3: Ump evaluates strings as i128 and u256, but not as i256.
([mono/src/rust/ump/app/src/evaluator.rs#L965-L978](#))

Exploit Scenario

Bob adds a smart contract interface with an `int` argument to his organization. He repeatedly tries to sign a transaction to call this smart contract with a huge negative number. Turnkey rejects these attempts since this argument type is not supported. Bob is annoyed.

Recommendations

Short term, explicitly warn in the public documentation that large signed integers are not supported by the Turnkey Policy Engine, despite the fact that Ethereum does support them.

Long term, consider adding support for signed 256-bit integers to the Turnkey Policy Engine.

27. System account Auth Proxy user can be made root user

Severity: **Undetermined**

Difficulty: **High**

Type: Access Controls

Finding ID: TOB-TK3-27

Target: `mono/src/rust/organization/src/mutations/root.rs`,
`mono/src/rust/organization/src/mutations/users.rs`

Description

The Auth Proxy service account is designed as a limited-privilege system account that should only perform authentication-related operations on behalf of end users (figure 27.1).

```
/// Returns true if the intent is one of the "auth proxy" intents, update this when
new proxy necessary activities are added.
fn auth_proxy_allowed_intent(intent: &IntentEnum) -> bool {
    use IntentEnum::{
        CreateOauthProvidersIntent, CreateSubOrganizationIntentV7, InitOtpIntent,
        InitUserEmailRecoveryIntent,
        Oauth2AuthenticateIntent, OauthLoginIntent, OtpLoginIntent, VerifyOtpIntent,
    };
    matches!(
        intent,
        InitUserEmailRecoveryIntent(_)
        | InitOtpIntent(_)
        | VerifyOtpIntent(_)
        | OtpLoginIntent(_)
        | CreateSubOrganizationIntentV7(_)
        | CreateOauthProvidersIntent(_)
        | OauthLoginIntent(_)
        | Oauth2AuthenticateIntent(_)
    )
}
```

Figure 27.1: Excerpt of `decider.rs`

(`mono/src/rust/ump/app/src/routes/decider.rs#L1059-L1076`)

When enabling the Auth Proxy, the system creates a dedicated service account with a single API key and keeps track of the associated `uuid` field in `FeatureName::AuthProxy` (figure 27.2).

```
pub fn enable_auth_proxy(
    data: &mut OrganizationData,
    _intent: &EnableAuthProxyIntent,
    auth_proxy_public_key: &P256Public,
    timestamp: Timestamp,
```

```

) -> Result<(ResultEnum, Hazmat), MutationError> {
    let mut user_ids = vec![];
    let mut api_keys = vec![];

    let target_public_key =
qos_hex::encode(&auth_proxy_public_key.to_bytes()[..65]);
    let (api_key, encrypted_private_key) = generate_api_key(
        &timestamp,
        "Auth Proxy API Key",
        CredentialType::ApiKeyP256,
        &target_public_key,
        None,
    )?;

    api_keys.push(api_key);

    let new_user = User {
        uuid: Uuid::new_v4().to_string(),
        alias: "Auth Proxy User".to_string(),
        email: None,
        api_keys,
        // [...]
    };

    let user_id = new_user.uuid.clone();
    user_ids.push(user_id.clone());
    data.users.push(new_user);

    data.set_feature(FeatureName::AuthProxy, Some(user_ids[0].clone()));

    Ok((
        ResultEnum::EnableAuthProxyResult(EnableAuthProxyResult {
            user_id: user_ids[0].clone(),
        }),
        encrypted_private_key,
    ))
}

```

Figure 27.2: Excerpt of `users.rs`

([mono/src/rust/organization/src/mutations/users.rs#L525-L571](#))

However, nothing prevents the Auth Proxy service account from being added to the organization root quorum. The validation logic only verifies that the specified user IDs exist in the organization and that the threshold is valid; it does not prevent service accounts from being added to the root quorum.

```

/// updates the root quorum
pub fn update(data: &mut OrganizationData, intent: &UpdateRootQuorumIntent) ->
Result<ResultEnum, MutationError> {
    data.root_quorum = Some(Quorum {
        threshold: intent.threshold,
        user_ids: intent.user_ids.clone(),
    })
}

```

```
});  
  
Ok(ResultEnum::UpdateRootQuorumResult(UpdateRootQuorumResult {}))  
}
```

Figure 27.3: Excerpt of root.rs
(mono/src/rust/organization/src/mutations/root.rs#L81-L89)

The root quorum grants users elevated privileges to approve sensitive organizational operations.

Exploit Scenario

An organization administrator, unfamiliar with the security implications of root quorum membership, grants the Auth Proxy user elevated permissions. An attacker later compromises the Auth Proxy API keys through a vulnerability in the service. The attacker uses the compromised API keys to submit sensitive activities that require root quorum approval, such as modifying organization policies, creating new administrative users under their control, or exfiltrating encrypted private keys without approval from legitimate administrators.

Recommendations

Short term, update the `UpdateRootQuorumIntent` handler to reject attempts to add the Auth Proxy service account to the root quorum by checking whether any user ID in the intent corresponds to the one stored `FeatureName::AuthProxy`. This will prevent the issue because it will enforce a policy decision that service accounts should not hold root quorum privileges.

Long term, reduce coupling with the Auth Proxy feature by introducing a user attribute distinguishing the various kinds of identities (human operators, service accounts, etc.) to allow restriction of privileges based on account types.

28. Potential race condition while disabling the Auth Proxy feature

Severity: Low

Difficulty: High

Type: Timing

Finding ID: TOB-TK3-28

Target: `mono/src/rust/organization/src/mutations/users.rs`,
`mono/src/rust/ump/app/src/routes/decider.rs`

Description

Disabling the Auth Proxy setting sends the activity `ACTIVITY_TYPE_DISABLE_AUTH_PROXY`, which first removes the feature from the organization settings and then removes the associated Auth Proxy user service account (figure 28.1).

```
/// disable user initiated auth
pub fn disable_auth_proxy(
    data: &mut OrganizationData,
    _intent: &DisableAuthProxyIntent,
    _timestamp: Timestamp,
) -> Result<ResultEnum, MutationError> {
    let user_id_opt = data.get_feature(FeatureName::AuthProxy);

    // Always remove the feature regardless of user existence
    data.remove_feature(FeatureName::AuthProxy);

    if let Some(user_id) = user_id_opt {
        if let Some(val) = &user_id.value {
            data.users.retain(|u| u.uuid != *val);
        } else {
            return Err(MutationError::UserNotFound(
                "No user ID in AuthProxy feature".to_string(),
            ));
        }
    } else {
        return Err(MutationError::UserNotFound("No AuthProxy user
found".to_string()));
    }

    Ok(ResultEnum::DisableAuthProxyResult(DisableAuthProxyResult {}))
}
```

Figure 28.1: Excerpt of `users.rs`
(`mono/src/rust/organization/src/mutations/users.rs#L573-L597`)

The Ump enclave uses the UUID stored in the `FeatureName::AuthProxy` setting to determine if a request is coming from the Auth Proxy user account and to restrict its intents.

```
// If the auth proxy user is trying to perform an action outside
// of its expected actions, we deny it explicitly _even if it's explicitly
// allowed by org policies. This might be too strong a measure:
// we can lift this later if needed, and err on the side of caution for now.
if consensus.is_auth_proxy_request() &&
!auth_proxy_allowed_intent(&consensus.get_intent()) {
    return true;
}
```

Figure 28.2: Excerpt of `decider.rs`

(`mono/src/rust/ump/app/src/routes/decider.rs#L1085-L1091`)

```
/// Returns true if the current request is an auth proxy request.
pub fn is_auth_proxy_request(&self) -> bool {
    match self.voting_organization.get_feature(FeatureName::AuthProxy) {
        Some(feature) => match &feature.value {
            Some(proxy_uuid) => self.current_request.voter_uuid == *proxy_uuid,
            None => false, // or panic/log error if value should *always* exist
        },
        None => false,
    }
}
```

Figure 28.3: Excerpt of `consensus.rs`

(`mono/src/rust/user/src/consensus.rs#L264-L273`)

However, since the feature setting is cleared before the Auth Proxy user account is removed, this creates a short time window during which the account still exists and has no intent restriction.

Exploit Scenario

An attacker compromises the Auth Proxy API keys through a vulnerability in the service. The attacker uses the compromised API keys to repeatedly submit non-allowlisted intents in parallel. If the administrator decides to disable the Auth Proxy feature, one of these intents could pass.

Note that we were unable to conduct this scenario on a live instance within the time allotted for the assessment.

Recommendations

Short term, update `disable_auth_proxy()` to first remove the service account and then disable the Auth Proxy user at the organization level.

29. Use of unverified signed tokens

Severity: Low

Difficulty: Low

Type: Cryptography

Finding ID: TOB-TK3-29

Target: `mono/src/go/auth-proxy/internal/server/public_api_server.go`

Description

As part of the available authentication flows on the Auth Proxy, users have to provide a signed token to Auth Proxy endpoints like `Signup()` or `GetAccount()` to prove their successful authentication at the previous step:

- After an OTP flow, the Coordinator signs a `verificationToken`, with the user's contact (email address, phone number, and the OTP method used).
- After an OAuth flow, the external provider (e.g., Google) signs an OIDC ID token (`IdcToken`), which contains information about the user's email address and a nonce field derived from their public key, effectively binding this key pair to the token.

The Auth Proxy generally extracts information from these tokens without first verifying their signature. While the downstream components (i.e., enclave code) are more consistent in their handling of signed tokens, this creates validation gaps where none of the components may verify the signature of a given token.

This issue was not proven to enable bypass of access controls at the Auth Proxy level.

```
// extract contact info from JWT payload
func ExtractContactFromJWT(token string) (filterType, contact string, err error) {
    parts := strings.Split(token, ".")
    if len(parts) < 2 {
        return "", "", fmt.Errorf("malformed JWT")
    }

    payloadRaw, err := base64.RawURLEncoding.DecodeString(parts[1])
    if err != nil {
        return "", "", err
    }

    var payload map[string]any
    if err := json.Unmarshal(payloadRaw, &payload); err != nil {
        return "", "", err
    }
}
```

```

contactVal, ok := payload["contact"].(string)
if !ok || contactVal == "" {
    return "", "", fmt.Errorf("contact claim missing")
}

switch payload["verification_type"] {
case constants.OTPTYPEEmail:
    return constants.FilterEmail, contactVal, nil
case constants.OTPTypesms:
    return constants.FilterPhoneNumber, contactVal, nil
default:
    return "", "", fmt.Errorf("unknown verification_type")
}
}

```

Figure 29.1: High-level helper skipping signature validation of verification tokens
([mono/src/go/auth-proxy/internal/helpers/auth_proxy_helpers.go#L34-L64](#))

```

func (p *ProxyServer) OtpLogin(ctx context.Context, req *auth_proxy.OtpLoginRequest)
(*auth_proxy.OtpLoginResponse, error) {
    proxyID := ctx.Value(middleware.ProxyIDKey).(string)
    cfg := ctx.Value(middleware.ProxyConfigKey).(*models.AuthProxyConfig)

    filterType, contact, err :=
helpers.ExtractContactFromJWT(req.VerificationToken)
    if err != nil {
        return nil, status.Errorf(codes.InvalidArgument, "invalid token: %v",
err)
    }
}

```

Figure 29.2: Use of this dangerous helper in the OtpLogin endpoint
([mono/src/go/auth-proxy/internal/server/public_api_server.go#L380-L387](#))

```

// Parse an encoded OIDC token into its parts and return (iss, aud, sub)
func ParseOidcToken(tokenString string) (string, string, string, string, bool,
error) {
    // Split the token into its components
    parts := strings.Split(tokenString, ".")
    if len(parts) != 3 {
        return "", "", "", "", false, tk_errors.New(codes.InvalidArgument,
"invalid token format", v1.TurnkeyErrorCode_INVALID_OIDC_TOKEN)
    }
    // Decode the middle part which is the payload
    payload, err := encoding.DecodeBase64URLEncodedStringToBytes(parts[1])
    if err != nil {
        return "", "", "", "", false, err
    }

    // Unmarshal the payload into a map
    var claims map[string]interface{}
    if err := json.Unmarshal(payload, &claims); err != nil {
        return "", "", "", "", false, err
    }
}

```

```

    }

    // [...]
    return aud, sub, iss, email, email_verified, nil
}

```

Figure 29.3: High-level helper skipping validation of OIDC ID tokens
([mono/src/go/pkg/oidc/oidc.go#L160-L208](#))

```

func (p *ProxyServer) Signup(ctx context.Context, req *auth_proxy.SignupRequest)
(*auth_proxy.SignupResponse, error) {
    // [...]
    if req.UserEmail == nil && req.OauthProviders != nil {
        for _, prov := range req.OauthProviders {
            if prov == nil || prov.OidcToken == "" {
                continue
            }

            _, _, iss, email, verified, err :=
oidc.ParseOidcToken(prov.OidcToken)
            if err != nil {
                return nil, status.Errorf(codes.InvalidArgument, "invalid
OIDC token: %v", err)
            }

            if verified && email != "" && helpers.IsGoogleIssuer(iss) {
                req.UserEmail = &email
                break
            }
        }
    }
}

```

Figure 29.4: Use of this dangerous helper in the Signup() endpoint
([mono/src/go/auth-proxy/internal/server/public_api_server.go#L127-L166](#))

Exploit Scenario

An attacker can tamper with signed tokens to raise verbose error messages (TOB-TK3-30) and attempt to trigger other unintended behaviors of the Auth Proxy.

Recommendations

Short term, enforce strict validation of all signed tokens by the Auth Proxy.

Long term, deprecate all internal helpers that parse tokens without validating their signature, like ParseOidcToken or ExtractContactFromJWT in all components of the Turnkey platform.

30. Verbose technical error messages

Severity: Informational

Difficulty: Low

Type: Error Reporting

Finding ID: TOB-TK3-30

Target: N/A

Description

Turnkey services often return verbose technical error messages to end users, revealing information about their inner workings. Note that the examples below are not exhaustive; this is a systemic issue across all services.

When initiating an OTP-based authentication flow with an invalid organization identifier (figure 30.1), the end-user receives a verbose coordinator error (figure 30.2).

```
POST /v1/otp_login HTTP/2
Host: authproxy.preprod.turnkey.engineering
[...]

{
  "verificationToken": "[...]",
  "organizationId": "x"
}
```

Figure 30.1: New OTP-based authentication flow with an invalid organizationId

```
{
  "code": 404,
  "message": "coordinator error: {\\\"code\\\":5,\\\"message\\\":\\\"no organization found with given ID organizationId=x: invalid organization id (\\\\\\\\x\\\\\\\\)\\\",\\\"details\\\":[\\\"@type\\\":\\\"type.googleapis.com/errors.v1.TurnkeyErrorDetail\\\",\\\"turnkeyErrorCode\\\":\\\"ORGANIZATION_NOT_FOUND\\\"]},\\\"turnkeyErrorCode\\\":\\\"ORGANIZATION_NOT_FOUND\\\"}"
}
```

Figure 30.2: Verbose error received when initiating an authentication flow with an invalid organizationId

We identified another instance of verbose error messages while documenting [TOB-TK3-31](#). When an invalid OTP template is configured for the organization (i.e., a string that cannot be parsed as a UUID), `GetEmailTemplateContents()` then raises the internal SQL error every time a user initiates a new OTP authentication flow (figure 30.3).

```
{
  "code": 400,
  "message": "coordinator error: {\\\"code\\\":3,\\\"message\\\":\\\"failed to get email template contents: failed to fetch email template: failed to read email template: ERROR: invalid input syntax for type uuid: '\\\\\\\\x\\\\\\\\' (SQLSTATE 22P02)\\\",\\\"details\\\":[],\\\"turnkeyErrorCode\\\":\\\"\\\"}"
}
```

Figure 30.3: Internal error message raised during OTP authentication

Exploit Scenario

An attacker is looking for vulnerabilities in Turnkey services from a closed-box perspective. Verbose technical errors help them obtain information about the underlying technology stack to tailor their attacks (e.g., gRPC, PostgreSQL). These errors also help the attacker find when one of their test cases caused unexpected behavior (e.g., an SQL error may indicate the alteration of the semantics of the original query, hinting at a possible injection vector).

Recommendations

Short term, update high-level error handlers so that error details are not available to end users. Instead, log them internally and display the associated log record identifier for easier debugging and support.

31. ACTIVITY_TYPE_UPDATE_AUTH_PROXY_CONFIG does not restrict Enterprise-only features

Severity: Informational

Difficulty: Low

Type: Access Controls

Finding ID: TOB-TK3-31

Target:
`mono/src/go/coordinator/internal/dashboard/dashboard_api_server.go`

Description

On the organization dashboard, several Auth Proxy-related settings are reserved for organizations enrolled in an Enterprise plan: enablement of SMS-based OTPs, templates and variables, and so on.

This requirement is enforced only client-side, and organization administrators can tamper with the issued activity (ACTIVITY_TYPE_UPDATE_AUTH_PROXY_CONFIG) to change these settings even if they are not part of such plans.

Outside of business logic concerns, this increases the attack surface of the coordinator, as these features are not expected to be directly exposed to most customers.

The severity of this issue is marked as informational because the enterprise features of Turnkey are not in scope for this audit.

Recommendations

Short term, update `DashboardApiServer.processActivity` to validate ACTIVITY_TYPE_UPDATE_AUTH_PROXY_CONFIG activities and reject them if they include settings that should not be available to the current organization, based on the plan it is enrolled in.

Long term, review the implementation of all Enterprise-only features to ensure they are available only to the intended organizations.

32. Unreachable SQL injection in GetKeyAndWalletCountInRangeWithInterval

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TK3-32

Target: `mono/src/go/pkg/db/organization/organization.go`

Description

In `GetKeyAndWalletCountInRangeWithInterval`, the `interval` parameter is directly interpolated in a SQL query with `fmt.Sprintf` without prior validation against an allowlist (figure 32.1).

```
func GetKeyAndWalletCountInRangeWithInterval(
    ctx context.Context,
    dbc db.Querier,
    orgId string,
    after time.Time,
    until time.Time,
    interval string,
) (map[time.Time]int64, error) {
    // [...]
    query := fmt.Sprintf(`
    WITH grid AS (
        SELECT start_time,
               lead(start_time, 1, 'infinity') OVER (ORDER BY start_time) AS
end_time
        FROM (
            SELECT generate_series(@after::TIMESTAMP, @until::TIMESTAMP,
INTERVAL '1 %s') AS start_time
            ) sub
        )
    SELECT
        grid.start_time AS "%s",
        (
        [...]
    WHERE
        (organizations.id = @orgid OR parent_organization_id = @orgid);
`, interval, interval)

    rows, err := dbc.Query(ctx, query, pgx.NamedArgs{
        "after": after,
        "until": until,
        "orgid": orgId,
    })
}
```

Figure 32.1: Excerpt of `organization.go`

(`mono/src/go/pkg/db/organization/organization.go#L359-L431`)

The severity of this finding is marked as informational because we were not able to find cross-references to this function and it may not be reachable on production systems.

Exploit Scenario

If this backend endpoint ever starts calling this function with an attacker-controlled `interval` parameter, an attacker can perform a SQL injection attack to exfiltrate database records. The attacker could also use verbose technical error messages ([TOB-TK3-30](#)) to obtain information surrounding their injection point and perform an error-based attack to leak database records with the help of the error message.

Recommendations

Short term, replace the direct interpolation with query placeholders and pass `interval` in a `NamedArgs` map. Alternatively, if this code is not associated with existing or upcoming features, remove it.

Long term, strongly type these `interval` parameters to force their validation against an allowlist or enumeration before their use in queries. All SQL queries should be parametrized (regardless of whether the parameters are user-controlled, validated, etc.).

33. Lack of network isolation for Auth Proxy pods

Severity: Informational

Difficulty: High

Type: Configuration

Finding ID: TOB-TK3-33

Target: N/A

Description

By default, **Calico allows all ingress and egress traffic** if no network policy is attached to a pod:

Default allow means all traffic is allowed by default, unless otherwise specified. Default deny means all traffic is denied by default, unless explicitly allowed. Kubernetes pods are default allow, unless network policy is defined to specify otherwise.

Through discussions with Turnkey developers, we learned that the Auth Proxy pod does not have a specific network policy attached to either ingress or egress flows (figure 33.1).

```
$ kubectl --context=tk-prod -n tkhq-auth-proxy get pods
NAME                                READY   STATUS    RESTARTS   AGE
auth-proxy-64b44897bf-g5q4f        1/1     Running   0           17h
auth-proxy-64b44897bf-ggsjl        1/1     Running   0           17h
auth-proxy-64b44897bf-xqn5f        1/1     Running   0           17h

$ kubectl --context=tk-prod -n tkhq-auth-proxy get networkpolicies
No resources found in tkhq-auth-proxy namespace.
```

Figure 33.1: Auth Proxy pods and network policies directly attached to them

Based on IaC scripts defining Calico global network policies, all ingress and egress traffic is allowed between pods (figures 33.2, 33.3).

```
apiVersion: crd.projectcalico.org/v1
kind: GlobalNetworkPolicy
metadata:
  name: deny-aws-imds
spec:
  order: 2000 # Lets Kubernetes NetworkPolicy (runs with order 1000) add an Allow
  types:
    - Egress
  egress:
    - action: Log
      metadata: {}
      protocol: TCP
```

```

    destination:
      ports:
        - 80
        - 443
      nets:
        - 169.254.169.254/32
        - fd00:ec2::254/128
  - action: Deny
    metadata: {}
    protocol: TCP
    destination:
      ports:
        - 80
        - 443
      nets:
        - 169.254.169.254/32
        - fd00:ec2::254/128

```

Figure 33.2: Global network policy that denies all egress traffic to AWS IMDS services

```

# Once there is a `GlobalNetworkPolicy` then the default is Deny
apiVersion: crd.projectcalico.org/v1
kind: GlobalNetworkPolicy
metadata:
  name: default-allow
spec:
  # default order is last
  types:
    - Egress
  egress:
    - action: Allow

```

Figure 33.3: Global network policy that re-allows any other egress traffic

Recommendations

Short term, attach ingress and egress network policies to the Auth Proxy pod to isolate it from unrelated services. For instance, it should receive HTTP requests only from the reverse proxy and should communicate only with the public API and the PgBouncer instance.

Long term, review the network policies of other pods to confirm they have the expected and appropriate network isolation. Ultimately, the goal should be to enforce a deny-all policy by default on the whole cluster.

34. OIDC ID tokens can be replayed on other organizations because audience claims are not validated

Severity: Undetermined

Difficulty: High

Type: Authentication

Finding ID: TOB-TK3-34

Target: `mono/src/go/auth-proxy/internal/server/public_api_server.go`

Description

Organization administrators are intentionally able to access the details of all journalized activities performed on the Auth Proxy, including OIDC ID tokens from `ACTIVITY_TYPE_OAUTH_LOGIN` activities. The audience claims of the tokens are not validated by the Auth Proxy, enabling cross-organization attacks.

Unlike the tokens affected by [TOB-TK3-35](#), these tokens are tied to a specific public key from the beginning of the authentication flow, with their nonce claim verified to be `SHA256(public_key)` during authentication, but the associated public key is also available to administrators as part of the activity intent parameters.

OIDC ID tokens serve as proof of the user's identity and are not associated with a specific organization (as the identity provider is not aware of them), which allows malicious administrators to replay them before they expire in order to sign up or log in on behalf of a user in another Turnkey organization. This finding appears to have a limited impact thanks to the nonce claim, which binds the session to a key pair that is not under the malicious administrator's control.

We verified this finding by authenticating to <https://wallet.tx.xyz> via Google (client ID `776352896366-07enngvt2217cnq1ctf5a9ddcm1pv1sc.apps.googleusercontent.com`) and relaying the OIDC ID token to sign up and obtain a session on an unrelated organization (`b4a953c8-2e75-4d96-b2af-6f77a26c67e8`) configured with the invalid Google client ID `XXXXXX`.

Recommendations

Short term, update the associated code to reject all OIDC ID tokens whose audience claims (`aud` and `azp`) differ from the configured client ID configured for the Auth Proxy of the current organization, and to check the uniqueness of all configured client IDs across the Auth Proxy settings of all organizations.

Another mitigation would be to hash both the public key and the organization identifier in the nonce claim and verify that both match the parameters of the `ACTIVITY_TYPE_OAUTH_LOGIN` intent. Note that the nonce claim—which, by definition,

should be one-time-use, temporary data—may not be the best candidate for the storage of the proposed hash. Likely a new, unique claim should be added to the token instead.

35. Journalized OTP verification tokens can be front run

Severity: **Medium**

Difficulty: **Medium**

Type: Authentication

Finding ID: TOB-TK3-35

Target: `mono/src/go/auth-proxy/internal/server/public_api_server.go`

Description

On the coordinator dashboard, all activities under a given organization are visible in the Activities tab, allowing users with correct permissions to validate their authenticity based on public keys of all users involved in the vote, as well as the full intent and result fields.

OTP authentication via the Auth Proxy has three steps:

1. Request to `/v1/otp_init` with the OTP type (`OTP_TYPE_EMAIL`, `OTP_TYPE_SMS`) and the contact information. This creates an `ACTIVITY_TYPE_INIT_OTP` activity and returns the UUID of the new OTP flow.
2. Request to `/v1/otp_verify` with the OTP flow identifier from step 1 and the code received at the contact address or phone number. This creates an `ACTIVITY_TYPE_VERIFY_OTP` activity and returns a signed JWT with the authenticated user's identity. This token is referred to as `verificationToken`.
3. Request to `/v1/otp_login` with the verification token and the public key to use for the lifetime of the user's session. This issues an `ACTIVITY_TYPE_OTP_LOGIN` activity under the user's suborganization and grants them a session tied to this public key and organization.

Adversaries can intercept the verification token between steps 2 and 3, submitting it to `/v1/otp_login` before the user completes their OTP authentication flow. This would result in a new session with the attackers' key pair instead of the user's, whose authentication will fail.

These verification tokens can be used only once and cannot be replayed by adversaries; they have to consume the token faster than the victim. Note that the OAuth authentication flow is not affected by this issue, as the token is tied to a specific public key from the beginning via the nonce claim of the resulting OIDC ID token.

Exploit Scenario

An organization administrator enables OTP authentication on their Auth Proxy. An adversary, like a malicious user with sufficient permissions to read a user's activities or a

Turnkey employee, generates a new P-256 key pair offline and starts polling the API for new activities. As soon as the attacker notices an `ACTIVITY_TYPE_VERIFY_OTP` activity, they attempt to use the resulting `verificationToken` to complete a login flow with their public key before the user does. The adversary obtains a signed read-write session token with the user's identity and the attacker's public key.

Recommendations

Short term, associate OTP verification tokens to a specific public key in earlier stages of the OTP flow, like `/v1/otp_verify`. While this will not prevent adversaries from front running users and obtaining sessions on their behalf, these sessions would be tied to a key pair that is not under the attacker's control.

This field already exists in the Protocol Buffers message `VerifyOtpRequest` but is still optional ([PR tkhq/mono#4610](#)); it should be made mandatory.

Long term, review all other secrets that transit via the Auth Proxy to ensure they are tied to data outside of the control of the possible adversaries.

36. Verification tokens can be replayed outside of signup and login endpoints

Severity: Informational

Difficulty: High

Type: Access Controls

Finding ID: TOB-TK3-36

Target: `src/go/auth-proxy/internal/server/public_api_server.go`

Description

Organization administrators are intentionally able to access the details of all journalized activities, including verification tokens part of `ACTIVITY_TYPE_VERIFY_OTP` results and `ACTIVITY_TYPE_OTP_LOGIN` parameters. To exploit [TOB-TK3-35](#), administrators would have to consume such tokens before the victim to obtain a signed session with the victim's identity, as they are invalidated after first use with `consumeVerificationTokenId()` (`src/go/pkg/otp/otp.go`), removing the entry from Redis.

However, these tokens are still valid until their exp claim and can be replayed on other endpoints that verify their signature and expiration but do not rely on the Redis instance to determine if they have been consumed.

For instance, the handler `GetAccount()` in the Auth Proxy behaves this way when filtering based on email addresses or phone numbers (figure 36.1).

```
func (p *ProxyServer) GetAccount(ctx context.Context, req
*auth_proxy.GetAccountRequest) (*auth_proxy.GetAccountResponse, error) {
    proxyID := ctx.Value(middleware.ProxyIDKey).(string)
    cfg := ctx.Value(middleware.ProxyConfigKey).(*models.AuthProxyConfig)
    // [...]
    if (req.FilterType == "EMAIL" || req.FilterType == "PHONE_NUMBER") &&
        cfg.VerificationTokenRequiredForGetAccountPii {
        // [...]
        if err := helpers.VerifyVerificationToken(
            ctx,
            *p.otpVerificationPublicKey,
            *req.VerificationToken,
            req.FilterValue,
        ); err != nil {
            return nil, status.Errorf(codes.InvalidArgument, "invalid
verification_token: %v", err)
        }
    }
}
```

Figure 36.1: Excerpt of `public_api_server.go`

([mono/src/go/auth-proxy/internal/server/public_api_server.go#L542-L565](#))

This also opens the door to cross-tenant attacks, as all verification tokens are signed with the same key (OTP_VERIFICATION_PUBLIC_KEY from environment variables), regardless of the current organization.

Recommendations

Short term, and after addressing **TOB-TK3-29**, introduce a `jti` (JWT ID) claim to all verification tokens. When a new token is signed, the `jti` should be added to Redis and should be assigned an expiry equal to the token's `exp` claim. When a token is consumed and its signature is validated, tokens whose `jti` is not in Redis should be rejected; otherwise, they should be removed from Redis.

Long term, introduce per-organization JWT signing keys for verification tokens to prevent their replay across organizations.

37. Segwit bitcoin transactions vulnerable to fee overpayment attack

Severity: **Medium**

Difficulty: **Medium**

Type: Data Validation

Finding ID: TOB-TK3-37

Target: evm_parser

Description

The Parser enclave does not require previous witness UTXOs to be provided in the partially signed bitcoin transaction (PSBT). This enables fee overpayment attacks on witness (segwit) transactions. The root cause of this vulnerability is the fact that only the total amount spent by a transaction is committed to by the signature, not the amounts of specific UTXOs. The specific UTXOs must be explicitly provided in PSBT for the Parser to be able to validate their amounts.

A similar attack for legacy (non-witness) UTXOs is prevented in the `get_script_and_value_from_input` method. The same kind of validation is needed for the witness transactions. Note that **taproot transactions are not affected by this issue**, as the taproot signatures commit directly to all the amounts.

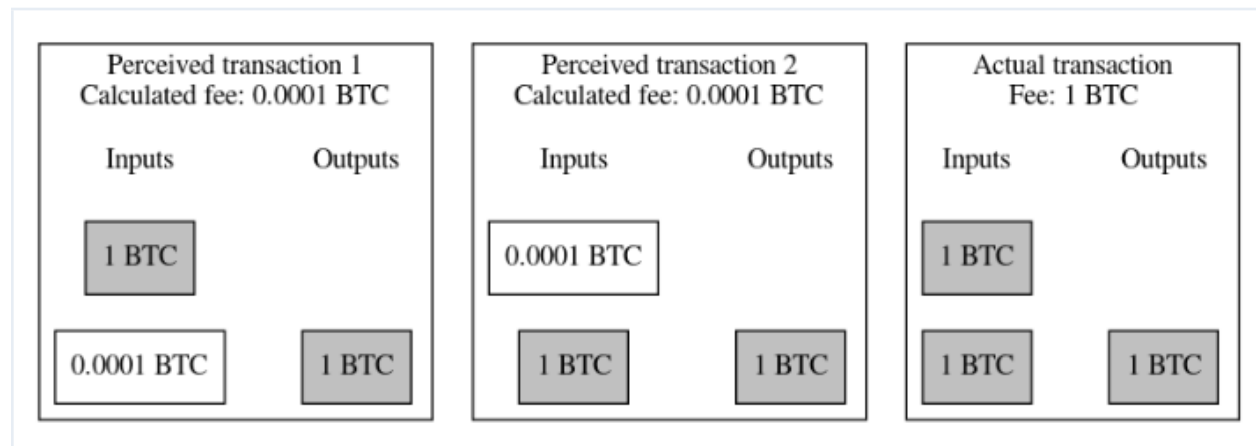


Figure 37.1: A graphical representation of the attack
(<https://bitcoinops.org>)

Exploit Scenario

An organization has a policy that limits bitcoin transactions by maximal allowed amount and fee.

An attacker (a user within an organization) sends two transactions to Turnkey that both transfer X BTC with fee Y BTC (where X is greater than Y). Both transactions are

semantically invalid (would not be accepted on-chain) as they reference invalid UTXOs. However, the transactions are syntactically valid and accepted by the Parser enclave. Both have an amount and fee value smaller than the values accepted by the policy, so both are signed by Turnkey. Four signatures are generated in total (two transactions times two UTXOs).

The attacker takes the signatures and recombines them to construct a single, semantically valid transaction that spends X BTC with an X BTC fee. The attacker bypasses the policy.

Recommendations

Short term, add validations for witness UTXOs in the `get_script_and_value_from_input` method to prevent the attack. The new validations will require users to provide full copies of the previous transactions in PSBTs. Note that this change will increase the size of PSBTs and that the client-side scripts to generate PSBT must support inclusion of full previous transactions.

Add unit tests for validation that the attack is not possible.

Long term, consider adding explicit support for bitcoin fees in the Policy Engine. Currently, users would have to write a policy that sums all inputs and subtracts all outputs to determine the fee. Review relevant BIPs to ensure that fees are correctly computed.

Note that the fee overpayment attack spans two (or more) transactions. It is possible that policies vulnerable to the attack are also vulnerable to a simpler “attack”: a malicious user simply sending multiple transactions with small amounts to the Turnkey system. The lack of temporal protections (policy rules spanning multiple separate transactions) requires further investigation to determine associated risks and mitigations.

References

- [Bitcoin Optech Newsletter #101](#)

38. Missing validations defined in BIP-0174

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-TK3-38

Target: `evm_parser/app/src/parsing_utilities/btc_sighash_creator.rs`

Description

BIP-174 defines a few validations that must be implemented when parsing PSBTs. Some of them are not implemented by Turnkey:

- If a `redeemScript` is provided, the `scriptPubKey` must be for that `redeemScript`.
- If a `witnessScript` is provided, the `scriptPubKey` or the `redeemScript` must be for that `witnessScript`.

Note that the `redeemScript` validation must be implemented for both witness and non-witness inputs (see the “[Simple Signer Algorithm](#)” section in the BIP).

The test case in figure 38.1 demonstrates that a PSBT violating the first bullet point above is successfully parsed, which is incorrect behavior.

```
#[test]
fn bip174_nonwitness_redeem_pubkey_mismatch() {
    let sign_with = Address {
        address: "2MtgN5EvHUm2kNVvqKgqsZ9v2fGH3jCpXVF".to_string(),
        format: AddressFormat::BitcoinTestnetP2sh as i32,
    };
    let psbt_hex =
        "70736274ff010071020000000158e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb
        2abdd7500000000000ffffffffff0270aaf00800000000160014d85c2b71d0060b09c9886aeb815e50991dd
        a124d00e1f5050000000016001400aea9a2e5f0f876a588df5546e8742d1d87008f0000000000100bb0
        200000001aad73931018bd25f84ae400b68848be09db706eac2ac18298abee71ab656f8b00000000484
        73044022058f6fc7c6a33e1b31548d481c826c015bd30135aad42cd67790dab66d2ad243b02204a1ced2
        604c6735b6393e5b41691dd78b00f0c5942fb9f751856faa938157dba01feffffff0280f0fa020000000
        017a9140fb9463421696b82c833af241c78c17ddbde493487d0f20a270100000017a91429ca74f8a08f8
        1999428185c97b5d852e4063f618765000000220202dab61ff49a14db6a7d02b0cd1fbb78fc4b18312b5
        b4e54dae4dba2fbfef536d7483045022100f61038b308dc1da865a34852746f015772934208c6d244543
        93cd99bdf2217770220056e675a675a6d0a02b85b14e5e29074d8a25a9b5760bea2816f661910a006ea0
        1010304010000000104475221029583bf39ae0a609747ad199addd634fa6108559d6c5cd39b4c2183f1a
        b96e07f2102dab61ff49a14db6a7d02b0cd1fbb78fc4b18312b5b4e54dae4dba2fbfef536d752af22060
        29583bf39ae0a609747ad199addd634fa6108559d6c5cd39b4c2183f1ab96e07f10d90c6a4f000000800
        00000800000080220602dab61ff49a14db6a7d02b0cd1fbb78fc4b18312b5b4e54dae4dba2fbfef536d"
```

```

710d90c6a4f000000800000008001000080000000";
    let result = parse_transaction(&psbt_hex, sign_with);
    assert!(result.is_err());
}

```

Figure 38.1: A test case that currently does not pass

The code in figure 38.2 could be added to the `get_script_and_value_from_input` method to make the test pass.

```

match input.redeem_script {
    Some(ref redeem_script) => {
        let hash = hash160::Hash::hash(&redeem_script.as_ref());
        if &hash[..] != prev_out.script_pubkey.as_bytes() {
            return Err(TurnkeyError::new(
                Code::InvalidArgument,
                &format!("non_witness_utxo redeem script mismatches scriptPubKey for
input {index}"),
            ));
        }
    }
    None => {}
}

```

Figure 38.2: Added validation for `get_script_and_value_from_input` (`non_witness_utxo` branch) that makes the test pass

The finding's severity is undetermined. Likely the issue does not pose any security risk; in the worst case scenario, an invalid transaction may be accepted and signed by Turnkey. However, we could not prove that the missing validations do not allow attackers to construct valid (in the on-chain sense) transactions that would be misinterpreted by the Parser enclave.

Recommendations

Short term, add missing validations in the bitcoin parsing code. Consult the "Simple Signer Algorithm" section in BIP-0174 for detailed guidance.

39. Supported versions of PSBTs are not validated explicitly and are not documented

Severity: Informational

Difficulty: High

Type: Undefined Behavior

Finding ID: TOB-TK3-39

Target: evm_parser

Description

Turnkey does not support version 2 of PSBT; only version 0 (also called version 1) is supported. This limitation **is also present in the bitcoin SDK used by Turnkey**.

The finding does not pose security risks, but this behavior is not publicly documented in Turnkey and may be surprising to users.

Moreover, there is no explicit validation of the received PSBT version in Turnkey. An update to the bitcoin SDK crate may silently introduce support for the newer version. This, in turn, may enable policy bypass if PSBTv2 enables features that are not handled or handled incorrectly by Turnkey code (which expects only old versions of PSBT).

Recommendations

Short term, document the lack of support for PSBT version 2. Add explicit version validation for received PSBTs in the Parser enclave, in the `parse_transaction` method of the `bitcoin.rs` file.

40. Lack of test vectors for BIP-0174 in the code

Severity: Informational

Difficulty: High

Type: Testing

Finding ID: TOB-TK3-40

Target: `vm_parser/app/src/routes/bitcoin.rs`

Description

The Parser enclave parses PSBTs according to BIP-0174. The BIP specifies a set of unit tests (test vectors) that are provided to ensure that conforming implementations work correctly. Turnkey does not implement these tests.

Note that some of the provided test vectors need to be modified to correctly test the intended property of the Parser. For example, the “redeemScript with witness UTXO does not match the scriptPubKey” case in the “Fails Signer checks” subsection has a PSBT that has both witness and non-witness UTXOs that will make the Parser reject the PSBT even before the redeemScript would be validated.

Recommendations

Short term, implement all test vectors from BIP-0174 in the Parser enclave. Ensure that all tests pass.

Long term, based on the test vectors, add new tests: ones that test the expected property if the original vector fails to do so, and ones that test additional properties that Turnkey expects the Parser to uphold (if any). Add the test vectors to the fuzz testing seed corpus. Also add test vectors from other BIPs (based on the data from official bitcoin tests).

41. Users can add invalid public keys

Severity: Informational

Difficulty: Low

Type: Cryptography

Finding ID: TOB-TK3-41

Target: `mono/src/go/auth-proxy/internal/server/public_api_server.go`

Description

We identified two cases where users can attach invalid public keys to their accounts without prior validation of their cryptographic soundness:

- When users complete their OTP authentication flow via the Auth Proxy, they can supply a `public_key` parameter that will be associated with their session. No validation is performed on the key outside of its hexadecimal encoding.
- When adding a new key via the “Create API key” feature on the coordinator dashboard, the user can import a public key they generated using the Turnkey CLI. The key only needs to start with 02 or 03, be a hexadecimal, and be 66 characters long.

This validation gap allows malformed or invalid public keys to propagate to other components, which could cause other bugs down the line.

Note that we were unable to explore this attack vector within the allotted time for this audit, as its potential impact appeared low compared to other findings.

Recommendations

Short term, implement cryptographic validation of public keys on the `ACTIVITY_TYPE_OTP_LOGIN` and `ACTIVITY_TYPE_CREATE_API_KEYS_V2` activities to ensure that the decoded point is on the expected curve.

Long term, review the entire API key creation flow(s) and ensure every component validates public keys before their storage and use.

42. Valid Solana IDLs are not accepted due to “types” key requirement

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-TK3-42

Target:

`mono/src/rust/evm_parser/app/src/parsing_utilities/idl_parser.rs`

Description

The Turnkey evm_parser enclave requires a “types” key to be present when validating Solana IDLs. There are valid Solana programs that generate IDLs without “types” keys, but they are rejected by the validation. An example of such a program is [this “Hello, World!” program](#).

We rated the severity of this finding as informational because we do not see any immediate security threat. Users could have trouble using the service due to this limitation; however, any real-world Solana program will likely generate an IDL with the “types” key present.

Recommendations

Short term, either drop the requirement that a “types” key be present, or document this fact so that users are aware of it.

43. Weak validation of Tron transactions

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-TK3-43

Target: `mono/src/rust/evm_parser/app/src/routes/tron.rs`

Description

As shown in the test in figure 43.1, the Tron transaction parser performs very little validation on transaction data and signs obviously invalid transactions. For instance, the expiration timestamp can be set to a negative number because it is represented with a signed integer (figure 43.1). We rated this finding's severity as informational because signing invalid transactions is not an explicit attack vector. However, given that there is some validation in place—for instance, for the `contract` field—the other fields should likely also be validated for obvious violations.

```
let raw = Raw {
    ref_block_bytes: [].into(),
    ref_block_num: -1,
    ref_block_hash: [].into(),
    expiration: -1,
    auths: vec![],
    data: vec![],
    contract: vec![contract],
    scripts: vec![],
    timestamp: -1,
    fee_limit: -1,
};

let mut raw_bytes = vec![];

assert!(raw.encode(&mut raw_bytes).is_ok());

let tron_parse_request = TronParseRequest {
    unsigned_payload: qos_hex::encode(&raw_bytes),
};

assert!(parse(tron_parse_request, &quorum_key).is_err());
```

Figure 43.1: A fragment of a test illustrating very little validation when parsing Tron transactions

Recommendations

Short term, add validation of the Tron transaction fields for obvious issues so that there is a consistent level of validation throughout the structure.

44. Bitcoin address type is not case insensitive in Policy Engine

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-TK3-44

Target: Ump enclave

Description

The BitcoinTxOutput address_type field in the Policy Engine is a case sensitive field, where the expected format is all-lowercase (e.g., p2pkh). However, it is more common to express bitcoin address types in all-uppercase (e.g., P2PKH). This may confuse users and easily lead to invalid policies being written.

Recommendations

Short term, make the BitcoinTxOutput address_type field case insensitive and document that fact in the public documentation.

Long term, review other supported fields of Bitcoin, Tron, Solana, and Ethereum transactions against case sensitivity, and document wherever that property is not obvious. Consider documenting even known case sensitivity properties for users' convenience. For example, BitcoinTxOutput addresses should be treated as case-sensitive strings. Document other properties like hexadecimal- or Base64-encoding of fields.

45. Risk of mix-up attacks on ciphertext in Auth Proxy

Severity: Low

Difficulty: High

Type: Cryptography

Finding ID: TOB-TK3-45

Target: `mono/src/go/auth-proxy/internal/signer/signer.go`

Description

The Auth Proxy database table stores organization IDs and Turnkey signer user IDs along with encrypted API keys (figure 45.1). The encrypted API key consists of an encapsulation public key (for the HPKE algorithm) and the ciphertext, where the corresponding plaintext is a 32-byte signing key (figure 45.2).

The ciphertext is not bound to the organization nor the signer user. An attacker with sufficient access to the database may swap rows in the database to make their Turnkey user use decrypted API keys belonging to different organizations.

```
type AuthProxyConfig struct {
    ID                uuid.UUID          `db:"id"`
    OrganizationID    uuid.UUID          `db:"organization_id"`
    AllowedOrigins    []string           `db:"allowed_origins"`
    AllowedAuthMethods []string           `db:"allowed_auth_methods"`
    EncryptedApiKey   string             `db:"encrypted_api_key"`
    TurnkeySignerUserID uuid.UUID          `db:"turnkey_signer_user_id" `
```

Figure 45.1: Excerpt of `auth_proxy_config.go`
(`mono/src/go/pkg/models/auth_proxy_config.go#L11-L17`)

```
func (s *AuthProxySigner) decryptApiKeyBundle(bundle string) ([]byte, error) {
    [skipped]

    recv := server.IntoEnclaveServerRecv()
    encBytes := enclave_encrypt.Bytes(uncompressedPub)
    ctBytes := enclave_encrypt.Bytes(ciphertext)
    msg := enclave_encrypt.ClientSendMsg{
        EncappedPublic: &encBytes,
        Ciphertext:     &ctBytes,
    }

    return recv.Decrypt(msg)
}
```

Figure 45.2: The `decryptApiKeyBundle` method of the Auth Proxy
(`mono/src/go/auth-proxy/internal/signer/signer.go#L83-L129`)

We were not able to determine the severity of the issue during the audit. Likely it is of low severity, as an attacker with enough privileges to exploit this issue is likely to be able to perform more severe attacks (like simply decrypting all ciphertexts by gaining access to the proxy configuration). Moreover, even in the exploit scenario below, it is possible that enclaves would stop the attack as the API key used by the attacker via the Auth Proxy would not match the organization ID provided by the Auth Proxy.

Nevertheless, we recommend fixing the issue to minimize the risks.

Exploit Scenario

An adversary with write access to the Auth Proxy database (the `auth_proxy_config` table specifically), but without access to the proxy configuration (the `ProxyTablePrivateKey` configuration specifically), swaps the `encrypted_api_key` rows of his organization with another organization. The attacker uses the Auth Proxy to access the Turnkey backend. The Auth Proxy decrypts the swapped `encrypted_api_key` ciphertext to the victim organization's API key. The Auth Proxy uses the API key to access the backend, resulting in an access control bypass.

Recommendations

Short term, bind encrypted API keys to organization ID and possibly to the Turnkey signer user ID to prevent mix-up attacks. This can be done by including the IDs in the ciphertexts and making the Auth Proxy validate the decrypted IDs against the expected ones, or by using additional authenticated data of GCM cipher.

46. Auth Proxy does not validate enclave signatures

Severity: Low

Difficulty: High

Type: Cryptography

Finding ID: TOB-TK3-46

Target: auth-proxy

Description

The Auth Proxy communicates with Nitro enclaves via public Turnkey infrastructure. All responses from the enclaves are signed. The Auth Proxy should verify the signatures to ensure that a malicious internal attacker (inside Turnkey infrastructure) has not tampered with the responses.

Because the Auth Proxy and public Turnkey infrastructure are in almost the same trust zone, the finding is only of low severity. However, if the Auth Proxy is further isolated from other components of the system, then the importance of enclave signature verification increases.

Exploit Scenario

An attacker compromises the Coordinator, but not the Auth Proxy. The attacker can return fake data to clients that opted in to use the Auth Proxy.

Recommendations

Short term, add signature verification for enclave responses to the Auth Proxy.

Long term, create documentation for the whole system from a cryptographic perspective that describes which components hold what type of keys and do what kind of signature verifications, encryptions, and so on. Then, use the documentation to determine gaps in the implemented cryptographic protocols.

47. Pre-production environment is incorrectly set up in the export-and-sign iframe

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TK3-47

Target: frames/export-and-sign/src/turnkey-core.js

Description

The `verifyEnclaveSignature` method defines two public keys, for the production and pre-production environments (figure 47.1). The latter is never used.

```
const TURNKEY_SIGNERS_ENCLAVES = {
  prod:
    "04cf288fe433cc4e1aa0ce1632feac4ea26bf2f5a09dcfe5a42c398e06898710330f0572882f4dbdf0f5304b8fc8703acd69adca9a4bbf7f5d00d20a5e364b2569",
  preprod:
    "04f3422b8afbe425d6ece77b8d2469954715a2ff273ab7ac89f1ed70e0a9325eaa1698b4351fd1b23734e65c0b6a86b62dd49d70b37c94606aac402cbd84353212",
};

// Use window.__TURNKEY_SIGNER_ENVIRONMENT__ if available (for testing), otherwise
// use the webpack replacement
const environment =
  (typeof window !== "undefined" && window.__TURNKEY_SIGNER_ENVIRONMENT__) ||
  "__TURNKEY_SIGNER_ENVIRONMENT__";
const TURNKEY_SIGNER_ENCLAVE_QUORUM_PUBLIC_KEY =
  TURNKEY_SIGNERS_ENCLAVES["prod"];
```

Figure 47.1: Excerpt of `turnkey-core.js`
(`frames/export-and-sign/src/turnkey-core.js#L330-L341`)

Recommendations

Short term, fix the signer key selection. Consider adding the keys to configuration files instead of in the code, and update the key selection so that the right key is selected during compilation (packaging) of the iframe instead of in the runtime.

Long term, add tests that cover signature verification in the pre-production environment.

48. The decryptedKey is not cleared from memory after expiration

Severity: Low

Difficulty: High

Type: Data Exposure

Finding ID: TOB-TK3-48

Target: frames/export-and-sign/src/event-handlers.js

Description

The decryptedKey variable in the export-and-sign iframe has an expiration time. After the expiration, the key is no longer usable, but it is not cleared from the memory. This unnecessarily exposes the key.

Exploit Scenario

A user exported his key with the export-and-sign iframe once, a long time ago. The key expired but was kept in browser memory. The browser crashes and a memory dump is generated. An attacker steals the memory dump and extracts the key from it.

Recommendations

Short term, have the export-and-sign iframe clear the decryptedKey variable when it expires. The iframe can call the existing onClearEmbeddedPrivateKey method for the task.

49. The decryptedKey is not removed from DOM after expiration and clear events

Severity: Low

Difficulty: High

Type: Data Exposure

Finding ID: TOB-TK3-49

Target: frames/export-and-sign/src/event-handlers.js

Description

The plaintext private key held in the decryptedKey variable in the export-and-sign iframe may expire or be cleared with an EMBEDDED_PRIVATE_KEY_CLEARED event. In such cases, the variable is zeroized. However, the key is persisted in the key-div DOM element via the displayKey function.

Exploit Scenario

A user exports his private key. After some time, the key expires. However, the key is still visible in the iframe DOM. An attacker steals the visible key by shoulder surfing.

Recommendations

Short term, update the export-and-sign iframe so that whenever the decryptedKey variable is zeroized (cleared), the data is removed from the key-div DOM element.

Long term, consider making the iframe not display plaintext private keys at all unless an explicit event is received from the parent website. This change would minimize key exposure. Also consider completely removing the display key functionality from the code so that the iframe can only generate signatures.

50. The decryptedKey is always displayed and is not masked on the UI level

Severity: Low

Difficulty: High

Type: Data Exposure

Finding ID: TOB-TK3-50

Target: frames/export-and-sign/src/event-handlers.js

Description

The plaintext private key held in the decryptedKey variable in the export-and-sign iframe is always displayed by default. The parent (customer's) website embedding the iframe may have no need to show users the keys, and may only need to use the iframe for signing. In such cases, the key is unnecessarily exposed both in the visible UI and in the browser memory.

Exploit Scenario

A customer's website uses the export-and-sign iframe to allow users to efficiently sign data. The website embeds the iframe but does not display iframe content.

An attacker finds an XSS vulnerability in the customer's website and uses it to make the iframe visible. The attacker uses some form of a cross-site leaks attack to steal the now visible private key.

Recommendations

Short term, update the export-and-sign iframe so that it does not call the displayKey function automatically. Instead, have it display the key only when explicitly requested by the parent iframe via a new message type. Change the display of the key to be a masked password instead of just plaintext.

Long term, consider completely removing the display key functionality from the code so that the iframe can only generate signatures.

51. Expiration time of decryptedKey is incorrectly calculated

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-TK3-51

Target: frames/export-and-sign/src/event-handlers.js

Description

The private key held in the decryptedKey variable expires in 86.4 seconds (approximately 1.44 minutes) instead of the expected 24 hours. This is because the time calculation does not take milliseconds modified into account.

Recommendations

Short term, change the DEFAULT_TTL_SECONDS variable to be equal to $1000 * 24 * 60 * 60$.

Long term, add tests for key expiration.

52. Redundant event listeners in export-and-sign

Severity: **Informational**

Difficulty: **High**

Type: Data Validation

Finding ID: TOB-TK3-52

Target: frames/export-and-sign

Description

The event listeners created by the export-and-sign iframe are difficult to reason about, and some are likely redundant.

The `index.js` file has two listeners:

- For the `DOMContentLoaded` event
- For the `TURNKEY_INIT_MESSAGE_CHANNEL` message

Both listeners call the `initEventHandlers` method, which creates the following listeners:

- Listeners for various click events (added by the `addDOMEventListeners` method)
- Listeners for messages handled by the `initMessageEventListener` method
- Listener for the `TURNKEY_INIT_MESSAGE_CHANNEL` message

The two listeners in the `index.js` file may collide.

The second listener in `index.js` may collide with the one created by the `initEventHandlers` method.

Recommendations

Short term, clear up the event listeners in the export-and-sign iframe. Dynamically (manually) test that only the expected listeners exist at every point in the iframe lifetime.

Long term, add tests for listener lifetimes in the export-and-sign iframe.

54. Single client website opened in multiple tabs cannot be used with the export-and-sign iframe

Severity: Informational

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-TK3-54

Target: frames/export-and-sign

Description

The export-and-sign iframe uses local storage to hold some data like temporary (“embedded”) keys. The local storage is shared between iframe instances if the iframes and top-level parent website have the same origins. This behavior means that if a user navigates a parent (customer’s) website in multiple browsers tabs simultaneously, then all the export-and-sign iframes loaded in the tabs will behave incorrectly.

Recommendations

Short term, consider replacing the local storage with session storage (which is not shared between tabs). This should fix the usability issues; however, further investigation is required to determine impact of this change on the overall system security (e.g., on the security of the HPKE protocol).

Long term, add tests for the behavior of iframes in multi-tab contexts.

55. The fromDerSignature function in the export-and-sign iframe is missing validations

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-TK3-55

Target: frames/export-and-sign/src/turnkey-core.js

Description

The fromDerSignature function is missing some validations for input DER data. The validations are not strictly necessary for security, but their absence may introduce some exploitable vectors in the future.

Here are some examples of missing validations on input DER data:

- The SEQUENCE tag is not checked to be 0x30.
- There are no checks for canonical (minimal) representation of DER-encoded numbers.
- Bounds for array slicings are not checked.

Moreover, not all important features and formats that DER supports are implemented:

- No support for short and long form of lengths
- No support for negative integers

Recommendations

Short term, improve the fromDerSignature function with more strict validations. Add support for possibly important DER encodings like long form of length fields.

Long term, add cross-library tests, possibly fuzzing tests, that validate the correctness of the fromDerSignature function.

56. Possible mitigations for supply chain attacks are not implemented

Severity: Informational

Difficulty: High

Type: Patching

Finding ID: TOB-TK3-56

Target: frames/export-and-sign/package.json

Description

The build and deployment process of the export-and-sign iframe was not audited. However, based on what we saw in the source code, we propose the following supply chain mitigations:

- Use the **minimumreleaseage** option to delay updates. This will help to prevent compromise by massive attacks on new dependency versions, assuming that the attack is spotted before the delay finishes. Note that implementing this feature may require adding monitoring for urgent security updates that may need to be applied before the delay passes.
- Specify the dependency versions exactly, instead of the minimal required versions (specified with the `^` operator). Use the **save-exact** and/or **save-prefix** option to do so.
 - The transitive dependencies may still be referenced via mutable versions. Consider using the **overrides** option and lockfiles to mitigate the impact of attacks on transitive dependencies.
- Disable lifecycle scripts with the **ignore-scripts** flag.
- Check **provenance attestations** for dependencies whenever such data is available. Use the `npm audit signatures` command to do so.

Recommendations

Short term, implement the mitigations described in this finding. Review the npm documentation for additional ideas about prevention of supply chain attacks that could work for the export-and-sign application and the build and deployment processes of Turnkey.

58. Dependencies of the export-and-sign iframe are not minimal

Severity: Informational

Difficulty: High

Type: Patching

Finding ID: TOB-TK3-58

Target: frames/export-and-sign/package.json

Description

The dependencies of the export-and-sign iframe can be minimalized. This improvement would make supply chain attacks less likely.

At least the following improvements can be made:

- The `tweetnacl-util` dependency can be removed, as it is imported only in `event-handlers.js` file but is never used.
- The `nacl` dependency is only used to sign messages, which should be possible to implement with the already imported `@noble/ed25519` package. If this package can be used for message signing, then the `nacl` package can be removed.
- The `solana/web3.js` package could be replaced with a selection of **more compact** `@solana/kits` packages.

Recommendations

Short term, minimize the dependencies of the export-and-sign iframe, as described in the finding.

59. Signer does not validate the number of private keys to be created

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TK3-59

Target: `mono/src/rust/signer/app/src/routes/private_keys.rs`

Description

The `routes::private_keys::create` endpoint in the Signer enclave does not validate the number of private keys to be created from the `key_params` vector. Notably, the vector can be empty or contain a very large amount of keys (figure 59.1).

While this issue alone is not necessarily a vulnerability, having a signed response with a pathological number of keys can facilitate exploitation of other bugs. The case of a large number of keys can be used to carry out a denial-of-service attack if key generation or the entropy source is slow. An empty response could be used to exploit a separate area of the code that might assume there is at least one key and panics otherwise. While we did not find such a specific place, we identified similar assumptions in [TOB-TK3-19](#), [TOB-TK3-43](#), and [TOB-TK3-60](#). This proves the importance of defense-in-depth mechanisms that could render a severe bug unexploitable.

```
let mut keys: Vec<SignerKey> = vec![];
for params in key_params {
    let key = create_key(organization.data.uuid.clone(), params, quorum_key)?;
    keys.push(key);
}
```

Figure 59.1: The `key_params` vector is user-controlled.

([mono/src/rust/signer/app/src/routes/private_keys.rs#L53-L57](#))

Recommendations

Short term, add validation to `routes::private_keys::create` to ensure that there is at least one key and that the number of keys does not exceed some reasonable amount.

Long term, audit Turnkey for similar issues and add extensive test suites to exercise pathological payload cases.

60. Signer crashes on empty transaction unsigned_payload

Severity: Medium

Difficulty: High

Type: Data Validation

Finding ID: TOB-TK3-60

Target: `mono/src/rust/signer/app/src/routes/signing.rs`

Description

The `routes::signing::sign_raw_payload` endpoint has an out-of-bounds vector read bug that causes a panic if an empty `unsigned_transaction` is sent. The length validation is in place, but it is after the read (figure 60.1).

The `unsigned_transaction` is not fully controllable by the user because it comes from a signed Ump ruling, which in turn relies on validation from the `evm_parser`. The `evm_parser` checks the length of the `unsigned_transaction`, and we could not find any path for exploiting the issue; for these reasons, we marked the difficulty of this finding as high. However, given the right bug in the `evm_parser` validation, this issue could become exploitable. We uncovered a similar bug, described in [TOB-TK3-24](#).

```
// Given a Solana transaction, we want to extract and sign the transaction message
specifically
let entire_transaction = &i.unsigned_transaction;

// 1. Hex-decode transaction
let decoded_transaction: Vec<u8> = qos_hex::decode(entire_transaction)?;

// 2. Remove signatures
// Ignore the first byte (indicator for num signatures), and all subsequent
signatures
let num_signatures = decoded_transaction[0] as usize;
...
let message_offset_bytes = 1 + num_signatures * 64;

// Sanity check: ensure we're inbounds
if decoded_transaction.len() < message_offset_bytes {
    return Err(TurnkeyError::new(Code::InvalidArgument, "invalid transaction
length"));
}
```

Figure 60.1: Reading the first byte of `decoded_transaction` without a prior check of its length ([mono/src/rust/signer/app/src/routes/signing.rs#L119-L143](#))

Exploit Scenario

An attacker uses a bug similar to the one described in [TOB-TK3-24](#) that allows him to bypass the `unsigned_transaction` structure validation in `evm_parser`. The attacker

performs a denial-of-service attack by repeatedly sending the signed intent with an empty `unsigned_transaction` to the Signer enclave, which crashes.

Recommendations

Short term, move the validation of the length of `decoded_transaction` to before the `num_signatures` value is read. Add a test case to ensure there are no regressions in the future.

Long term, improve the `fuzz target` code to make sure the fuzzer can reach the vulnerability. The effectiveness of this fuzzer is currently poor, as the fuzzer cannot pass the initial intent verification in `get_allowed_intent`.

61. Signer leaks malformed decrypted mnemonics

Severity: High

Difficulty: High

Type: Data Exposure

Finding ID: TOB-TK3-61

Target: `mono/src/rust/signer/app/src/routes/wallets.rs`

Description

The wallet import endpoint (`routes::wallets::import`) leaks the mnemonic in an error message if the mnemonic is an invalid UTF-8 bytestring (figure 61.1). The error message is constructed with a debug error message of the `e` error, which is of the type `FromUtf8Error`. The error message contains the whole `decrypted_bytes` vector (an example of the behavior is shown in figure 61.2). It is unlikely that this branch can be reached in a real-world scenario, as it would essentially require generating or altering a mnemonic so that it is not a valid UTF-8 string. Because of this, we rated the difficulty as high.

```
String::from_utf8(decrypted_bytes).map_err(|e| {
    TurnkeyError::new(
        Code::InvalidArgument,
        &format!("failed to convert decrypted mnemonic bytes to string: {e:?}"),
    )
})?
```

Figure 61.1: Mnemonic leaks in a debug error message.
(`mono/src/rust/signer/app/src/routes/wallets.rs`)

```
fn main() {
    let value = String::from_utf8(vec![0, 159]);
    println!("{value:?}");
}
// Output of the program above:
Err(FromUtf8Error { bytes: [0, 159], error: Utf8Error { valid_up_to: 1, error_len:
Some(1) } })
```

Figure 61.2: The whole vector content is shown in the `FromUtf8Error` type's debug error message.

Exploit Scenario

A bug in the plaintext mnemonic handling adds an extra byte to the mnemonic, which makes the bytestring not a valid UTF-8 string before the encryption. A Turnkey user performs an import of the mnemonic, which is leaked in the debug error message.

Recommendations

Short term, remove the debug message from the error message returned from the request.

Long term, remove all debug messages returned as part of the responses unless they are explicitly enabled in the build-time configurations for development builds.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Findings

This appendix contains findings that do not have immediate or obvious security implications. However, these findings may facilitate exploit chains targeting other vulnerabilities, become exploitable in future releases, or decrease code readability.

1. **The handler for creating smart contract interfaces has an error message with a typo.** The message should be “smart contract **interface** signature missing” to match the previous error messages.
2. **A malformed UUID will cause a Go panic in `uuid.MustFromString`.** In `src/go/auth-proxy/internal/server/public_api_server.go`, the `proxyID` variable is obtained from the request header `X-Auth-Proxy-Config-Id` and flows into `uuid.MustFromString`. This function panics if the argument cannot be parsed as a UUID. It only crashes the goroutine handling the current request, not the entire server.

```
// Run implements harness.Runner. It should block serving until ctx is Done.
func (p *ProxyServer) Run(ctx context.Context) error {
    grpcServer := grpc.NewServer(

    grpc.UnaryInterceptor(middleware.ExtractAuthProxyConfigHeader(p.log, func(ctx
context.Context, proxyID string) (*models.AuthProxyConfig, error) {
    return p.dbh.AuthProxyConfig().Get(ctx,
auth_proxy_config.WithID(uuid.MustFromString(proxyID)))
    })),
    )
    // [...]
    return http.ListenAndServe(":"+p.port,
        middleware.InjectAuthProxyConfigHeader(p.log, func(ctx
context.Context, proxyID string) (*models.AuthProxyConfig, error) {
    return p.dbh.AuthProxyConfig().Get(ctx,
auth_proxy_config.WithID(uuid.MustFromString(proxyID)))
    }))(
        middleware.WithCORS(p.log, func(ctx context.Context,
proxyID string) (*models.AuthProxyConfig, error) {
    return p.dbh.AuthProxyConfig().Get(ctx,
auth_proxy_config.WithID(uuid.MustFromString(proxyID)))
    }, mux),
    ),
}
```

Figure B.1: Excerpt of `public_api_server.go`

([mono/src/go/auth-proxy/internal/server/public_api_server.go#L67-L93](#))

3. **Signing up with an email address or a phone number but without a verification token causes a panic in the goroutine handling the request.** In `src/go/auth-proxy/internal/server/public_api_server.go`, `*req.VerificationToken` is optional but dereferenced in this code path.

```
func (p *ProxyServer) Signup(ctx context.Context, req
*auth_proxy.SignupRequest) (*auth_proxy.SignupResponse, error) {
    proxyID := ctx.Value(middleware.ProxyIDKey).(string)
    cfg := ctx.Value(middleware.ProxyConfigKey).(*models.AuthProxyConfig)

    if req.UserName == "" || req.OrganizationName == "" {
        return nil, status.Error(codes.InvalidArgument, "user_name and
organization_name are required")
    }

    if req.UserEmail != nil {
        if err := helpers.VerifyVerificationToken(ctx,
*p.otpVerificationPublicKey, *req.VerificationToken, *req.UserEmail); err !=
nil {
            return nil, status.Errorf(codes.InvalidArgument, "invalid
verification_token: %v", err)
        }
    }

    if req.UserPhoneNumber != nil {
        if err := helpers.VerifyVerificationToken(ctx,
*p.otpVerificationPublicKey, *req.VerificationToken, *req.UserPhoneNumber);
err != nil {
            return nil, status.Errorf(codes.InvalidArgument, "invalid
verification_token: %v", err)
        }
    }
}
```

Figure B.2: Excerpt of `public_api_server.go`

([mono/src/go/auth-proxy/internal/server/public_api_server.go#L127-L145](#))

4. In `src/go/auth-proxy/internal/server/public_api_server.go`, **middleware context variables `ProxyIDKey` and `ProxyConfigKey` are type asserted while they can be `nil`.** For instance, sending requests to endpoints like `/v1/signup` without passing an `X-Auth-Proxy-Config-Id` header can make this assertion panic the handler. The middleware must reject requests to these handlers if either the configuration identifier or the proxy configuration is missing.

```
func (p *ProxyServer) Signup(ctx context.Context, req
*auth_proxy.SignupRequest) (*auth_proxy.SignupResponse, error) {
    proxyID := ctx.Value(middleware.ProxyIDKey).(string)
    cfg := ctx.Value(middleware.ProxyConfigKey).(*models.AuthProxyConfig)
```

Figure B.3: Excerpt of `public_api_server.go`
([mono/src/go/auth-proxy/internal/server/public_api_server.go#L127-L129](#))

5. **This comment** in `mono/src/rust/evm_parser/app/src/routes/tron.rs` has a **typo**. The word “coule” should be “could.”
6. **Repetitive length checks of the `raw.contract` array can be error-prone**. Store the contract on the first check and reuse the variable in the subsequent places.
7. **The `TronContract` structure construction has unnecessary code duplication**. Only the contract fields differ between the variants, but the whole structure is constructed in all cases. Factor out the `TronContract` structure construction code from the match cases to remove code duplication and make the function easier to reason about.
8. **The `TURNKEY_EMBEDDED_KEY_ORIGIN` constant is not used in the export-and-sign iframe**. Specifically, an item with this key is never set in `localStorage`.
9. **The export-and-sign iframe calls the `isDoublyIframed` function in the `initEmbeddedKey` function, but the export iframe does not**. Likely the export-and-sign behavior is more secure, but this discrepancy requires further investigation.
10. **The `wallet-export-sign` demo application has a few issues:**
 - a. The `NEXT_PUBLIC_EXPORT_SIGN_IFRAME_URL` variable is missing in the `.env.local.template` file.
 - b. The site must be run over HTTPS, and this requirement is not mentioned in the README. Running the site on an HTTP server makes the demo not function correctly, as the `crypto.subtle` interface is not exposed in the browser.
11. **The result of this `nacl.sign.detached.verify` call is not used**. The verification is done only to check that the newly generated signature was correctly generated, which should always be true. Either remove the call, as it is mostly redundant, or add validation of the returned value.
12. **Add minimal node and npm versions required to build the export-and-sign iframe**. For example, the “engines” field in `package.json` file could be used.
13. **The `private_keys` vector in `create` and `create2` is unnecessarily marked as `mut`**. Change the append method to `extend`, which does not require the mutable argument.

C. Fuzzing Recommendations

This appendix summarizes our recommendations for improving the existing Rust fuzzing setup, as well as simple fuzzers we wrote to validate the behavior of low-level Go code of the Auth Proxy.

Rust Code

Our recommendations for improving the existing Rust fuzzing setup are based on our observations and an informal workshop with Christian Reitter, the developer leading current fuzzing efforts for Turnkey, on October 27, 2025.

We noted that Turnkey benefits from deep in-house knowledge of fuzzing and its associated challenges, as well as a fuzz testing development approach that avoids impacting production code generation. Turnkey's efforts have established robust fuzzing capabilities that can identify new findings without requiring significant changes to the existing setup. We were able to easily iterate on existing harnesses with only minimal tweaks and discover new bugs in enclave code ([TOB-TK3-1](#), [TOB-TK3-8](#)).

Most of the suggestions below were discussed during the informal workshop and are documented here for reference. Recommendations for modifications to existing code and new harnesses have been shared with Turnkey developers during the audit.

Automation in Continuous Integration

We recommend making the fuzz testing suite an automated step in the continuous integration (CI) system. Adding automated fuzzing in CI systems is not straightforward, as existing tools are either proprietary, with harnesses running on third-party SaaS platforms, or tightly coupled to specific stacks, like Google's `ClusterFuzz`. However, the operation of existing fuzzers is currently a manual process that integrally depends on the lead developer.

As part of the initial rollout, the goal should be to ensure testing consistency and to provide common artifacts useful for validating the effectiveness of the harnesses, such as human-readable coverage reports. It should not interfere with the development process of the code under test.

We recommend adding all harnesses to a GitHub Actions workflow to run them in parallel on a regular basis—every day for a fixed period of time with the option `-max_total_time`. A separate weekly workflow could focus on creating global and per-fuzzer coverage reports with `llvm-cov` by running all harnesses on their respective persistent corpus, as well as corpus minimization and cross-harness merge tasks when no fuzzer is running.

Note that GitHub-backed compute and storage are usually more expensive than those of other providers; running this workflow on UbiCloud (cf.

.github/workflows/end-to-end.yml) would be beneficial from both a performance and cost standpoint.

Creating More Fuzzers to Ease the Separation of Concerns

The codebase could benefit from a wider variety of fuzzers with a lower-level focus.

For instance, `policy's 1_validate_evaluate` and `2_validate_evaluate_filter` fuzzers both attempt to simulate a realistic data flow by first parsing the policy, then re-parsing and validating it with both `ConsensusValidator` and `ConditionValidator` before evaluating the policy. As sole harnesses for this crate, they attempt to exercise too many unrelated portions of the codebase simultaneously, resulting in less efficiency. The first parsing will reject all syntactically invalid test cases, preventing them from reaching later stages. They are parsed two more times as part of the validators, incurring a performance hit for the same coverage, and then using the same test case for validators with distinct behavior.

This can be improved by also implementing fuzzers for every concern, in addition to the existing high-level ones. In the case of `policy`, parsing could be tested with a naive fuzzer and a good dictionary, and every implementation of an `Evaluator` can be tested individually by skipping the parsing step altogether (see our recommendations for adopting structured fuzzing below). While this approach may yield abstract syntax trees (ASTs) that could not have been produced by the parser, both features are developed separately, and this will help catch bugs on both sides. Another common option is to consume bytes from the input to exercise different parts of the code (e.g., `0x00` would call `ConditionEvaluator`, `0x01` would call `ConsensusEvaluator`, etc).

This model also simplifies the identification of slow harnesses without relying on profiling traces, as neighbor harnesses are good indicators of the speed one can expect. For instance, while fuzzing `Evaluator` implementations individually, speed should be in the same order of magnitude for all harnesses, and slower ones could indicate issues similar to [TOB-TK3-8](#) without impacting other fuzzers.

The recent additions to `evm_parser` with [PR #4067](#) are a good example of distinct and simple fuzzers targeting one specific concern of the codebase.

Consistent Detection of Timeouts

As stated in the kick-off call, the code running in the enclaves has strong availability requirements. The process of fuzzing typically relies on the unexpected termination of a program as an oracle, such as crashes or operator-implemented assertions.

Slow test cases can be categorized as timeouts if they cross a certain time threshold (by default, 1,200 seconds with `cargo-fuzz's libFuzzer`). Note that in `fork` mode, the `ignore_timeouts` and `ignore_ooms` options are enabled by default and must be toggled off for proper detection of these test cases.

We recommend taking a more consistent approach to timeouts by configuring fuzzers with a more sensible value (e.g., 1 second for policy evaluations) in the automated GitHub Actions and during manual testing (through a `Makefile` or specific scripts).

Use of Coverage Reports

The goal of automated coverage reports should be to identify under-tested code and potential blockers that the engine cannot easily circumvent, such as checksums, invalid logic, or cryptographic code. We already recommended automating coverage reports as part of CI workflows.

Ultimately, adopting tooling like `fuzz-introspector` can help automate these discovery tasks based on many factors, such as their cross-references and complexity; note that Rust support for `fuzz-introspector` is *still under development*.

Extending Use of Arbitrary Crate for Structure-Aware Fuzzing

[PR tkhq/mono#4780](#) extends the use of `Arbitrary` to automatically create structured test cases, helping the harnesses generate better test cases. It usually allows deserialization layers to be skipped—those that are either out of scope or already better fuzzed by another harness—and focuses on the intended feature with syntactically correct test cases. This also makes the minimization process also more efficient.

We recommend continuing the effort and bringing `Arbitrary` data to harnesses that wrap parsing or deserialization code, such as `eip_712`, `eip_7702`, and `policy`. Dedicated harnesses should still cover the parser code, separately from validation and evaluation. For the `policy` component, it can also be used to dynamically change the evaluation context and variables (properties of the transaction, organization, user).

A downside of this approach is that it makes it harder to understand the generated program, as test cases now consist of unstructured bytes transformed by `Arbitrary` instead of, for instance, human-readable policies. All structures deriving from `Arbitrary` must implement `Debug` to benefit from `cargo fuzz fmt`.

While one can easily adapt tools like `GLADE` and `domato` to the Turnkey policy syntax, we expect speed improvements to surpass this approach and yield better results on the `policy` parser. Additionally, evaluators are already achieving improved coverage with the adoption of `Arbitrary`. These generators are better suited for more complex grammars and would bring negligible benefits to `policy` harnesses.

Performance Profiling

Even outside of pathologic test cases, the harnesses can be slower than expected because of the setup and teardown of a resource with every test case. We recommend using [flamegraph-rs/flamegraph](#) to trace the performance of a fuzzer going over a minimized corpus to identify such bottlenecks.

For instance, implementing `Arbitrary` on `policy::ast::nodes::Expr` or performing a static initialization of `policy::gen::rules::ExprParser` once during harness startup would allow the evaluation loop to be exercised without reinitializing `policy::gen::rules::ExprParser` on every test case. We took this approach during our fuzz testing, and it resulted in a 100x speedup on Evaluator-related fuzzers, rediscovering [TOB-TK3-1](#) within seconds from an empty corpus.

Continuing to Automate Dictionary and Corpus Generation

Several fuzzers already have scripts to automate the creation of their dictionaries and/or corpora—for instance, in the `eip_712` harness with `src/rust/eip_712/fuzz/unit_test_extractor.py`, which extracts JSON documents from the crate's unit tests to seed the initial corpus. A script is also available [for webauthn](#), and [grep commands for evm_parser](#). However, as identified in the `policy` crate, some other dictionaries may not always reflect the latest features of the code under test.

We recommend continuing efforts to automate the generation of dictionaries and new test cases for integration into the existing corpora (for instance, in a separate GitHub action). This will ensure that existing dictionary generation scripts are run before every fuzzing loop.

This task does not need to be entirely deterministic, and large language models (LLMs) can be effective tools for extracting such information from code, applying simple mutations, and formatting it. However, this can create additional costs and security challenges depending on the current adoption of AI-assisted tooling by Turnkey. The [public Trail of Bits Testing Handbook](#) and the `cargo-test-fuzz` crate suggest more options.

Use of Runtime Sanitizers

While the workspace forbids the presence of unsafe Rust code, this does not apply to third-party dependencies. On the other hand, running all harnesses with AddressSanitizer (ASan) would typically result in a 2x slowdown, and more performant probabilistic sanitizers are not supported by `cargo-fuzz` out of the box. Sensitive code areas can also benefit from Trail of Bits' `cast_checks` sanitizer to panic on dangerous casts, as well as enabling `overflow-checks` in fuzzing builds. The tool `Miri` can also help identify undefined behavior in code under test. All inputs generated by fuzzing as part of the corpus should be executed one by one against the harness (in normal testing mode, not as a fuzzing run) with `Miri` enabled.

Once the automation is in place, we recommend running all harnesses on their persisted corpus to identify additional issues, without attempting to create new test cases. Publishing these builds to facilitate crash triage can also help in understanding issues like [TOB-TK3-1](#) more easily.

Long-Term Fuzzing Projects

External crates may not be receiving the same amount of testing. Tooling like Trail of Bits' **siderophile** can help detect good interfaces to fuzz in dependencies based on the amount of unsafe code they embed.

Another long-term fuzzing project worth investigating is testing for parsing differentials in code related to transactions. This would involve comparing the output of in-house parsers against those produced by third-party ones to identify inconsistencies in how transactions are interpreted between Turnkey enclaves and chain virtual machines. The primary challenge associated with such a project is that third-party parsers are likely to produce outputs in a format that differs significantly from the one generated by Turnkey parsers. This, in turn, requires new parser-like code to be written to make it possible to compare outputs from the two parsers. This new code is likely to have bugs that will be detected by fuzzing, rather than the actual bugs in Turnkey parsers. Moreover, the fuzzing may find issues in the third-party parser instead of Turnkey code.

These problems indicate that writing fuzzers for parsing differentials is valuable primarily as part of manual code review, rather than as part of a long-term fuzzing campaign. We recommend creating and storing relevant harnesses in the codebase to aid manual reviews. If a third-party parser is available that outputs data in a similar format to Turnkey parsers, it may be valuable for a long-term campaign.

A similar model could be adopted for low-level cryptographic primitives on which transaction signing depends. A fuzzer—or even a simple deterministic bit flipper—could mutate bits of a signed structure and verify that the signature is invalidated. See also the recommendation to “ensure that all serializable fields are signed” from the 2023 report.

Go Code

As part of our manual review of the Auth Proxy, we wrote simple fuzzers to validate the behavior of low-level code.

```
package signer

import (
    "bytes"
    "testing"

    "github.com/btcsuite/btcutil/base58"
)

func Fuzz_decodeBase58Check(f *testing.F) {
    f.Fuzz(func(t *testing.T, data string) {
        _, _ = decodeBase58Check(data)
    })
}

func Fuzz_normalizeBundle(f *testing.F) {
    f.Fuzz(func(t *testing.T, data string) {
        _, _ = normalizeBundle(data)
    })
}

func Fuzz_external_base58_Decode(f *testing.F) {
    f.Fuzz(func(t *testing.T, data string) {
        _ = base58.Decode(data)
    })
}

func Fuzz_external_base58_Encode(f *testing.F) {
    f.Fuzz(func(t *testing.T, data []byte) {
        _ = base58.Encode(data)
    })
}

func Fuzz_external_base58_Encode_then_Decode_equal(f *testing.F) {
    f.Fuzz(func(t *testing.T, data []byte) {
        encoded := base58.Encode(data)
        decoded := base58.Decode(encoded)
        if !bytes.Equal(decoded, data) {
            t.Fatalf("%q != %q", decoded, data)
        }
    })
}
```

Figure C.1: Simple fuzzers for low-level Go code of the Auth Proxy

D. Open-Sourcing of the Signer Enclave

As part of the engagement, we were tasked with reviewing the Signer enclave before Turnkey open-sources it. The Signer enclave depends on many internal crates; however, we focused mainly on the Signer enclave application code. We performed a review focusing on identifying any remaining security issues and on evaluating its code quality and operational aspects. We group our recommendations in the following categories.

Handling of Code Generated from Protocol Buffers Definitions

Protocol Buffers definitions are stored under `mono/proto` and serve as the source of truth for communication between all components of the Turnkey infrastructure, including both public and private ones. The definition files are meant to be compiled to Go, Rust, and TypeScript interfaces that can be imported from the rest of the codebase.

Generated code is committed to the code repository alongside other code changes. For instance, in the introduction of the Auth Proxy in [PR tkhq/mono#4220](#), approximately 70% of the modified lines of code are attributed to compiling Protocol Buffers definitions.

This process makes it harder for other Turnkey developers and future external readers to review code changes and spot manual modifications to generated code (debugging, backdoors, etc.) without an exact copy of the toolchain.

We recommend removing such code from the repository and automatically building it from definitions instead.

Unnecessary Uses of Unwrap and Expect

Calling `unwrap` and `expect` is a last-resort measure to bypass the limitations of Rust's type systems. Reviewers have to be careful and ensure the preconditions are met so that the program does not crash. In many cases, the code can be reorganized to remove the need for using those functions. We collected a few specific examples from the Signer enclave:

- The `expect` call in [signer/app/src/routes/wallets.rs](#) on line 395 can be rewritten as shown in figure D.1. Also, the original `expect` message contains a typo: "conver" instead of "convert."

```
fn create_app_proof(app_proof_payload: AppProofPayload, ephemeral_key:
&P256Pair) -> Result<AppProof, TurnkeyError> {
    let json_str: String = serde_json::to_string(&app_proof_payload)
        .map_err(|e| TurnkeyError::internal(&format!("failed to serialize app
proof payload to json: {e:?}")))?;
    // ephemeral_key.sign() hashes then signs, so we call sign on the raw
json_bytes not the digest
    let signature = ephemeral_key.sign(json_str.as_bytes()).map_err(|e| {
```

```

        TurnkeyError::internal(&format!("failed to sign app proof payload
digest ephemeral key: {e:?}"))
    )?;

    Ok(AppProof {
        scheme: SignatureScheme::EphemeralKeyP256 as i32,
        public_key: qos_hex::encode(&ephemeral_key.public_key().to_bytes()),
        proof_payload: json_str,
        signature: qos_hex::encode(&signature),
    })
}

```

Figure D.1: A rewrite removing the need for expect

- The expect call in `signer/app/src/routes/wallets.rs` on line 744 can be rewritten as shown in figure D.2.

```

let signer_encryption_key = match signer_encryption_keys.as_slice() {
    [key] => key,
    _ => ...
}

```

Figure D.2: A rewrite using slice destructuring, shortening the code and removing the need for expect

- The expect call in `mono/src/rust/signer/app/src/routes/wallets.rs` on line 414 is a byproduct of mandatory option types generated from the Protocol Buffers definitions. We already highlighted this issue in the 2024 review of Turnkey in finding TOB-TK2-30, "Protobuf-generated code has imprecise types." At the time of this review, the prost library still has no option to alter this behavior. We recommend adding an additional type layer and transforming the data as soon as it enters the system to avoid repeated option checks.

Code Duplication

We noticed the following substantial code duplication:

- The encryption_server construction is duplicated across `signer/app/src/routes/private_keys.rs` and `signer/app/src/routes/wallets.rs`. The code should be refactored into a shared function.
- The CreateSubOrganizationIntentV* case handling is duplicated in the `sign` function and the `create_accounts` function in `signer/app/src/routes/wallets.rs`. The common branches can be deduplicated with the pattern shown in figure D.3.

```

IntentEnum::CreateSubOrganizationIntentV4(CreateSubOrganizationIntentV4{wallet

```

```

, ..})
|
IntentEnum::CreateSubOrganizationIntentV5(CreateSubOrganizationIntentV5{wallet
, ..})
|
IntentEnum::CreateSubOrganizationIntentV6(CreateSubOrganizationIntentV6{wallet
, ..})
|
IntentEnum::CreateSubOrganizationIntentV7(CreateSubOrganizationIntentV7{wallet
, ..}) => {
    let wallet_params =
        wallet
        .ok_or_else(|| TurnkeyError::internal("missing wallet params"))?;

```

Figure D.3: A change in pattern matching can deduplicate common branches.

Documentation

We recommend rewriting the README file with the goal of communicating the transparency of the enclave build. Any additional information on nonobvious design decisions is also important. We spotted the following obvious issues in the documentation:

- This [README.md paragraph](#) has a typo: “minimizes” instead of “minimize.”
- The signer [request example](#) does not work; it has the wrong service name and payload.
- The README.md file mentions [here](#) and [here](#) that the Signer enclave supports only secp256k, but [ed25519 is also supported](#).
- The [comment](#) regarding SHA-256 hashes incorrectly documents the hash function as keccak256.

Unfinished Code

Unfinished code does not inspire trust in potential reviewers and users. We recommend addressing all “TODO” comments and places that signal uncertainty in the code flow. Here are some examples:

- A [“TODO” comment](#) for transforming an error to a more appropriate one
- A [“TODO” comment](#) on consolidating two functions with duplicated logic

Verbose Error Messages

We identified [TOB-TK3-61](#) while reviewing the enclave with a focus on verbose error messages. However, this is not the only place where overly verbose error messages are returned from the enclave. We recommend limiting the error messages returned from the enclave to the bare minimum to prevent sensitive information leakage.

Code Releases

Turnkey plans to release the enclave code periodically, and in every release, a subset of the code will be copied over to a public repository. This means that there is a need for an automated process to avoid copying too much information or missing essential files. We recommend exploring already existing solutions such as the [Copybara](#) tool from Google, which was created for this use case.

Reproducibility Concerns

To achieve the desired transparency goal of open-sourcing the enclave, the code should be easy to audit and build along with the dependencies up to the bootstrap seed.

The Turnkey enclave builds are based on [StageX](#), which is still a novel solution, thus requiring additional care when communicating with users.

We recommend the following:

- Document instructions on how to build the enclave, what is important to check, and how to verify reproducibility. In particular, provide clear steps on how a local build can be compared with a live Turnkey deployment and why it holds the guarantees (with links to the relevant sections of the [white paper](#)).
- Any Containerfiles should only pull images that can be reproduced with StageX.
- Any artifacts downloaded in Containerfiles should be checksum-verified.
- Add examples on how to verify StageX images that the enclave build depends on.

E. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From February 17 to February 18, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Turnkey team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

Turnkey provided the following targets for the fix review:

- `tkhq/frames@b4a71d9f52774e7e5df272b6255c4087d9e766a9`
- `tkhq/mono@6cf60d71a2417d370afe52e9dd69ac20806e30cb`
- `tkhq/docs@ac3bbca58f1a2749e87188fb67438aa5503f5ceb`

In summary, of the 59 issues described in this report, Turnkey has resolved 44 issues, has partially resolved one issue, and has not resolved the remaining 14 issues. Of the unresolved issues, nine are of informational severity. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Nested policy expressions can overflow available stack space	Medium	Resolved
2	Parser enclave uses local filesystem	Informational	Resolved
3	Incorrect Organization encoding for signature verification	Medium	Resolved
4	Notarizer does not validate smart_contract_address from user intent versus smart_contract_address from the context	Medium	Resolved
5	Notarizer signs smart contract interfaces from Parser context instead of user intent	Medium	Resolved
6	Weak validation of parsed Solana IDLs against notarized organization data	Medium	Resolved

7	Turnkey accepts ABIs/IDLs with colliding functions	Medium	Resolved
8	ConditionValidator membership operator has exponential time complexity	Medium	Resolved
9	Integer overflow in constrained vector allocation method may lead to panics in the Parser enclave	Medium	Resolved
10	Solana transactions with repeated accounts are accepted	Informational	Unresolved
11	Organization policies may misinterpret WSOL transfers	Medium	Unresolved
12	Weak validation of SmartContractInterfaceType in enclaves	Medium	Resolved
13	Solana dynamic address lookups may hinder policies	Medium	Resolved
14	Solana programs cannot be resolved via dynamic address lookup tables	Informational	Resolved
15	Uploading Solana IDLs with colliding addresses is allowed	Medium	Resolved
16	Solana transaction metadata for policies does not include transfers made by calls to programs	Low	Resolved
17	Solana transaction metadata for policies does not include transfers made by some methods of supported system programs	Medium	Resolved
18	Incomplete mitigation of user existence oracles in coordinator	Informational	Resolved
19	Discrepancy between Parser and Signer transaction parsing	Informational	Unresolved
20	Inconsistent enforcement of AllowedAuthMethods	Medium	Resolved

21	Desynchronization between Ump intents allowlist and Auth Proxy configuration	Medium	Unresolved
22	Incomplete mitigation of user existence oracles via Auth Proxy signup	Informational	Resolved
23	Defined types in IDLs can overflow available stack space	Medium	Resolved
24	Parser enclave bug allows anything to be signed as a Solana IDL	Medium	Resolved
25	ValidateSmartContractInterface does not validate addresses	Informational	Resolved
26	Ethereum int256 type is not supported	Low	Resolved
27	System account Auth Proxy user can be made root user	Undetermined	Resolved
28	Potential race condition while disabling the Auth Proxy feature	Low	Resolved
29	Use of unverified signed tokens	Low	Unresolved
30	Verbose technical error messages	Informational	Resolved
31	ACTIVITY_TYPE_UPDATE_AUTH_PROXY_CONFIG does not restrict Enterprise-only features	Informational	Unresolved
32	Unreachable SQL injection in GetKeyAndWalletCountInRangeWithInterval	Informational	Resolved
33	Lack of network isolation for Auth Proxy pods	Informational	Resolved
34	OIDC ID tokens can be replayed on other organizations because audience claims are not validated	Undetermined	Unresolved

35	Journalized OTP verification tokens can be front run	Medium	Partially Resolved
36	Verification tokens can be replayed outside of signup and login endpoints	Informational	Unresolved
37	Segwit bitcoin transactions vulnerable to fee overpayment attack	Medium	Resolved
38	Missing validations defined in BIP-0174	Informational	Unresolved
39	Supported versions of PSBTs are not validated explicitly and are not documented	Informational	Resolved
40	Lack of test vectors for BIP-0174 in the code	Informational	Unresolved
41	Users can add invalid public keys	Informational	Unresolved
42	Valid Solana IDLs are not accepted due to "types" key requirement	Informational	Resolved
43	Weak validation of Tron transactions	Informational	Unresolved
44	Bitcoin address type is not case insensitive in Policy Engine	Informational	Resolved
45	Risk of mix-up attacks on ciphertext in Auth Proxy	Low	Unresolved
46	Auth Proxy does not validate enclave signatures	Low	Resolved
47	Pre-production environment is incorrectly set up in the export-and-sign iframe	Informational	Resolved
48	The decryptedKey is not cleared from memory after expiration	Low	Resolved
49	The decryptedKey is not removed from DOM after expiration and clear events	Low	Resolved

50	The decryptedKey is always displayed and is not masked on the UI level	Low	Resolved
51	Expiration time of decryptedKey is incorrectly calculated	Informational	Resolved
52	Redundant event listeners in export-and-sign	Informational	Resolved
54	Single client website opened in multiple tabs cannot be used with the export-and-sign iframe	Informational	Unresolved
55	The fromDerSignature function in the export-and-sign iframe is missing validations	Informational	Resolved
56	Possible mitigations for supply chain attacks are not implemented	Informational	Resolved
58	Dependencies of the export-and-sign iframe are not minimal	Informational	Resolved
59	Signer does not validate the number of private keys to be created	Informational	Resolved
60	Signer crashes on empty transaction unsigned_payload	Medium	Resolved
61	Signer leaks malformed decrypted mnemonics	High	Resolved

Detailed Fix Review Results

TOB-TK3-1: Nested policy expressions can overflow available stack space

Resolved in [tkhq/mono#5249](#). The Turnkey team decided to abandon the sandboxing approach to focus on implementing a metering system for policy evaluations. It adds configuration settings on maximum recursion depth, maximum evaluation steps, and maximum elapsed time, and instruments the policy evaluator to enforce these restrictions.

Note that the suggested default maximum recursion depth and maximum evaluation steps are set to 999 999 999; this limit could be too high for legitimate use cases and could still crash appliances depending on their runtime configuration. While the timeout would likely catch recursion issues, [TOB-TK3-1](#) showed that only the evaluator would crash on the auditor's environment after about 35 recursive accesses, while the timeout is verified every 500 steps.

We recommend establishing more conservative limits based on existing customer policies and tests on appliances, which appears to already be tracked in [ENG-3581](#), and adding the auditor's PoC policies to unit tests to validate the implementation of this feature.

TOB-TK3-2: Parser enclave uses local filesystem

Resolved in [tkhq/mono#5076](#). Filesystem-based ABI loading was removed.

TOB-TK3-3: Incorrect Organization encoding for signature verification

Resolved in [tkhq/mono#4868](#). A new organization version (v16.1) that uses the correct schema was added. Migration scripts were updated to support the new version.

TOB-TK3-4: Notarizer does not validate smart_contract_address from user intent versus smart_contract_address from the context

Resolved [tkhq/mono#4870](#). The smart_contract_address fields in intent and context are now validated against each other.

TOB-TK3-5: Notarizer signs smart contract interfaces from Parser context instead of user intent

Resolved [tkhq/mono#4870](#). The digest of smart contract interfaces from the context is now validated against the digest of interface contracts based on data from the intent. Only the smart_contract_interface_id field is not validated, but users do not know that value and cannot provide it in the intent.

TOB-TK3-6: Weak validation of parsed Solana IDLs against notarized organization data

Resolved [tkhq/mono#4894](#). New validation was added to the verify_sol_parsed_tx_idls method, preventing IDLs that are not in notarized data from being accepted. The defense-in-depth recommendations were not implemented.

TOB-TK3-7: Turnkey accepts ABIs/IDLs with colliding functions

Resolved in [tkhq/mono#5101](#). Validation was added, and ABIs/IDLs with colliding functions are now rejected.

TOB-TK3-8: ConditionValidator membership operator has exponential time complexity

Resolved in [tkhq/mono#5249](#). See the fix description for TOB-TK3-1 for reference.

TOB-TK3-9: Integer overflow in constrained vector allocation method may lead to panics in the Parser enclave

Resolved in [tkhq/mono#4885](#). The multiplication now uses the `checked_mul` method.

TOB-TK3-10: Solana transactions with repeated accounts are accepted

Unresolved. Turnkey considers validation of semantic on-chain correctness of transactions that the system signs to be out of the system's scope.

TOB-TK3-11: Organization policies may misinterpret WSOL transfers

Unresolved. Turnkey leaves it to users to understand SOL-WSOL semantics and to write safe policies.

TOB-TK3-12: Weak validation of SmartContractInterfaceType in enclaves

Resolved in [tkhq/mono#4870](#) and [tkhq/mono#5202](#). Mix-up attacks are now prevented in the Notarizer and Parser enclaves. As discussed, the attack is not possible against the Ump enclave, so no fixes are needed there.

TOB-TK3-13: Solana dynamic address lookups may hinder policies

Resolved in [tkhq/docs#504](#). Handling of dynamic address lookups is now documented. No changes to enclaves were made.

TOB-TK3-14: Solana programs cannot be resolved via dynamic address lookup tables

Resolved in [tkhq/docs#532](#). The limitation is now documented.

TOB-TK3-15: Uploading Solana IDLs with colliding addresses is allowed

Resolved in [tkhq/mono#4976](#). The Notarizer now rejects IDLs with known program addresses. The Notarizer was already rejecting IDLs with addresses that exist in an organization's notarized data, but that validation was missed during the audit.

TOB-TK3-16: Solana transaction metadata for policies does not include transfers made by calls to programs

Resolved in [tkhq/docs#480](#). The documentation was updated to mention the known limitations of the Parser.

TOB-TK3-17: Solana transaction metadata for policies does not include transfers made by some methods of supported system programs

Resolved in [tkhq/docs#480](#). The documentation was updated to mention the known limitations of the Parser.

TOB-TK3-18: Incomplete mitigation of user existence oracles in coordinator

Resolved in [tkhq/mono#5152](#) and [tkhq/mono#5162](#). The SendLoginMagicLink handler now sends a new registration email to the user and does not expose this edge case in the API responses. A potential timing attack is left, but is deemed to be unexploitable in realistic conditions.

TOB-TK3-19: Discrepancy between Parser and Signer transaction parsing

Unresolved. The Turnkey team provided the following context for this finding's fix status:

The signer does this and the only reason to have it in the parser would be defense in depth (I think).

TOB-TK3-20: Inconsistent enforcement of AllowedAuthMethods

Resolved in [tkhq/mono#4897](#). Checks were added to Signup, InitOtp, and OtpLogin to verify whether SMS-based or email-based OTPs are enabled. While this addresses the main concern of the original finding, as OTPs are the weakest form of authentication available to Turnkey customers, there are still gaps and it is not possible to prevent the use of passkey and wallet authentication methods. We recommend also adding validation that SMS-based or email-based OTPs are enabled in the VerifyOtp handler.

TOB-TK3-21: Desynchronization between Ump intents allowlist and Auth Proxy configuration

Unresolved. The Turnkey team provided the following context for this finding's fix status:

The auth proxy config is not signed, it is stored outside of the organization data. Passing in the config wouldn't accomplish much from a security perspective, as an attacker could just fake that too. To make any progress here we'd have to add the auth proxy config to the organization data.

The allowed authentication methods at the auth proxy level are just UI toggles essentially: unless they're part of org data we can't consider them security-relevant.

One thing we could do is change the name from "allowed_auth_methods" to "displayed_auth_method", but the effort is not worth it at the moment.

TOB-TK3-22: Incomplete mitigation of user existence oracles via Auth Proxy signup

Resolved in [tkhq/mono#4687](#). A new VerificationToken was introduced that is signed with the new otpVerificationPublicKey and is required for the Signup and GetAccount operations when this setting is toggled on. This is an opt-in setting.

TOB-TK3-23: Defined types in IDLs can overflow available stack space

Resolved in [tkhq/mono#5003](#). Defined types are now checked recursively, and IDL size limits the recursion implicitly.

TOB-TK3-24: Parser enclave bug allows anything to be signed as a Solana IDL

Resolved in [tkhq/mono#4923](#). The missing operator was added.

TOB-TK3-25: ValidateSmartContractInterface does not validate addresses

Resolved in [tkhq/mono#4918](#). The missing validations were added to Ethereum and Solana smart contract interface parsing.

TOB-TK3-26: Ethereum int256 type is not supported

Resolved in [tkhq/docs#532](#). The documentation now notes the known limitation.

TOB-TK3-27: System account Auth Proxy user can be made root user

Resolved in [tkhq/mono#4898](#) and

[tkhq/mono#f656bfb5363d3921dbec6e92f2497cde7da6d79e](#). It is now impossible for organizations to have a user with FeatureName : :AuthProxy in their root quorum. Four organizations were already in this situation and are grandfathered-in to prevent changes these organizations need to make from being blocked. We recommend working with these organizations to ultimately remove the Auth Proxy user from their root quorums, which will allow the grandfathering exception to be removed.

TOB-TK3-28: Potential race condition while disabling the Auth Proxy feature

Resolved in [tkhq/mono#5153](#). The call to remove_feature now requires prior successful removal of the Auth Proxy service account.

TOB-TK3-29: Use of unverified signed tokens

Unresolved. The Turnkey team provided the following context for this finding's fix status:

The logic to verify these tokens would be duplicated from what happens downstream. Given this and "This issue was not proven to result in a bypass of access controls at the Auth Proxy level.", we decided to de-prioritize

TOB-TK3-30: Verbose technical error messages

Resolved in [tkhq/mono#5154](#). When an activity returns an internal error, the Auth Proxy now intercepts it and shows a nontechnical error message instead (e.g., "Turnkey Public API error: invalid request").

TOB-TK3-31: ACTIVITY_TYPE_UPDATE_AUTH_PROXY_CONFIG does not restrict Enterprise-only features

Unresolved. The Turnkey team provided the following context for this finding's fix status:

The only enterprise-only feature is SMS, and we check that server side here when a request to send an sms message comes through the activity pipeline.

Since we don't allow this client side, someone would have to work hard to break this and even then they still wouldn't be able to send sms messages, they'd just have a broken wallet kit config in the ui. Would still be nice to fix, but de-prioritized for now.

TOB-TK3-32: Unreachable SQL injection in GetKeyAndWalletCountInRangeWithInterval

Resolved in [tkhq/mono#4899](#). The affected code was removed.

TOB-TK3-33: Lack of network isolation for Auth Proxy pods

Resolved in [tkhq/gitops#3425](#). The auditors did not have direct access to the repository for the fix review, but the Turnkey team shared the associated commit instead. Auth Proxy pods now have minimal Kubernetes Network Policies both on ingress and egress:

- TCP/8090 from namespace traefik, pod traefik
- TCP/8092 from namespace grafana-agent, component metrics of grafana-agent
- UDP/53 to namespace kube-system, pod kube-dns
- TCP/8081 to namespace tkhq-coordinator, component gateway of tkhq-coordinator
- TCP/5432 to namespace tkhq/tkdb, pod pgbouncer

TOB-TK3-34: OIDC ID tokens can be replayed on other organizations because audience claims are not validated

Unresolved. The Turnkey team provided the following context for this finding's fix status:

De-prioritized, we couldn't find anything malicious that could be done with the captured oidc token given the nonce claim.

TOB-TK3-35: Journalized OTP verification tokens can be front run

Partially resolved in [tkhq/mono#4938](#). The backend accepts a public key during the OTP verification step to bind the verification token to a specific one, and the pull request adds code to verify a signature from the associated private key during signup and login. This effectively prevents attacks from finishing any of these flows on behalf of their target. This change cannot be enforced yet as it would break compatibility with previous SDKs, but the Turnkey team will attempt to ship it in the coming months.

TOB-TK3-36: Verification tokens can be replayed outside of signup and login endpoints

Unresolved. The Turnkey team provided the following context for this finding's fix status:

De-prioritized because we do want this to be reusable for other endpoints - specifically GetAccount.

TOB-TK3-37: Segwit bitcoin transactions vulnerable to fee overpayment attack

Resolved in [tkhq/mono#5135](#) and [tkhq/mono#5410](#). Non-witness UTXOs are now required for witness transactions (except for taproot transactions).

TOB-TK3-38: Missing validations defined in BIP-0174

Unresolved. The Turnkey team provided the following context for this finding's fix status:

De-prioritized, but left as an onboarding ticket for later, given that there isn't a direct security threat - just possibly incorrect behavior.

TOB-TK3-39: Supported versions of PSBTs are not validated explicitly and are not documented

Resolved in [tkhq/mono#5094](#). PSBTs versions are now explicitly validated.

TOB-TK3-40: Lack of test vectors for BIP-0174 in the code

Unresolved. The Turnkey team provided the following context for this finding's fix status:

De-prioritized since it's not a security issue and behavior is correct, we're just missing tests. But left as an onboarding ticket for new engineers.

TOB-TK3-41: Users can add invalid public keys

Unresolved. The Turnkey team provided the following context for this finding's fix status:

Some of our customers are, unfortunately, relying on this incorrect behavior.

TOB-TK3-42: Valid Solana IDLs are not accepted due to "types" key requirement

Resolved in [tkhq/docs#532](#). The behavior is now documented.

TOB-TK3-43: Weak validation of Tron transactions

Unresolved. The Turnkey team provided the following context for this finding's fix status:

De-prioritized since this isn't an explicit attack vector and it's a low usage component for us.

TOB-TK3-44: Bitcoin address type is not case insensitive in Policy Engine

Resolved in [tkhq/mono#5090](#). The address is now compared in a case-insensitive manner.

TOB-TK3-45: Risk of mix-up attacks on ciphertext in Auth Proxy

Unresolved. The Turnkey team provided the following context for this finding's fix status:

This is a low severity finding that requires the auth proxy to be under attacker control. Further, the enclaves will likely prevent this attack despite the compromised auth proxy. The engineering lift required to address this is not justified by the marginal security gain.

TOB-TK3-46: Auth Proxy does not validate enclave signatures

Resolved in [tkhq/docs#537](#). The Turnkey team accepts the risk and considers that downstream consumers should verify App Proofs. This assumption is now documented as part of public security notes for the Auth Proxy component.

TOB-TK3-47: Pre-production environment is incorrectly set up in the export-and-sign iframe

Resolved in [tkhq/frames#96](#) and [tkhq/frames#113](#). The key is now correctly selected based on the environment.

TOB-TK3-48: The decryptedKey is not cleared from memory after expiration

Resolved in [tkhq/frames#98](#). Key clearing code was added.

TOB-TK3-49: The decryptedKey is not removed from DOM after expiration and clear events

Resolved in [tkhq/frames#97](#). The displayKey function was removed from the code.

TOB-TK3-50: The decryptedKey is always displayed and is not masked on the UI level

Resolved in [tkhq/frames#97](#). The displayKey function was removed from the code.

TOB-TK3-51: Expiration time of decryptedKey is incorrectly calculated

Resolved in [tkhq/frames#97](#). The incorrect calculation was fixed.

TOB-TK3-52: Redundant event listeners in export-and-sign

Resolved in [tkhq/frames#105](#). The redundant event listener was removed.

TOB-TK3-54: Single client website opened in multiple tabs cannot be used with the export-and-sign iframe

Unresolved. The Turnkey team provided the following context for this finding's fix status:

Currently, the fact that we're using local storage (which is shared between tabs) helps usability, as the same embedded key can be used to decrypt multiple export bundles. If we used session storage (prevents sharing between tabs), yes this would benefit security (prevent key re-use), but at the cost of hampered devex: developers would have to perform a fresh export per tab (aka per embedded key). The reality is customers would like to do something like: on initial page load, batch call Turnkey's Export activity (whether it's for private key or wallet account), persist the resulting export bundles in memory, and then inject when used for signing.

TOB-TK3-55: The fromDerSignature function in export-and-sign frame is missing validations

Resolved in [tkhq/frames#102](#). The implementation of fromDerSignature was swapped with a more robust one from @turnkey/crypto.

TOB-TK3-56: Possible mitigations for supply chain attacks are not implemented

Resolved in [tkhq/mono#106](#). The ignore-script and save-exact mitigations were enabled. Lifecycle scripts have been disabled. A check for provenance attestation was added.

TOB-TK3-58: Dependencies of the export-and-sign iframe are not minimal

Resolved in [tkhq/frames#104](#). The tweetnacl and tweetnacl-util dependencies were removed.

TOB-TK3-59: Signer does not validate the number of private keys to be created

Resolved in [tkhq/mono#5031](#). Validation was added to check for empty key_params and to check the key_params upper bounds. Unit tests were also added.

TOB-TK3-60: Signer crashes on empty transaction unsigned_payload

Resolved in [tkhq/mono#5053](#). An emptiness check was added for the payload along with a unit test.

TOB-TK3-61: Signer leaks malformed decrypted mnemonics

Resolved in [tkhq/mono#4970](#). The debug message was removed from the error message.

F. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Turnkey under the terms of the project statement of work and has been made public at Turnkey's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.